UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Campus de São José do Rio Preto

André Furlan

Autenticação Biométrica de Locutores Drasticamente Disfônicos Aprimorada pela *Imagined Speech*

São José do Rio Preto
2024

André Furlan

Autenticação Biométrica de Locutores Drasticamente Disfônicos Aprimorada pela *Imagined Speech*

Qualificação apresentada como parte dos requisitos para obtenção do título de Doutor em Ciência da Computação, junto ao Programa de Pós-Graduação em Ciência da Computação, do Instituto de Biociências, Letras e Ciências Exatas da Universidade Estadual Paulista 'Júlio de Mesquita Filho", Campus de São José do Rio Preto.

Orientador: Prof. Dr. Rodrigo Capobianco Guido

São José do Rio Preto

2024

ficha catalografica

André Furlan

Autenticação Biométrica de Locutores Drasticamente Disfônicos Aprimorada pela *Imagined Speech*

Qualificação apresentada como parte dos requisitos para obtenção do título de Doutor em Ciência da Computação, junto ao Programa de Pós-Graduação em Ciência da Computação, do Instituto de Biociências, Letras e Ciências Exatas da Universidade Estadual Paulista 'Júlio de Mesquita Filho", Campus de São José do Rio Preto.

Comissão Examinadora

Prof. Dr. Rodrigo Capobianco Guido
UNESP - Câmpus de São José do Rio Preto
Orientador

Profª Drª Liliane Santana Oliveira Kashiwabara
Universidade Positivo - Campus Londrina

Prof. Dr. Everthon Silva Fonseca
Instituto Federal de São Paulo - Câmpus São José do Rio Preto

São José do Rio Preto
16 de Agosto de 2024

Haha…

Agradecimentos

Agradece

"Citação"

-O Autor.

Resumo

A autenticação biométrica baseada em fala é comprometida quando o locutor apresenta disfonia severa, condição que altera substancialmente as características acústicas da voz. Este trabalho propõe complementar os sinais de voz fonada com sinais de eletroencefalograma (EEG) capturados durante a fala imaginada, fenômeno em que o indivíduo articula mentalmente sem emitir som, ativando regiões cerebrais análogas às da fala fonada. Duas estratégias de extração de características são comparadas: a manual, baseada na Transformada *Wavelet-Packet* de Tempo Discreto (DTWPT) combinada com a Engenharia Paraconsistente de Características; e a automatizada, por meio de *autoencoders* profundos. As características extraídas são classificadas por Redes Neurais de Pulso (RNP) em uma arquitetura residual, motivadas pela eficiência energética e pela capacidade nativa de processar sequências temporais. A abordagem é avaliada sobre a base pública de EEG para fala imaginada (CORETTO; GAREIS; RUFINER, 2017), sob protocolo de verificação — em que os locutores de teste não aparecem no treino — tendo como métricas primárias a Taxa de Erro Igual (EER) e a área sob a curva ROC (AUC); o Erro Médio Quadrático é usado apenas na reconstrução dos *autoencoders*. A seleção do extrator de características é feita pela engenharia paraconsistente, sem treinar classificador, e apenas a combinação vencedora alimenta a etapa de autenticação. Os resultados obtidos mostram que a extração manual supera as variantes baseadas em *autoencoder* segundo o critério paraconsistente, e que o desempenho de autenticação permanece próximo do nível de acaso (melhor EER de 0,4534), indicando que a fusão de voz e EEG, nas condições avaliadas, ainda não sustenta uma autenticação prática.

Abstract

Speech-based biometric authentication degrades severely when the speaker presents pronounced dysphonia, as this condition substantially alters the acoustic characteristics of the voice. This work proposes to complement phonated speech signals with electroencephalogram (EEG) signals captured during imagined speech — a phenomenon in which an individual mentally articulates without producing sound, activating brain regions analogous to those involved in phonated speech. Two feature-extraction strategies are compared: a manual approach based on the Discrete Time Wavelet Packet Transform (DTWPT) combined with Paraconsistent Feature Engineering; and an automated approach using deep autoencoders. The extracted features are classified by Spiking Neural Networks (SNNs) in a residual architecture, motivated by their energy efficiency and inherent ability to process temporal sequences. The approach is evaluated on the public EEG imagined-speech database (CORETTO; GAREIS; RUFINER, 2017) under a verification protocol — in which test speakers do not appear in training — with Equal Error Rate (EER) and area under the ROC curve (AUC) as primary metrics; Mean Squared Error is used only for autoencoder reconstruction. Feature-extractor selection is performed by Paraconsistent Feature Engineering, without training any classifier, and only the winning combination feeds the authentication stage. The obtained results show that handcrafted extraction outperforms the autoencoder variants under the paraconsistent criterion, and that authentication performance remains close to chance level (best EER of 0.4534), indicating that the fusion of voice and EEG, under the evaluated conditions, does not yet support practical authentication.

Lista de ilustrações

Figura 1 – Sub-amostragem 21
Figura 2 – Cálculo de vetores de características com bandas lineares 23
Figura 3 – Cálculo de vetores de características com bandas Mel 24
Figura 4 – Cálculo de vetores de características com bandas Bark 25
Figura 5 – Cálculo de vetores de características com LFCC 26
Figura 6 – Cálculo de vetores de características com MFCC 26
Figura 7 – Cálculo de vetores de características com BFCC 27
Figura 8 – Platôs maximamente planos Daubechies 30
Figura 9 – Platôs maximamente planos outros filtros 31
Figura 10 – Cálculo do coeficiente α. 36
Figura 11 – Cálculo de β 37
Figura 12 – O plano paraconsistente 38
Figura 13 – Sistema 10-20 e lobos cerebrais 43
Figura 14 – Lobos Frontal, Parietal, Occipital e Temporal 44
Figura 15 – Lobo frontal 45
Figura 16 – Lobo parietal 45
Figura 17 – Lobo temporal 46
Figura 18 – Visão geral do telencéfalo 47
Figura 19 – Afasias e seus sintomas 48
Figura 20 – Fluxograma de ajuste/aplicação da normalização 51
Figura 21 – Pulsos de um sinal ruidoso. 52
Figura 22 – Modelo RC 53
Figura 23 – Dispersão em Redes Neurais de Pulsos. 54
Figura 24 – Atividade dispersa de uma RNP 54
Figura 25 – Modelo RC para correntes 55
Figura 26 – Decaimento do potencial da membrana. 57
Figura 27 – Aumento do potencial da membrana. 59
Figura 28 – Gráfico do LIF 60
Figura 29 – Retropropagação Através do Tempo (BPTT) 63
Figura 30 – Fluxograma do algoritmo BPTT 64
Figura 31 – Normalização em Lote Dependente do Limiar (tdBN) 67
Figura 32 – Distribuição dos pesos na inicialização de He/Kaiming 70
Figura 33 – Fluxograma da inicialização de He/Kaiming 71
Figura 34 – Representação funcional de um autoencoder 74
Figura 35 – Exemplo esquemático de um autoencoder sub-completo 74
Figura 36 – Exemplo esquemático de um autoencoder supra-completo 75

Figura 37 – Representação funcional de um *denoising autoencoder* 76
Figura 38 – Bloco residual 79
Figura 39 – Comparação de redes profundas não residuais 80
Figura 40 – Obtenção do resíduo 81
Figura 41 – Organização da base de sinais 91
Figura 42 – Estratégia proposta 94
Figura 43 – Pipeline em duas fases 94
Figura 44 – Fluxograma da Fase 00 99
Figura 45 – Fluxograma da Fase 01 103
Figura 46 – Demonstração numérica das transformadas Wavelet 131

Lista de tabelas

Tabela 1 – Particionamento espectral da DTWPT por sinal 28
Tabela 2 – Valores das escalas nos extremos e agrupamento resultante 29
Tabela 3 – Algumas das *wavelets* mais usadas e suas propriedades 31
Tabela 4 – Exemplo numérico da transformação *wavelet* 34
Tabela 5 – Exemplo numérico de *wavelet-packet* - porção das baixas frequências . 34
Tabela 6 – Exemplo numérico de *wavelet-packet* - porção das altas frequências . . 34
Tabela 7 – Tarefas cerebrais e suas regiões correspondentes 43
Tabela 8 – Configurações de autoencoder (EEG) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor) 106
Tabela 9 – Configurações de autoencoder (voz) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor) 106
Tabela 10 – Configurações de características manuais (EEG) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor); † = vencedor do sinal 107
Tabela 11 – Configurações de características manuais (voz) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor); † = vencedor do sinal 108
Tabela 12 – Configurações da Fase 01 (autenticação DSNN) ordenadas por EER médio 112

Lista de abreviaturas e siglas

ANN *Artificial Neural Network* (Rede Neural Artificial) 92

AUC Área sob a Curva (*Area Under the Curve*) 100

BFCC Coeficientes Cepstrais em Frequência Bark (*Bark-Frequency Cepstral Coefficients*) 18

BN *Batch Normalization* (Normalização em Lote) 65

BPTT Retropropagação Através do Tempo (*Backpropagation Through Time*) 15

CV Validação Cruzada 103

DCT Transformada Discreta de Cosseno 18

DFT Transformada Discreta de Fourier 25

DSNN *Deep Spiking Neural Network* (Rede Neural de Pulso Profunda) 19

DTWPT Transformada *Wavelet-Packet* em Tempo Discreto 27

DTWT Transformada Wavelet em Tempo Discreto 83

EEG Eletroencefalograma 17

EER Taxa de Erro Igual 100

GRU *Gated Recurrent Unit* (Unidade Recorrente com Portas) 93

JSON *JavaScript Object Notation* 103

LFCC Coeficientes Cepstrais em Frequência Linear (*Linear-Frequency Cepstral Coefficients*) 18

LIF *Leaky Integrate and Fire* (Integra-e-Dispara com Vazamento) 53

LSTM *Long Short-Term Memory* (Memória de Longo e Curto Prazo) 93

MFCC Coeficientes Cepstrais em Frequência Mel (*Mel-Frequency Cepstral Coefficients*) 18

PCA Análise de Componentes Principais (*Principal Component Analysis*) . . 23

RC Resistor-Capacitor 53

RNP Redes Neurais de Pulso 52

ROC Característica de Operação do Receptor (*Receiver Operating Characteristic*) 100

SNN *Spiking Neural Network* (Rede Neural de Pulso) 92

STDP Plasticidade Dependente do Tempo de Pulsos (*Spike-Timing-Dependent Plasticity*) 61

tdBN *Threshold-Dependent Batch Normalization* (Normalização em Lote Dependente do Limiar) 65

Sumário

1 INTRODUÇÃO 17
1.1 Considerações Iniciais e Objetivos 17
1.1.1 Objetivos 17
1.2 Estrutura do trabalho 19

2 REVISÃO BIBLIOGRÁFICA 20
2.1 Conceitos utilizados 20
2.1.1 Sinais digitais e sub-amostragem (*downsampling*) 20
2.1.2 Caracterização dos processos de produção da voz humana 21
2.1.2.1 Sinais vozeados *versus* não-vozeados 21
2.1.2.2 Frequência fundamental da voz 21
2.1.2.3 Formantes 22
2.1.3 Escalas e energias dos sinais 22
2.1.3.1 Pré-ênfase do sinal de voz 22
2.1.3.2 Energia de um sinal 23
2.1.3.3 Energias em bandas lineares 23
2.1.3.4 Energias em bandas Mel 24
2.1.3.5 Energias em bandas Bark 24
2.1.3.6 Coeficientes LFCC 24
2.1.3.7 Coeficientes MFCC 25
2.1.3.8 Coeficientes BFCC 27
2.1.3.9 Aplicabilidade das escalas Bark e Mel ao EEG 27
2.1.4 Filtros digitais *wavelet* 30
2.1.4.1 O algoritmo de Mallat para a Transformada *Wavelet* 31
2.1.4.2 O algoritmo de Mallat e a Transformada *Wavelet-Packet* 33
2.1.5 Engenharia Paraconsistente de características 35
2.1.5.1 Vulnerabilidade de $D_{1,0}$ isolado e métrica penalizada por contradição 39
2.1.6 Fala imaginada 41
2.1.7 Interfaces Humano-Máquina e EEG 42
2.1.8 Sistema 10-20 e as áreas do cérebro 43
2.1.9 Normalização de Características de Entrada 48
2.1.9.1 Normalização por característica (áudio) 49
2.1.9.2 Normalização por janela (EEG) 49
2.1.9.3 Evitando vazamento de dados 50
2.1.9.4 Exemplo numérico 51

2.1.10 Redes Neurais de Pulso (Spiking Neural Networks) . . . 51
2.1.10.1 Neurônio de Pulso . . . 53
2.1.10.2 Entendendo e modelando o LIF . . . 53
2.1.10.3 Outra interpretação do LIF . . . 60
2.1.10.4 Treinamento . . . 61
2.1.10.5 BPTT[1] em RNPs . . . 62
2.1.10.6 Gradiente substituto . . . 64
2.1.10.7 Normalização em Lote Dependente do Limiar (tdBN) . . . 65
2.1.10.8 Regularização da taxa de disparo . . . 67
2.1.10.9 Inicialização de Pesos . . . 69
2.1.10.10 Taxa de aprendizado por grupo de parâmetros . . . 71
2.1.11 *Autoencoders* . . . 73
2.1.11.1 Autoencoders sub-completos ou clássicos . . . 74
2.1.11.2 Autoencoders supra-completos . . . 75
2.1.11.3 *Autoencoders* regularizados . . . 75
2.1.11.4 *Denoising autoencoders* . . . 76
2.1.11.5 Colapso do latente no autoencoder pulsante e regularização da taxa de disparo . 77
2.1.12 Redes neurais residuais (ResNets) . . . 79
2.1.12.1 Aprendizado residual . . . 80
2.2 Trabalhos mais recentes . . . 82
2.2.1 Protocolo de coleta . . . 82
2.2.1.1 Critérios de inclusão e exclusão . . . 82
2.2.1.2 Base de dados e palavras-chaves . . . 82

3 ABORDAGEM PROPOSTA . . . 90
3.1 A Base de sinais . . . 90
3.2 Estrutura da estratégia proposta . . . 91
3.2.1 Protocolo experimental em duas fases . . . 93
3.2.1.1 Fase 00 — construção e seleção do vetor de características . . . 94
3.2.1.1.1 Codificação temporal e função de perda do treinamento. . . . 96
3.2.1.1.2 Comparação entre as codificações temporais. . . . 96
3.2.1.2 Fase 01 — autenticação com o vetor vencedor . . . 99
3.2.1.2.1 Arquitetura do classificador DSNN. . . . 99
3.2.1.2.2 Protocolo de verificação: como EER e AUC são calculados. . . . 100
3.2.1.2.3 Os quatro eixos experimentais. . . . 101
3.2.1.2.4 Protocolo de treino e de repetição. . . . 101
3.2.1.3 Organização dos perfis e transição entre fases . . . 103
3.2.2 Métricas . . . 103

[1] Retropropagação Através do Tempo (*Backpropagation Through Time*)

3.2.3 Desenvolvimento e Validação de Algoritmos 104

4 TESTES E RESULTADOS 105
4.1 Fase 00 — Ranking por grau de verdade paraconsistente 105
4.2 Fase 01 — Autenticação DSNN 112
4.3 Retomada das questões de pesquisa 113

5 CONCLUSÕES E TRABALHOS FUTUROS 117
5.1 Síntese dos resultados 117
5.2 Contribuições 117
5.3 Limitações 118
5.4 Trabalhos futuros 119

REFERÊNCIAS 120

APÊNDICE A – REGULARIZAÇÃO EM REDES NEURAIS . . . 127
A.1 Decaimento de Peso (Regularização L2) 127
A.2 Penalidade de Esparsidade 128
A.3 Regularização da Taxa de Disparo em Redes Neurais de Pulso . . 128

APÊNDICE B – APLICAÇÃO DE UM FILTRO *WAVELET* . . . 131

1 Introdução

1.1 Considerações Iniciais e Objetivos

Os experimentos realizados neste trabalho utilizam a base pública de EEG[1] para fala imaginada disponibilizada em (CORETTO; GAREIS; RUFINER, 2017), composta por registros de 15 voluntários falantes de espanhol que imaginaram vogais e comandos direcionais. Tal base permite a avaliação e a comparação das técnicas propostas com estudos anteriores.

A autenticação baseada em biometria vem despertando um interesse crescente, uma vez que se aproveita de características biométricas altamente correlacionadas com o indivíduo, resultando em um nível mais elevado de segurança. Algumas biometrias mostraram-se discriminatórias em relação a certos grupos, como indivíduos com pele mais escura (GROTHER; NGAN; HANAOKA, 2019), ou aqueles com deficiências físicas, que podem enfrentar dificuldades ao utilizá-las. Esse é o caso de boa parte dos tipos de biometrias utilizadas, incluindo a fala, que é um dos focos desse estudo. Por outro lado, o EEG evita tais problemas, ao mesmo tempo em que acrescenta algumas características únicas, como a exploração das variações neurais do usuário, criando, assim, uma contramedida às pressões externas e complementando eventuais falhas ou faltas de outros métodos.

1.1.1 Objetivos

Comparar o desempenho das estratégias de *feature learning* baseadas em *autoencoders* com técnicas de análise manuais como as *Wavelet-Packet* de Tempo Discreto e agrupamentos energéticos usando a engenharia paraconsistente de características.

O problema-alvo deste projeto é conceber algoritmos biométricos para autenticar, em princípio por meio da fala, indivíduos com locuções severamente degradadas complementando tais informações com aquelas provenientes dos sinais cerebrais extraídos durante a fonação (*imagined speech*).

Para guiar a avaliação dos métodos de extração de características, este trabalho investiga algumas questões:

Questão 1 — Agrupamento espectral: o esquema de particionamento espectral influencia a qualidade da representação dos sinais? Para responder a essa questão, comparam-se três representações de Categoria 1 — *Linear-band energies*, *Mel-band energies* e *Bark-band energies* — que diferem segundo o critério de agrupamento

[1] Eletroencefalograma

das bandas de frequência sobre a representação wavelet. A comparação é conduzida apenas para o sinal de **voz**: as escalas Mel e Bark são cocleares, modelam a resolução em frequência da audição humana e, aplicadas ao EEG, degeneram no mesmo particionamento da escala linear (subseção 2.1.3.9).

Questão 2 — Transformação cepstral: a aplicação do logaritmo e da DCT[2] sobre as energias por banda melhora a representação dos sinais? Para responder a essa questão, cada representação de Categoria 1 é comparada com seu correspondente cepstral de Categoria 2: *Linear-band energies* versus LFCC[3], *Mel-band energies* versus MFCC[4] e *Bark-band energies* versus BFCC[5]. Em todos os pares, a única diferença entre as representações é a presença ou ausência da etapa cepstral.

Questão 3 — Autoencoders: os autoencoders conseguem gerar representações mais discriminativas do que as técnicas manuais? Para responder a essa questão, as representações de Categoria 1 e Categoria 2 são comparadas com as representações de Categoria 3, geradas por autoencoders. A comparação é feita na Fase 00 (subseção 3.2.1) pelo critério paraconsistente $D_{\text{penalizado}}$, que mede a separabilidade das classes *antes* de qualquer classificador e, portanto, avalia a representação em si, sem confundi-la com a capacidade do classificador a jusante. Como consequência dessa escolha de desenho, as três categorias não são confrontadas através de um classificador comum — limitação retomada na seção 4.3.

[PENDENTE] *Esta questão ainda não está respondida. O experimento executado a responde apenas pelo critério paraconsistente da Fase 00; a comparação entre as três categorias através de um classificador comum — que é a forma como a questão foi originalmente formulada — ainda precisa ser executada.*

Questão 4 — Dependência de texto: o desempenho dos métodos de extração de características é afetado pela dependência ou independência do texto? Para responder a essa questão, o desempenho dos métodos de extração de características é comparado entre os cenários text-dependent e text-independent. O cenário text-dependent é aquele em que a mesma frase é falada e imaginada, enquanto o cenário text-independent é aquele em que qualquer frase pode ser falada ou imaginada, tanto no treinamento quanto nos testes.

---

[2] Transformada Discreta de Cosseno
[3] Coeficientes Cepstrais em Frequência Linear (*Linear-Frequency Cepstral Coefficients*)
[4] Coeficientes Cepstrais em Frequência Mel (*Mel-Frequency Cepstral Coefficients*)
[5] Coeficientes Cepstrais em Frequência Bark (*Bark-Frequency Cepstral Coefficients*)

1.2 Estrutura do trabalho

O Capítulo 2 reúne a fundamentação: a definição dos conceitos utilizados está na seção 2.1 — incluindo a engenharia paraconsistente de características e a métrica $D_{\text{penalizado}}$ adotada como critério de seleção (subseção 2.1.5.1) — e uma revisão do estado-da-arte, na seção 2.2.

O Capítulo 3 define a abordagem proposta: a organização da base de sinais (seção 3.1), a estrutura da estratégia (seção 3.2) e, dentro dela, o protocolo experimental em duas fases (subseção 3.2.1) — a Fase 00, que constrói e seleciona o vetor de características, e a Fase 01 (subseção 3.2.1.2), que treina o classificador de autenticação e onde estão descritos a arquitetura da DSNN[6], o protocolo de verificação e os quatro eixos experimentais. O capítulo encerra com as métricas adotadas e o cronograma (**??**).

O Capítulo 4 apresenta os resultados das duas fases e retoma, à luz deles, as quatro questões formuladas acima (seção 4.3). O Capítulo 5 discute as conclusões e os trabalhos futuros. Os apêndices detalham as técnicas de regularização empregadas (Apêndice A).

[6] *Deep Spiking Neural Network* (Rede Neural de Pulso Profunda)

2 Revisão Bibliográfica

Para desenvolver o trabalho, é necessário definir os conceitos de *downsampling*, filtros digitais *wavelets* e energia, que serão usados para a extração de características dos sinais. No processamento dos sinais de voz, serão revisadas as frequências fundamentais da voz, além das escalas BARK, MEL e Lineares. Também será revisada a engenharia paraconsistente de características, método utilizado para verificar a qualidade das características extraídas. Em seguida, serão estudadas as bases das interfaces Humano-Máquina para eletroencefalograma, que coletaram os dados das regiões do cérebro assim como as próprias regiões cerebrais. Concluída essa etapa, serão revisadas as redes neurais de pulso, autoencoders e redes neurais residuais.

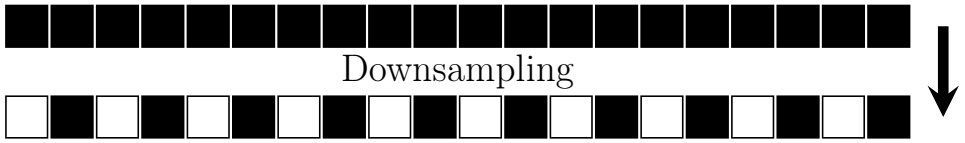
2.1 Conceitos utilizados

2.1.1 Sinais digitais e sub-amostragem (*downsampling*)

Sinais digitais, tanto de voz quanto de EEG, isto é, aqueles que estão amostrados e quantizados (HAYKIN; MOHER, 2011), constituem a base deste trabalho. Além do processo de digitalização, inerente ao ato de armazenar sinais em computadores, os mesmos podem sofrer, a depender da necessidade ou possibilidade, sub-amostragens ou *downsamplings* (POLIKAR *et al.*, 1996). Isso implica em uma estratégia de redução da dimensão e, ocorre após a conversão de domínio dos sinais com base em filtros digitais do tipo *wavelet*, a serem apresentados adiante. A Figura 1, na qual as partes pretas contêm dados e as brancas representam os elementos removidos, ilustra como o sinal é reamostrado. Tendo em vista que este trabalho está baseado em sinais digitais de voz e EEG com base em *wavelets*, o processo de sub-amostragem é parte integrante no processamento dos dados.

O fundamento teórico que legitima a sub-amostragem é o Teorema de Nyquist-Shannon (HAYKIN; MOHER, 2011), segundo o qual um sinal pode ser reconstruído sem perdas desde que a taxa de amostragem seja ao menos o dobro da frequência máxima presente no sinal — denominada frequência de Nyquist. Ao se reduzir à metade a taxa amostral sem essa precaução, componentes de frequência acima do novo limite de Nyquist se sobrepõem às frequências válidas, fenômeno conhecido como *aliasing*, corrompendo irreversivelmente o sinal. Neste trabalho, a sub-amostragem ocorre dentro da decomposição em *wavelets*: os filtros passa-baixa e passa-alta de cada nível limitam a banda do sinal antes da redução amostral, atuando como filtros anti-*aliasing* e garantindo que o critério de Nyquist seja respeitado em cada estágio da decomposição.

Figura 1 – Sub-amostragem



Fonte: Elaborado pelo autor, 2022.

2.1.2 Caracterização dos processos de produção da voz humana

A fala possui três grandes áreas de estudo: A fisiológica, também conhecida como fonética articulatória, a acústica, referida como fonética acústica, e ainda, a perceptual, que cuida da percepção da fala (KREMER; GOMES, 2014). Neste trabalho, o foco será apenas na questão acústica, pois não serão analisados aspectos da fisiologia relacionada à voz, mas sim os sinais sonoros propriamente ditos.

2.1.2.1 Sinais vozeados *versus* não-vozeados

Quando da análise dos sinais de voz, consideram-se as partes vozeadas e não-vozeadas. Aquelas são produzidas com a ajuda da vibração quase periódica das pregas vocais, enquanto estas praticamente não contam com participação da referida estrutura.

2.1.2.2 Frequência fundamental da voz

Também conhecida como F_0, é o componente periódico resultante da vibração das pregas vocais. Em termos de percepção, se pode interpretar F_0 como o tom da voz, isto é, o *pitch* (KREMER; GOMES, 2014). Vozes agudas têm uma frequência de *pitch* alta, enquanto vozes mais graves têm baixa. A alteração da frequência (jitter) e/ou intensidade (shimmer) do *pitch* durante a fala é definida como entonação, porém, também pode indicar algum distúrbio ou doença relacionada ao trato vocal (WERTZNER; SCHREIBER; AMARO, 2005).

A frequência fundamental da voz é o número de vezes na qual uma forma de onda característica, que reflete a excitação pulmonar moldada pelas pregas vocais, se repete por unidade de tempo. Sendo assim, as medidas de F_0 são geralmente apresentadas em Hz (FREITAS, 2013).

A medição de F_0 está sujeita a contaminações surgidas das variações naturais de *pitch* típicas da voz humana (FREITAS, 2013). A importância de se medir F_0 corretamente vem do fato de que, além de carregar boa parte da informação presente na fala, ela é a base para construção das outras medidas que compõem os sinais de voz, que são múltiplas de F_0.

2.1.2.3 Formantes

O sinal de excitação das pregas vocais é rico em harmônicas — frequências múltiplas da fundamental — que são atenuadas ou amplificadas pela geometria do trato vocal e nasal de cada locutor, gerando picos espectrais denominados formantes (VALENÇA *et al.*, 2014; FANT, 1960). Independentemente do idioma, cada formante apresenta articulações universais:

A primeira (F_1) é associada à cavidade oral posterior, reflete a altura da língua (/i/: ≈240 Hz; /u/: ≈300 Hz; /a/: ≈700 Hz).

A segunda (F_2), associada à cavidade oral anterior, codifica a posição horizontal da língua — vogais com recuo lingual e arredondamento labial apresentam F_2 baixo (/u/: ≈625 Hz; /o/: ≈780 Hz), vogais centrais apresentam F_2 intermediário (/a/: ≈1080 Hz) e vogais anteriores apresentam F_2 alto (/e/: ≈1800 Hz; /i/: ≈2250 Hz); articulações dentais e pré-palatais também elevam F_2, o arredondamento labial abaixa todas as formantes e a palatização eleva F_2 e F_3 e reduz F_1.

A terceira formante (F_3) está relacionada às cavidades em torno do ápice da língua, assume valores muito baixos (1500–1800 Hz) em articulações retroflexas — como o /r/ no inglês dos Estados Unidos — contra ≈2500 Hz nas vogais adjacentes.

Formantes acima de F_3 — notadamente F_4 (≈3400–3570 Hz), estão relacionadas à geometria da laringe e faringe. F_5 (≈4000–4655 Hz) — apresenta menor variação entre vogais e pouca distinção fonêmica, mas refletem propriedades anatômicas individuais do trato vocal (FANT, 1960).

Portanto, formantes caracterizam fortemente os locutores, e tais frequências, capturáveis pela Transformada *Wavelet*, são de suma importância na verificação de locutores.

2.1.3 Escalas e energias dos sinais

2.1.3.1 Pré-ênfase do sinal de voz

Antes de calcular energias e coeficientes espectrais do áudio, aplica-se a **pré-ênfase**, um filtro que realça as altas frequências. A motivação é física: no sinal de voz, a fonte glotal impõe uma queda espectral de cerca de -6 dB por oitava, de modo que as formantes superiores — justamente as mais discriminativas entre locutores (seção 2.1) — chegam atenuadas. A pré-ênfase compensa essa inclinação, equilibrando a energia entre bandas baixas e altas antes da extração de características (RABINER; JUANG, 1993).

Sejam $x[n]$ a amostra de áudio no instante n, $y[n]$ a amostra após a pré-ênfase e α o coeficiente de pré-ênfase (tipicamente $0{,}95 \leqslant \alpha \leqslant 0{,}97$; este trabalho usa $\alpha = 0{,}97$). O filtro é de primeira ordem:

$$y[n] = x[n] - \alpha\, x[n-1] \qquad . \tag{2.1}$$

Exemplo numérico. Para $\alpha = 0{,}97$ e as amostras $x = [1{,}0,\ 0{,}9,\ 0{,}6]$: $y[1] = 0{,}9 - 0{,}97 \times 1{,}0 = -0{,}07$ e $y[2] = 0{,}6 - 0{,}97 \times 0{,}9 = -0{,}273$. Trechos de variação lenta (baixa frequência) produzem saída pequena; transições rápidas (alta frequência) são realçadas.

2.1.3.2 Energia de um sinal

A energia de um sinal digital $s[\cdot]$ com M amostras é definida como

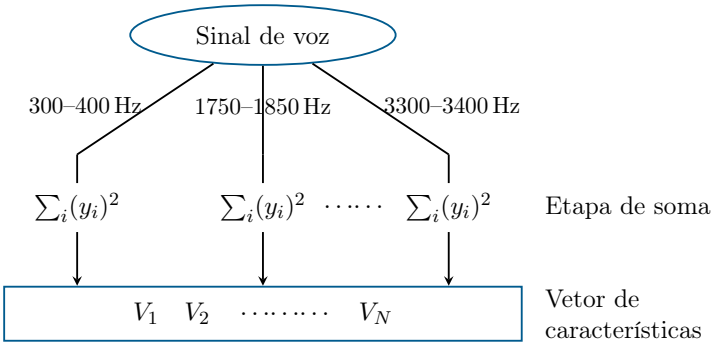
$$E = \sum_{i=0}^{M-1} (s_i)^2 \quad . \tag{2.2}$$

E pode ainda sofrer normalizações e ter a sua mensuração restrita a uma parte específica do sinal sob análise. Neste trabalho, as características extraídas são organizadas em três categorias: na **Categoria 1**, as energias por banda constituem diretamente o vetor de características; na **Categoria 2**, aplica-se adicionalmente o logaritmo e a Transformada Discreta Cosseno (DCT) sobre essas energias para obter coeficientes cepstrais. Por fim na **Categoria 3** utilizam-se *autoencoders* sobre os sinais janelados. A essas categorias soma-se a PCA[1], extrator de características linear clássico, usado como referência para a extração automatizada. As categorias 1 e 2 compreendem cada uma três esquemas de particionamento espectral — linear, Mel e Bark — resultando em seis tipos de representação descritos a seguir.

2.1.3.3 Energias em bandas lineares

No particionamento linear, as frequências são divididas em N bandas de largura constante e igualmente espaçadas. Para cada banda, a energia dos coeficientes da transformada wavelet correspondentes é acumulada, resultando no vetor $[E_1, E_2, \ldots, E_N]$, como ilustrado na Figura 2.

Figura 2 – Cálculo de vetores de características com bandas lineares



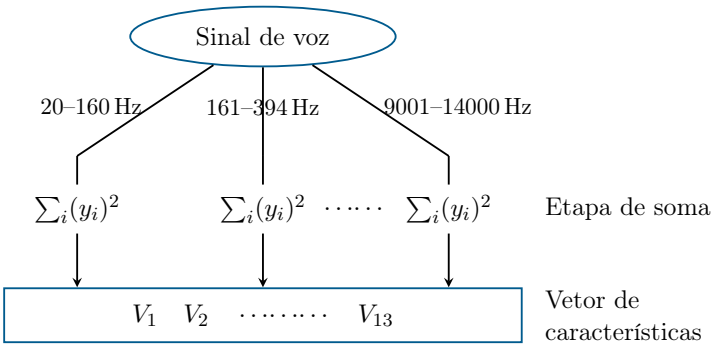
Fonte: Elaborado pelo autor, 2024.

[1] Análise de Componentes Principais (*Principal Component Analysis*)

2.1.3.4 Energias em bandas Mel

A escala Mel, advinda do termo *melody*, é uma escala perceptual independente que mapeia as frequências em bandas tonais percebidas pelo ouvido humano. Dentre as várias implementações, a adotada neste trabalho utiliza os seguintes limites de banda em Hz: **20, 160, 394, 670, 1000, 1420, 1900, 2450, 3120, 4000, 5100, 6600, 9000, 14000** (BERANEK, 1949). As energias calculadas nos 13 intervalos resultantes constituem diretamente o vetor de características $[E_1, E_2, \ldots, E_{13}]$, como mostrado na Figura 3.

Figura 3 – Cálculo de vetores de características com bandas Mel



Fonte: Elaborado pelo autor, 2024.

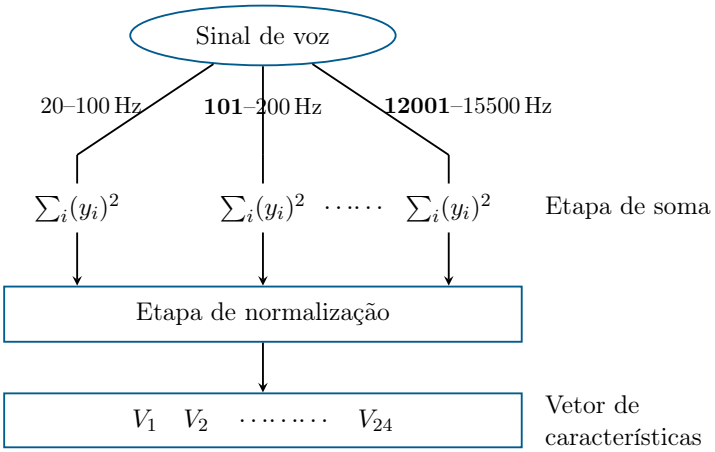
2.1.3.5 Energias em bandas Bark

A escala Bark foi definida tendo em mente vários tipos de sinais acústicos e corresponde ao conjunto de 25 bandas críticas da audição humana. Suas frequências-base de audiometria são, em Hz: **20, 100, 200, 300, 400, 510, 630, 770, 920, 1080, 1270, 1480, 1720, 2000, 2320, 2700, 3150, 3700, 4400, 5300, 6400, 7700, 9500, 12000, 15500** (ZWICKER, 1961). Assim como nos agrupamentos energéticos anteriores o sinal é separado nas bandas de frequências definidas (BOSI; GOLDBERG, 2002), resultando em um vetor de características com 24 componentes, como mostrado na Figura 4.

2.1.3.6 Coeficientes LFCC

Os coeficientes LFCC (*Linear Frequency Cepstral Coefficients*) distribuem os filtros de análise linearmente na faixa $[f_{\min}, f_{\max}]$, contrastando com as escalas Mel e Bark, que comprimem progressivamente as bandas em frequências mais altas para modelar a percepção auditiva humana. Essa compressão é vantajosa para reconhecimento de fala, mas prejudicial para verificação de locutores: as formantes superiores (F_3, F_4), associadas à geometria do trato vocal e à fonte glótica, são discriminativas entre indivíduos e residem justamente nas faixas descartadas pelas escalas perceptuais (ZHOU *et al.*, 2011; HANILÇI, 2018). Ao manter resolução espectral uniforme, o LFCC preserva esses componentes, sendo preferido ao MFCC para verificação de locutores, com custo computacional comparável (ALI *et al.*, 2022).

Figura 4 – Cálculo de vetores de características com bandas Bark



Fonte: Elaborado pelo autor, 2022.

O cálculo segue as etapas ilustradas na Figura 5: aplica-se a DFT[2] ao sinal janelado e distribui-se um banco de K filtros triangulares com centros equiespaçados linearmente (tipicamente entre 300 Hz e 3400 Hz (ZHOU *et al.*, 2011)):

$$f_k = f_{\min} + k\,\frac{f_{\max} - f_{\min}}{K+1}, \qquad k = 1, \ldots, K \qquad . \tag{2.3}$$

Exemplo numérico. Para $f_{\min} = 300$ Hz, $f_{\max} = 3400$ Hz e $K = 4$ filtros, tem-se $f_k = 300 + k \cdot \frac{3400-300}{5} = 300 + 620k$, resultando em $f_1 = 920$ Hz, $f_2 = 1540$ Hz, $f_3 = 2160$ Hz e $f_4 = 2780$ Hz. Seja E_k a energia logarítmica da banda k e N o número de coeficientes cepstrais desejado ($N \leqslant K$). E_k é então decorrelacionada pela Transformada Discreta de Cosseno (DCT) (ALI *et al.*, 2022), produzindo o coeficiente c_n de cada ordem n:

$$c_n = \sum_{k=1}^{K} \log(E_k)\,\cos\!\left(\pi n\,\frac{2k-1}{2K}\right), \qquad n = 0, \ldots, N-1 \qquad , \tag{2.4}$$

Exemplo numérico. Continuando com $K = 4$, suponha-se as energias logarítmicas já calculadas $\log(E_1) = 1$, $\log(E_2) = 2$, $\log(E_3) = 0{,}5$ e $\log(E_4) = 0$, e deseja-se $N = 2$ coeficientes (c_0 e c_1). Para $n = 0$ todos os cossenos valem 1, logo $c_0 = 1+2+0{,}5+0 = 3{,}5$. Para $n = 1$ os pesos $\cos\big(\pi(2k-1)/8\big)$ valem, para $k = 1, 2, 3, 4$: 0,9239, 0,3827, $-0{,}3827$ e $-0{,}9239$; logo

$$c_1 = 1(0{,}9239)+2(0{,}3827)+0{,}5(-0{,}3827)+0(-0{,}9239) = 0{,}9239+0{,}7654-0{,}1914 = 1{,}4979$$

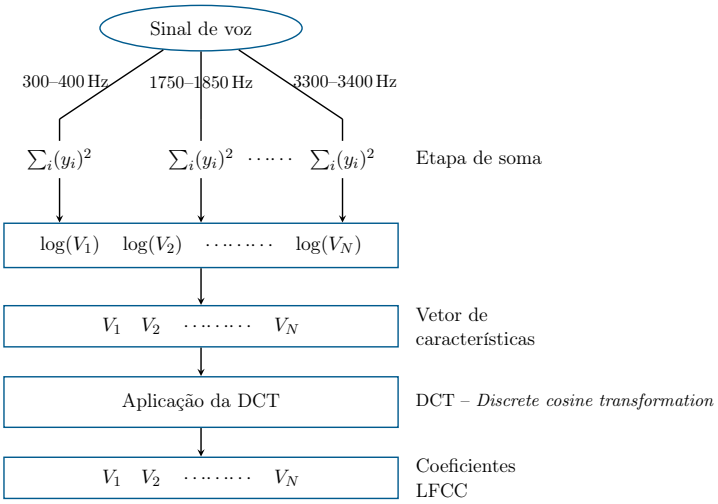
resultando no vetor final $[c_0, c_1, \ldots, c_{N-1}]$ de coeficientes LFCC.

2.1.3.7 Coeficientes MFCC

Os coeficientes MFCC (*Mel-frequency cepstral coefficients*) são obtidos aplicando-se o logaritmo às energias das bandas Mel e em seguida a Transformada Discreta Cosseno

[2] Transformada Discreta de Fourier

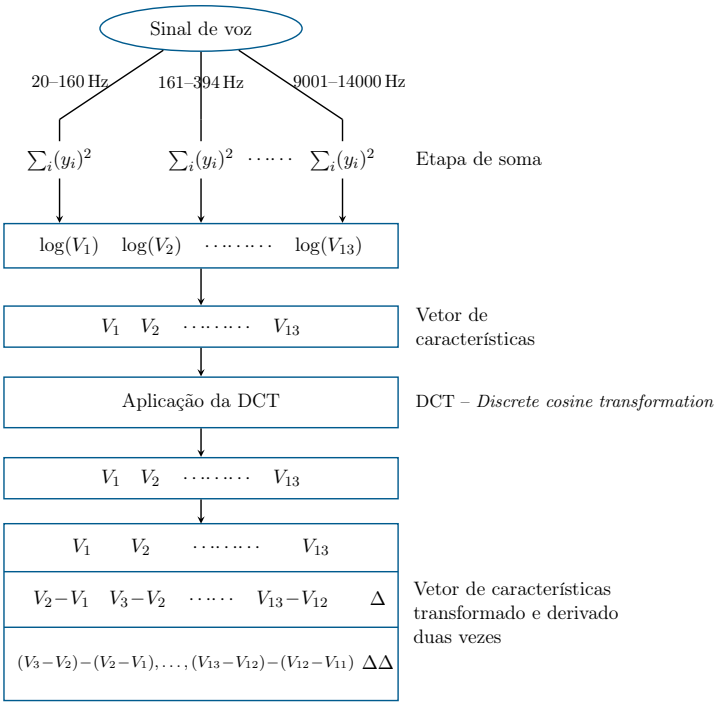
Figura 5 – Cálculo de vetores de características com LFCC



Fonte: Elaborado pelo autor, 2024.

(DCT) (SALOMON; MOTTA; BRYANT, 2007) para decorrelacionar os componentes, resultando em 13 coeficientes cepstrais ($V_1, V_2, \ldots, V_{13}$). Adicionalmente, são calculadas as primeiras e segundas derivadas temporais (Δ e $\Delta\Delta$), estendendo o vetor de características com informação dinâmica sobre a evolução temporal dos coeficientes, como ilustrado na Figura 6.

Figura 6 – Cálculo de vetores de características com MFCC

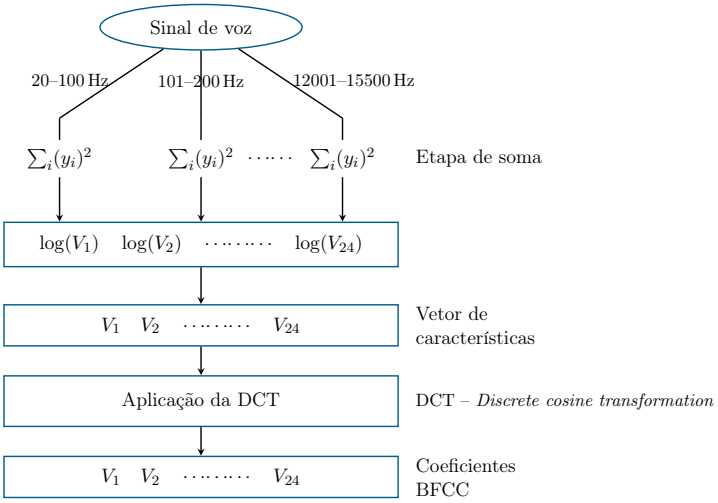


Fonte: Elaborado pelo autor, 2022.

2.1.3.8 Coeficientes BFCC

Os coeficientes BFCC (*Bark-frequency cepstral coefficients*) são obtidos aplicando-se o logaritmo às energias das bandas Bark e em seguida a Transformada Discreta Cosseno (DCT) para decorrelacionar os componentes, resultando em 24 coeficientes cepstrais como ilustrado na Figura 7.

Figura 7 – Cálculo de vetores de características com BFCC



Fonte: Elaborado pelo autor, 2024.

2.1.3.9 Aplicabilidade das escalas Bark e Mel ao EEG

As escalas Bark e Mel apresentadas acima não são escalas de frequência genéricas: ambas são **escalas cocleares**, isto é, modelos empíricos da resolução em frequência da *audição* humana. A Bark corresponde às bandas críticas da membrana basilar e a Mel a julgamentos perceptuais de igual distância tonal; ambas são definidas em Hz e calibradas sobre a faixa audível (≈ 24 Barks entre 0 e ≈ 16 kHz). Aplicá-las a um sinal de EEG — que não é som — carece de base fisiológica: não há membrana basilar segmentando o eletroencefalograma em bandas críticas. Esta subseção mostra que, além dessa objeção conceitual, o eixo da escala é *numericamente inócuo* para o EEG, e explica por quê.

Parâmetros concretos. O agrupamento parte da DTWPT[3] com `dtwpt_level` $= 4$, que particiona uniformemente o intervalo $[0, f_{Nyq}]$ em $2^4 = 16$ sub-bandas de largura $\Delta f = f_{Nyq}/16$. Os dois sinais deste trabalho diferem drasticamente em taxa de amostragem, e é daí que tudo decorre:

[3] Transformada *Wavelet-Packet* em Tempo Discreto

Tabela 1 – Particionamento espectral da DTWPT ($2^4 = 16$ sub-bandas uniformes) para cada sinal. A faixa útil do EEG é três ordens de grandeza mais estreita que a da voz.

	EEG	**Voz**
Taxa de amostragem f_s	1024 Hz	44100 Hz
Frequência de Nyquist f_{Nyq}	512 Hz	22050 Hz
Número de sub-bandas	16	16
Largura de cada sub-banda Δf	32 Hz	1378,125 Hz
Sub-banda 0 (faixa / centro)	0–32 Hz / 16 Hz	0–1378,1 Hz / 689,06 Hz
Sub-banda 15 (faixa / centro)	480–512 Hz / 496 Hz	20671,9–22050 Hz / 21360,9 Hz

Fonte: Elaborado pelo autor, 2026.

O mecanismo do agrupamento. Para *bark* e *mel*, cada sub-banda p tem sua frequência central $f_p = (p+\frac{1}{2})\Delta f$ mapeada pela escala, e o valor resultante é normalizado **pelo valor da escala na frequência de Nyquist do próprio sinal** antes de ser distribuído entre N bins ($N = 24$ para Bark, $N = 20$ para Mel):

$$\mathrm{bin}(p) = \left\lfloor \frac{\mathcal{S}(f_p)}{\mathcal{S}(f_{Nyq})} \cdot N \right\rfloor, \qquad \mathcal{S} \in \{\mathrm{Bark}, \mathrm{Mel}\}. \tag{2.5}$$

Sub-bandas que caem no mesmo bin têm seus coeficientes concatenados num único grupo; bins vazios são descartados. A escala *lfcc* não passa por isso: cada sub-banda é um grupo, dando sempre 16 grupos. A normalização por $\mathcal{S}(f_{Nyq})$ em Equação 2.5 é decisiva — ela remove qualquer ancoragem *absoluta* em Hz e estica a curva para preencher os N bins seja qual for a taxa de amostragem.

Por que o EEG degenera. Tanto Bark quanto Mel são aproximadamente *lineares* perto da origem ($\arctan x \approx x$ e $\log_{10}(1+x) \approx x/\ln 10$ para $x \ll 1$) e só se tornam fortemente compressivas em altas frequências. A faixa do EEG (0–512 Hz) está inteiramente nesse trecho quase-linear; a da voz (0–22 kHz) cobre justamente o trecho compressivo. A Tabela 2 quantifica isso: mede-se a razão entre o valor da escala no centro da última e da primeira sub-banda e compara-se com a razão que uma escala perfeitamente linear daria ($496/16 = 21360{,}9/689{,}06 = 31$).

Tabela 2 – Valores das escalas nos centros das sub-bandas extremas, grau de linearidade sobre a faixa usada, e número de grupos resultante. Para o EEG o mapeamento é injetor (16 sub-bandas → 16 grupos), degenerando exatamente na escala linear; para a voz há fusão real de sub-bandas.

Sinal	**Escala**	$\mathcal{S}(f_{Nyq})$	$\mathcal{S}(\text{centro}_0)$	$\mathcal{S}(\text{centro}_{15})$	**Linearidade**	**Grupos**
EEG	Bark	4,84	0,158	4,702	95,9%	**16**
EEG	Mel	618,66	25,47	603,68	76,5%	**16**
EEG	*lfcc*	—	—	—	(linear)	**16**
Voz	Bark	24,74	6,301	24,689	12,6%	9
Voz	Mel	3923,34	772,33	3888,68	16,2%	11
Voz	*lfcc*	—	—	—	(linear)	16

Fonte: Elaborado pelo autor, 2026.

Com 95,9% de linearidade (Bark) sobre a faixa do EEG, a Equação 2.5 vira uma função praticamente afim e monótona, e como há 24 (ou 20) bins para apenas 16 sub-bandas, o mapeamento resulta **injetor**: os bins obtidos são $[0, 2, 3, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20, 21, 23]$ para Bark e $[0, 2, 3, 5, 6, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 19]$ para Mel — todos distintos. Nenhuma sub-banda funde com outra, cada grupo contém exatamente uma sub-banda, e o agrupamento coincide *termo a termo* com o da escala *lfcc*. Já para a voz, a compressão em altas frequências faz as sub-bandas superiores se aglomerarem: os bins da Bark são $[6, 12, 15, 17, 19, 20, 21, 21, 22, 22, 23, 23, 23, 23, 23, 23]$ — as seis últimas sub-bandas caem todas no bin 23 — restando 9 grupos; a Mel deixa 11. Aí, sim, a escala carrega informação: ela decide *quais* sub-bandas são somadas.

Consequência. Para o EEG, a "escala Bark" nunca foi Bark: era uma reescala linear disfarçada, produzindo um vetor de características idêntico ao da escala linear. Isso foi confirmado empiricamente sobre as execuções da Fase 00: nos 46 pares *wavelet* × categoria do EEG, as três escalas produziram D_{verdade} idêntico *bit a bit* — 46 de 46. As únicas exceções aparentes, da *wavelet* `daub32`, diferiam em $\approx 1{,}5 \times 10^{-6}$, ruído de ponto flutuante originado da reexecução isolada daqueles perfis. Dos 138 perfis manuais de EEG, portanto, apenas 46 eram genuinamente distintos: 92 execuções eram duplicatas exatas. Por esse motivo o eixo da escala é aplicado somente à voz, e o EEG usa apenas a escala linear (Capítulo 3), reduzindo a grade da Fase 00 de 300 para 208 execuções sem perda de informação alguma.

Vale registrar a consequência metodológica de não ter percebido isso antes. Como as três variantes empatavam exatamente, o vencedor da Fase 00 para o EEG era determinado pela ordem de desempate da rotina de ranqueamento, e vinha rotulado como *bark*. Afirmar que "a escala Bark venceu para o EEG" seria falso: a Bark não produziu efeito algum, e o vetor vencedor é literalmente o da escala linear. A rotina de ranqueamento passou a detectar e reportar empates exatos de forma explícita, de modo que um desempate

arbitrário não possa ser confundido com um resultado experimental.

2.1.4 Filtros digitais *wavelet*

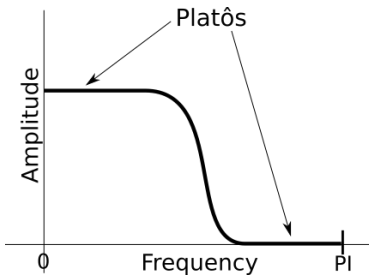
Filtros digitais *wavelet* têm sido utilizados com sucesso para suprir as deficiências de janelamento de sinal apresentadas pelas Transformadas de Fourier e de Fourier de Tempo Reduzido. *Wavelets* contam com variadas funções-filtro e têm tamanho de janela variável, o que permite uma análise multirresolução (ADDISON; WALKER; GUIDO, 2009). Particularmente, as *wavelets* proporcionam a análise do sinal de forma detalhada tanto no espectro de baixa frequência quanto no de alta contando com diferentes funções-base não periódicas diferentemente da tradicional transformada de Fourier que utilizam somente as bases periódicas senoidal e cossenoidal.

É importante observar que, quando se trata de Transformadas *Wavelet*, seis elementos estão presentes: dois filtros de análise, dois filtros de síntese e as funções ortogonais *scaling* e *wavelet*. No tocante a sua aplicação, só a transformada direta, e não a inversa, será usada na construção dos vetores de características. Portanto, os filtros de síntese, a função *scaling* e a função *wavelet* não serão elementos abordados aqui: eles somente interessariam caso houvesse a necessidade da transformada inversa.

No contexto dos filtros digitais baseados em *wavelets*, o tamanho da janela recebe o nome de **suporte**. Janelas definem o tamanho do filtro que será aplicado ao sinal. Quando esse é pequeno (limitado), se diz que a janela tem **um suporte compacto** (POLIKAR *et al.*, 1996).

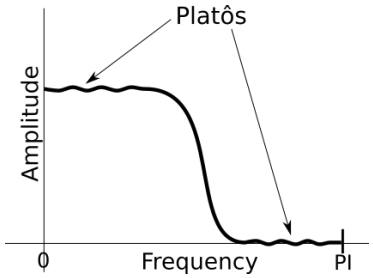
Se diz que uma *wavelet* tem boa **resposta em frequência** quando, na aplicação da mesma para filtragem, não são causadas muitas perturbações indesejadas ao sinal, no domínio da frequência. Os filtros *wavelet* de Daubechies (DAUBECHIES, 1992) se destacam nesse quesito por serem *maximamente planos* (*maximally-flat*) (BUTTERWORTH, 1930) (BIANCHI, 2007) nos platôs de resposta em frequência como indicado na Figura 8 ao contrário do que ocorre na Figura 9.

Figura 8 – Platôs maximamente planos em um filtro digital: característica da família de Daubechies



Fonte: Elaborado pelo autor, 2022.

Figura 9 – Platôs não maximamente planos de um filtro digital: características de outros filtros *wavelet*, distintos da família de Daubechies



Fonte: Elaborado pelo autor, 2022.

Além da resposta em frequência, na aplicação de um filtro digital *wavelet* também é possível considerar a **resposta em fase**, que constitui um atraso ou adiantamento do sinal filtrado em relação ao sinal original, ambos no domínio temporal. Esse deslocamento pode ser **linear**, **quase linear** ou **não linear**:

- na resposta em fase **linear**, há o mesmo deslocamento de fase para todos os componentes do sinal;
- quando a resposta em fase é **quase linear** existe uma pequena diferença no deslocamento dos diferentes componentes do sinal;
- finalmente, quando a resposta é **não linear**, acontece um deslocamento significativamente heterogêneo para as diferentes frequências que compõem o sinal.

Idealmente, é desejável que todo filtro apresente boa resposta em frequência e em fase linear. Características de fase e frequência de algumas famílias de filtros *wavelet* constam na Tabela 3.

Tabela 3 – Algumas das *wavelets* mais usadas e suas propriedades

Wavelet	Resposta em frequência	Resposta em fase
Haar	Pobre	Linear
Daubechies	mais próxima da ideal à medida que o suporte aumenta; *maximally-flat*	Não linear
Symmlets	mais próxima da ideal à medida que o suporte aumenta; não *maximally-flat*	Quase linear
Coiflets	mais próxima da ideal à medida que o suporte aumenta; não *maximally-flat*	Quase linear

Fonte: Elaborado pelo autor, 2022.

2.1.4.1 O algoritmo de Mallat para a Transformada *Wavelet*

Baseando-se no artigo (GUIDO, 2015), percebe-se que o algoritmo de Mallat faz com que aplicação das *wavelets* seja uma simples multiplicação de matrizes. O sinal que

deve ser transformado se torna uma matriz linear vertical. Os filtros passa-baixa e passa-alta tornam-se, nessa ordem, linhas de uma matriz quadrada que será completada segundo regras que serão mostradas mais adiante. É importante que essa matriz quadrada tenha a mesma dimensão que o sinal a ser transformado.

Para que seja possível a transformação *wavelet*, basta ter disponível o vetor do filtro passa-baixas calculado a partir da *mother wavelet*, que é a função geradora desse filtro, já que o passa-alta pode ser construído a partir da ortogonalidade do primeiro.

Determinar a ortogonal de um vetor significa construir um vetor, tal que, o produto escalar do vetor original com sua respectiva ortogonal seja nulo.

Considerando $h[\cdot]$ como sendo o vetor do filtro passa-baixas e $g[\cdot]$ seu correspondente ortogonal, tem-se que $h[\cdot] \cdot g[\cdot] = 0$.

Portanto, se $h[\cdot] = [a, b, c, d]$ então seu ortogonal será $g[\cdot] = [d, -c, b, -a]$ pois:

$$h[\cdot]\cdot g[\cdot] = [a, b, c, d]\cdot[d, -c, b, -a] = (a\cdot d)+(b\cdot(-c))+(c\cdot b)+(d\cdot(-a)) = ad-ad+bc-bc = 0 \quad .$$

A título de exemplo, considera-se:

- o filtro passa baixa baseado na *wavelet* Haar: $h[\cdot] = [\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}]$
- o seu respectivo vetor ortogonal: $g[\cdot] = [\frac{1}{\sqrt{2}}, \frac{-1}{\sqrt{2}}]$
- e também o seguinte sinal-exemplo de entrada: $s = \{1, 2, 3, 4\}$

Se o tamanho do sinal a ser tratado é quatro e se pretende-se aplicar o filtro Haar, a seguinte matriz de coeficientes é construída:

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0 \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \tag{2.6}$$

Tendo em vista que a dimensão do sinal sob análise é diferente da dimensão do filtro, basta completar cada uma das linhas da matriz de coeficientes com zeros. A matriz é montada de forma que ela seja ortogonal.

Uma matriz W é dita ortogonal quando suas linhas (e colunas) são mutuamente ortogonais e têm norma unitária, o que implica $W^T W = W W^T = I$, sendo I a matriz identidade e W^T a transposta de W. Essa propriedade garante que a transformada preserve a energia do sinal — isto é, $\|W.s\|^2 = \|s\|^2$ — e que a inversão seja obtida simplesmente por $W^{-1} = W^T$, sem necessidade de cálculo de inversa.

Montada a matriz de filtros, segue-se com os cálculos da transformada:

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0 \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} \frac{3}{\sqrt{2}} \\ \frac{-1}{\sqrt{2}} \\ \frac{7}{\sqrt{2}} \\ \frac{-1}{\sqrt{2}} \end{pmatrix} \tag{2.7}$$

Realizada a multiplicação, é necessário montar o sinal filtrado. Isso é feito escolhendo, dentro do resultado, valores alternadamente de forma que o vetor resultante seja:

$$resultado = \left[\frac{3}{\sqrt{2}}, \frac{7}{\sqrt{2}}, \frac{-1}{\sqrt{2}}, \frac{-1}{\sqrt{2}}\right] \quad . \tag{2.8}$$

Percebe-se que, na transformação descrita na Equação 2.6, Equação 2.7 e Equação 2.8, a **aplicação dos filtros sobre o vetor de entrada ocorreu apenas uma vez**. Sendo assim, se diz que o sinal recebeu uma **transformação de nível 1**. A cada transformação, há uma separação do sinal em dois componentes: o de baixa e o de alta frequência.

Embora haja um limite, que será mencionado adiante, é possível aplicar mais de um nível de decomposição ao sinal. Para que se possa fazer isso, a Transformada *Wavelet* nível 2 deve considerar apenas a parte de baixas frequências da primeira transformada; a transformada de nível 3 deve considerar apenas a parte de baixas frequências da transformada nível 2, e assim consecutivamente.

A fim de facilitar o entendimento, para fabricar os exemplos numéricos mostrados na Tabela 4, Tabela 5 e Tabela 6, foi usado um filtro normalizado cujos coeficientes são $\{\frac{1}{2}, -\frac{1}{2}\}$. Os dados destacados em **verde** correspondem ao **vetor original** que será tratado. Cada uma das linhas são os resultados das transformações nos níveis 1, 2, 3 e 4, respectivamente. As partes em **azul** correspondem à porção de **baixas frequências**, enquanto que as partes em **laranja** correspondem às porções de **altas frequências**.

Percebe-se que na Tabela 4, a partir da transformação nível 2, apenas as partes de baixa frequência são modificadas. Isso implica que, no momento da implementação do algoritmo de Mallat **para níveis maiores que 1**, a abordagem será **recursiva**. Em outras palavras, a partir do nível 1 se deve aplicar Mallat apenas às porções de baixas-frequências geradas pela transformação anterior.

2.1.4.2 O algoritmo de Mallat e a Transformada *Wavelet-Packet*

Na Transformada *Wavelet-Packet*, os filtros aplicados são os mesmos da Transformada *Wavelet* e o procedimento recursivo de cálculo também é o mesmo, no entanto, realizada a transformação de nível 1, a transformada de nível 2 deve ser aplicada aos componentes de baixa e de alta frequência. Sendo assim a Transformada *Wavelet-Packet*

Tabela 4 – Exemplo numérico da transformação *wavelet* aplicada a um vetor. Tabela completa no Apêndice B.

Sinal	**32**	**10**	**20**	**38**	**37**	**28**	**38**	**34**	**18**	**24**	**24**	**9**	**23**	**24**	**28**	**34**
Nível 01	21	29	32,5	36	21	16,5	23,5	31	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 02	25	34,25	18,75	27,25	-4	-1,75	2,25	-3,75	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 03	29,62	23	-4,625	-4,25	-4	-1,75	2,25	-3,75	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 04	26,3125	3,3125	-4,625	-4,25	-4	-1,75	2,25	-3,75	11	-9	4,5	2	-3	7,5	-0,5	-3

Fonte: Elaborado pelo autor, 2022.

obtém um nível de detalhes em todo o espectro de frequência, maior do que uma transformação regular.

Os exemplos mostrados na Tabela 5 e Tabela 6 permitem perceber como se dão as transformações na porção de **baixa** e de **alta** frequências, respectivamente, após a transformação *wavelet-packet* de nível 1, 2, 3 e 4.

O *downsampling* aplicado às porções de alta frequência, fazem essas partes ficarem "espelhadas" no espectro (JENSEN; COUR-HARBO, 2001), ou seja, suas sequências ficam invertidas. Para resolver esse problema e preservar a ordem das sub-bandas no sinal transformado, os filtros são aplicados em ordem inversa nas porções de alta frequência. Isso altera como o algoritmo de Mallat deve ser implementado para a Transformada *Wavelet-Packet*, já que dessa vez é preciso se atentar a ordem da aplicação dos filtros passa-alta e passa-baixa.

Tabela 5 – Exemplo numérico de *wavelet-packet* Haar aplicada ao vetor da Tabela 4 (porção das baixas frequências). Tabela completa no Apêndice B.

Sinal	**32**	**10**	**20**	**38**	**37**	**28**	**38**	**34**
Nivel 01	21	29	32,5	36	21	16,5	23,5	31
Nivel 02	25	34,25	18,75	27,25	-4	-1,75	2,25	-3,75
Nivel 03	29,62	23	-4,625	-4,25	-1,125	3	-2,875	-0,75
Nivel 04	26,3125	3,3125	-0,1875	-4,4375	0,9375	-2,0625	-1,0625	-1,8125

Fonte: Elaborado pelo autor, 2022.

Tabela 6 – Exemplo numérico de *wavelet-packet* Haar aplicada ao vetor da Tabela 4 (porção das altas frequências). Tabela completa no Apêndice B.

Sinal	**18**	**24**	**24**	**9**	**23**	**24**	**28**	**34**
Nivel 01	11	-9	4,5	2	-3	7,5	-0,5	-3
Nivel 02	10	1,25	-5,25	1,25	1	3,25	2,25	-1,75
Nivel 03	5,625	-2	4,375	-3,25	-1,125	2	2,125	0,25
Nivel 04	1,8125	3,8125	3,8125	0,5625	0,4375	-1,5625	0,9375	1,1875

Fonte: Elaborado pelo autor, 2022.

2.1.5 Engenharia Paraconsistente de características

Nos processos de classificação, frequentemente surge a questão: "Os vetores de características criados proporcionam uma boa separação de classes?". A Engenharia Paraconsistente de Características (GUIDO, 2019), que usa a paraconsistência (COSTA; BÉZIAU; BUENO, 1998), (COSTA; ABE, 2000) é uma ferramenta que pode ser usada para responder essa questão.

O processo inicia-se após a aquisição dos vetores de características para cada classe C_n. Se o número de classes presentes for, por exemplo, quatro então estas poderão ser representadas por C_1, C_2, C_3, C_4.

Em seguida é necessário o cálculo de duas grandezas:

- a menor similaridade intraclasse, α.
- a razão de sobreposição interclasse, β.

α indica o quanto de similaridade os dados têm entre si, dentro de uma mesma classe, enquanto β é a razão de sobreposição entre diferentes classes. Idealmente, α deve ser maximizada e β minimizada assim, mesmo classificadores extremamente modestos podem apresentar um bom desempenho.

Particularmente, para calcular α e β, é necessária a normalização dos vetores de características de forma que todos os seus componentes estejam no intervalo entre 0 e 1. Em seguida, a obtenção de α se dá selecionando-se os maiores e os menores valores de cada uma das posições de todos os vetores de características para cada classe, gerando assim um vetor para os valores maiores e outro para os menores.

O **vetor de similaridade da classe**(svC_n) é obtido fazendo-se a diferença item-a-item dos maiores em relação aos menores. Finalmente, e para cada classe, é obtida a média dos valores para cada vetor de similaridade, sendo que α é o menor valor dentre essas médias. A Figura 10 contém uma ilustração do processo.

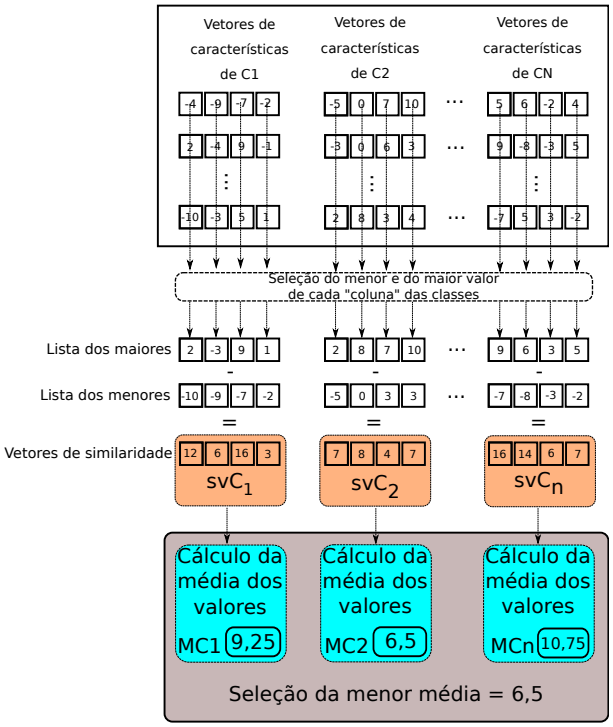
A obtenção de β, assim como ilustrado na Figura 11, também se dá selecionando os maiores e os menores valores de cada uma das posições de todos os vetores de características de cada classe, gerando assim um vetor para os valores maiores e outro para os menores.

Na sequência, realiza-se o cálculo de R cujo valor é a quantidade de vezes que um valor do vetor de características de uma classe se encontra entre os valores maiores e menores de outra classe.

Seja:

- N a quantidades de classes;

Figura 10 – Cálculo do coeficiente α.



Adaptado de (GUIDO, 2019)

- X a quantidade de vetores de características por classe;
- T o tamanho do vetor de características.

Então, F, que é o número máximo de sobreposições possíveis entre classes, é dado por (GUIDO, 2019):

$$F = N.(N-1).X.T \quad . \tag{2.9}$$

Finalmente, β é calculado da seguinte forma (GUIDO, 2019):

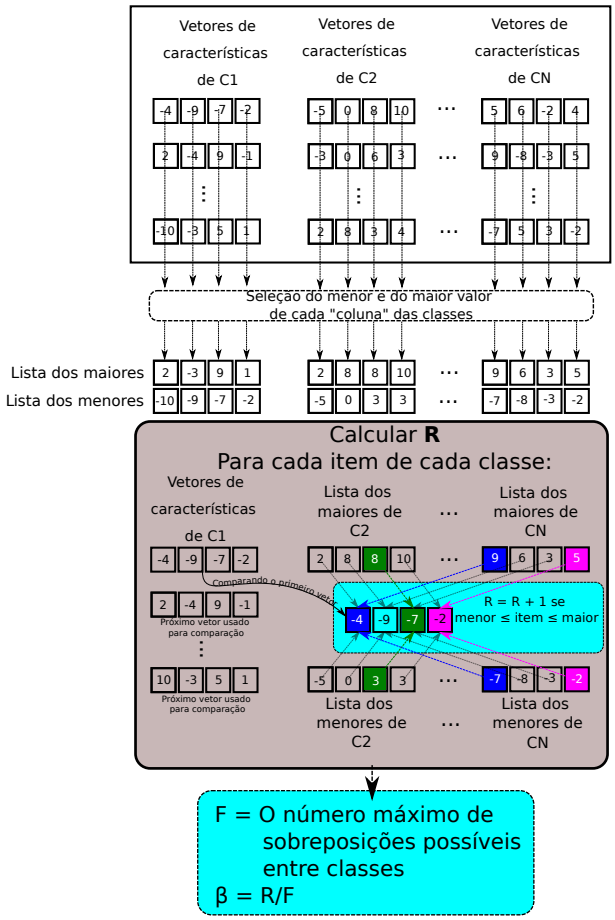
$$\beta = \frac{R}{F} \quad . \tag{2.10}$$

Exemplo numérico. Para $N = 4$ classes, $X = 10$ vetores por classe e $T = 5$ características por vetor, $F = 4 \times 3 \times 10 \times 5 = 600$. Se $R = 45$ sobreposições forem contadas, $\beta = 45/600 = 0{,}075$.

Neste ponto, é importante notar que $\alpha = 1$ sugere fortemente que os vetores de características de cada classe são similares e representam suas respectivas classes precisamente. Complementarmente, $\beta = 0$ sugere os vetores de características de classes diferentes não se sobrepõe (GUIDO, 2019).

Considerando os valores máximos e mínimos que α e β podem assumir é possível ter a seguinte interpretação:

Figura 11 – Cálculo de β: Os itens destacados em azul e rosa são aqueles pertencentes a classe C1 e CN que se sobrepõe, em verde, a sobreposição é entre C1 e C2. Para cada sobreposição verificada soma-se 1 ao valor R. Essa comparação é feita para todos os vetores de características de cada uma das classes.



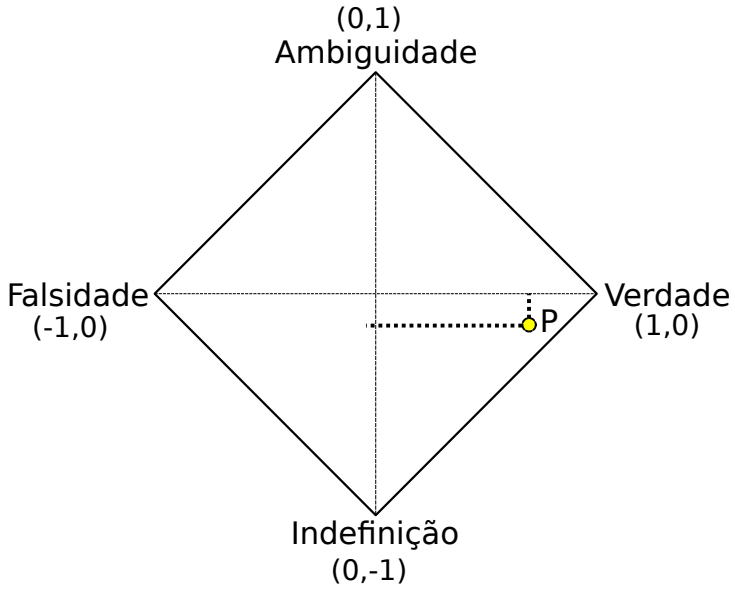
Adaptado de (GUIDO, 2019)

- Verdade → fé total ($\alpha = 1$) e nenhum descrédito ($\beta = 0$)
- Ambiguidade → fé total ($\alpha = 1$) e descrédito total ($\beta = 1$)
- Falsidade → fé nula ($\alpha = 0$) e descrédito total ($\beta = 1$)
- Indefinição → fé nula ($\alpha = 0$) e nenhum descrédito ($\beta = 0$) .

No entanto, é improvável que os valores de α e β assumam valores inteiros, como na listagem apresentada anteriormente, uma vez que $\alpha, \beta \in \mathbb{R}$ e estão restritos aos intervalos $0 \leqslant \alpha \leqslant 1$ e $0 \leqslant \beta \leqslant 1$,

Assim, para fins de visualização e maior rapidez na avaliação dos resultados, o **grau de certeza** (G_1) e o **grau de contradição** (G_2) devem ser calculados, esses valores são utilizados para criar o chamado **plano paraconsistente** ilustrado na Figura 12.

Figura 12 – O plano paraconsistente: O pequeno círculo $P(G_1, G_2)$ indica os graus de falsidade(-1,0), verdade(1, 0), indefinição(0,-1) e ambiguidade(0,1) de acordo com os valores do par ordenado (G_1, G_2)



Adaptado de (GUIDO, 2019)

G_1 e G_2 são obtidos pelas seguintes expressões (GUIDO, 2019):

$$G_1 = \alpha - \beta \quad , \tag{2.11}$$

$$G_2 = \alpha + \beta - 1 \quad , \tag{2.12}$$

Exemplo numérico. Continuando o exemplo anterior, seja $\alpha = 0{,}92$ (menor similaridade intraclasse medida) e $\beta = 0{,}075$ (calculado acima). Então $G_1 = 0{,}92 - 0{,}075 = 0{,}845$ e $G_2 = 0{,}92 + 0{,}075 - 1 = -0{,}005$: o ponto (G_1, G_2) fica próximo de $(1, 0)$, indicando boa separação entre classes. onde: $-1 \leqslant G_1$ e $1 \geqslant G_2$.

Os valores de G_1 e G_2, em conjunto, definem os graus entre verdade ($G_1 = 1$) e falsidade ($G_1 = -1$) e também os graus entre indefinição ($G_2 = -1$) e ambiguidade ($G_2 = 1$). Novamente, raramente tais valores inteiros serão alcançados já que G_1 e G_2 dependem de α e β.

Para se ter ideia em que área exatamente se encontram as classes avaliadas, as distâncias (D) do ponto $P = (G_1, G_2)$ até o limites supracitados podem ser computadas da seguinte forma (GUIDO, 2019):

$$D_{-1,0} = \sqrt{(G_1 + 1)^2 + (G_2)^2} \quad , \tag{2.13}$$

$$D_{1,0} = \sqrt{(G_1 - 1)^2 + (G_2)^2} \quad , \tag{2.14}$$

$$D_{0,-1} = \sqrt{(G_1)^2 + (G_2 + 1)^2} \quad , \tag{2.15}$$

$$D_{0,1} = \sqrt{(G_1)^2 + (G_2 - 1)^2} \qquad . \tag{2.16}$$

Exemplo numérico. Com $G_1 = 0{,}845$ e $G_2 = -0{,}005$ do exemplo anterior:

$$\begin{aligned} D_{-1,0} &= \sqrt{(1{,}845)^2 + (-0{,}005)^2} \approx 1{,}8451, \\ D_{1,0} &= \sqrt{(-0{,}155)^2 + (-0{,}005)^2} \approx 0{,}1551, \\ D_{0,-1} &= \sqrt{(0{,}845)^2 + (0{,}995)^2} \approx 1{,}3054, \\ D_{0,1} &= \sqrt{(0{,}845)^2 + (-1{,}005)^2} \approx 1{,}3131. \end{aligned}$$

$D_{1,0}$ é a menor distância, confirmando que (G_1, G_2) está mais próximo do ponto *"Verdade"*, ou seja, as classes deste exemplo estão bem separadas.

Na prática, ou seja, para fins de classificação, geralmente considera-se a distância em relação ao ponto *"(1,0)* $\rightarrow$ *Verdade"*, que é o ponto ótimo: quanto mais próximo o ponto (G_1, G_2) estiver de $(1, 0)$, mais os vetores de características das diferentes classes estão naturalmente separados. Isso implica, dentro da limitação de cada algoritmo, em resultados melhores.

2.1.5.1 Vulnerabilidade de $D_{1,0}$ isolado e métrica penalizada por contradição

$D_{1,0}$, usado isoladamente como critério de seleção na Fase 00 (Capítulo 3), tem uma fragilidade que só se manifesta quando um dos ramos de extração — os *autoencoders* — é capaz de produzir uma saída praticamente constante, o que ocorre quando o treinamento é insuficiente ou a taxa de aprendizado é excessivamente pequena. Como α mede a similaridade *dentro* de cada classe, uma rede que gera o mesmo vetor de características para toda e qualquer entrada atinge $\alpha = 1$ (dispersão intraclasse nula) — mas, pela própria definição de β como razão de sobreposição *entre* classes, essa mesma saída constante também força $\beta = 1$: se todas as classes ocupam o mesmo ponto do espaço de características, cada vetor de uma classe está necessariamente contido no intervalo de qualquer outra. O par $(\alpha, \beta) = (1, 1)$ corresponde exatamente ao vértice *Ambiguidade* do plano paraconsistente e produz

$$D_{1,0}\big|_{\alpha=\beta=1} = \sqrt{(1-1)^2 + 1^2} = \sqrt{2} \approx 1{,}4142 \qquad , \tag{2.17}$$

valor que, por ser relativamente baixo, pode colocar essa rede **sem nenhuma informação de classe** à frente de extratores genuinamente separáveis no ranking — o critério confunde "vetores idênticos entre si" (que também são, trivialmente, muito parecidos dentro de cada classe) com "vetores bem separados entre classes".

Esse não é um risco hipotético: durante os experimentos desta tese, uma variante do *autoencoder* de pulso treinada com uma taxa de aprendizado efetiva substancialmente menor que a configurada atingiu exatamente $\alpha = 1{,}0000$ e $\beta = 1{,}0000$, pontuando $D_{1,0} = 1{,}4142$ — o que a colocaria em primeiro lugar entre as 58 configurações avaliadas para o

sinal de EEG, superando inclusive o extrator manual vencedor. A causa raiz — uma taxa de aprendizado configurada incorretamente para os parâmetros biofísicos dos neurônios de pulso — é discutida em subseção 2.1.10.10; o ponto relevante aqui é que o critério de seleção, por si só, não protegia contra esse tipo de degenerescência.

A generalização do problema fica clara ao observar que $\alpha = \beta$ força $G_2 = \alpha+\beta-1 = 2\alpha - 1$ e $G_1 = \alpha - \beta = 0$, ou seja, $(\alpha, \beta) = (k, k)$ desliza sobre o eixo $G_1 = 0$ conforme k varia — o eixo que liga exatamente os vértices *Ambiguidade* $(0, 1)$ e *Indefinição* $(0, -1)$. Como $D_{1,0}$ só enxerga a distância até *Verdade*, ele é cego à proximidade de um ponto desse eixo, que é sempre um estado degenerado: em $(0, 1)$ há concordância perfeita mas sem qualquer poder discriminativo (*Ambiguidade*); em $(0, -1)$ não há concordância nem estrutura alguma (*Indefinição*).

A correção, portanto, precisa penalizar a proximidade desse eixo sem abrir mão da informação já contida em $D_{1,0}$. O grau de contradição G_2 (Equação 2.12) é exatamente a coordenada que mede essa proximidade: seus dois polos, $G_2 = +1$ e $G_2 = -1$, são justamente os vértices *Ambiguidade* e *Indefinição*. Uma primeira tentativa, mais literal — recompensar a proximidade de *Verdade* e penalizar apenas a proximidade de *Ambiguidade* e *Indefinição*, isto é, maximizar $D_{0,1} + D_{0,-1} - D_{1,0}$ — foi testada e descartada: por não penalizar $G_2 = 0$ oposto, isto é, o eixo G_1, essa formulação deixa de penalizar a proximidade do vértice *Falsidade* $(-1, 0)$, e o critério simplesmente desloca o problema — passa a favorecer configurações próximas de *Falsidade* em vez de *Ambiguidade*, já que nada as distingue nessa fórmula. Confirmou-se isso empiricamente: sob essa métrica, a pior codificação testada (a mais próxima de *Falsidade* entre as avaliadas) passava a vencer o ranking.

A métrica adotada preserva $D_{1,0}$ — que já penaliza *Falsidade*, por ser o vértice mais distante de *Verdade* — e soma a ele um termo proporcional a $|G_2|$, o valor absoluto do grau de contradição:

$$D_{\text{penalizado}} = D_{1,0} + \lambda\,|G_2| \qquad , \tag{2.18}$$

em que λ é um peso a determinar. Um critério natural para escolher λ é exigir que os três vértices não-*Verdade* — *Falsidade*, *Ambiguidade* e *Indefinição* — sejam penalizados igualmente, sem viés entre as três formas de degenerescência. Calculando $D_{\text{penalizado}}$ em cada um:

$$\begin{aligned}
\text{Falsidade } (-1,0): &\quad D_{1,0} = 2,\ G_2 = 0 \ \Rightarrow\ D_{\text{penalizado}} = 2,\\
\text{Ambiguidade } (0,1): &\quad D_{1,0} = \sqrt{2},\ G_2 = 1 \ \Rightarrow\ D_{\text{penalizado}} = \sqrt{2} + \lambda,\\
\text{Indefinição } (0,-1): &\quad D_{1,0} = \sqrt{2},\ G_2 = -1 \ \Rightarrow\ D_{\text{penalizado}} = \sqrt{2} + \lambda \qquad .
\end{aligned}$$

Ambiguidade e Indefinição já chegam empatadas por simetria; impondo que também se igualem a Falsidade,

$$\sqrt{2} + \lambda = 2 \implies \lambda = 2 - \sqrt{2} \approx 0{,}5858 \qquad , \tag{2.19}$$

valor que torna os três vértices degenerados igualmente penalizados ($D_{\text{penalizado}} = 2$), preservando $D_{\text{penalizado}} = 0$ apenas em *Verdade*.

Exemplo numérico. Para o caso degenerado discutido acima ($\alpha = \beta = 1$, $G_1 = 0$, $G_2 = 1$, $D_{1,0} = \sqrt{2}$): $D_{\text{penalizado}} = \sqrt{2} + 0{,}5858 \times 1 \approx 2{,}4142$ — o pior valor possível, e não mais um dos melhores. Já para o exemplo numérico apresentado mais acima nesta seção ($G_1 = 0{,}845$, $G_2 = -0{,}005$, $D_{1,0} \approx 0{,}1551$): $D_{\text{penalizado}} \approx 0{,}1551 + 0{,}5858 \times 0{,}005 \approx 0{,}1580$ — praticamente inalterado, pois esse ponto já está longe do eixo Ambiguidade–Indefinição.

A verificação sobre as 208 combinações da Fase 00 (Capítulo 4) confirma o efeito pretendido: a configuração degenerada citada acima cai da primeira para a última posição do ranking em ambos os sinais, e nenhuma outra configuração é afetada de forma perceptível, exceto aquelas cujo β já estava próximo de 1. Como $D_{\text{penalizado}}$ é função apenas de α e β — ambos já registrados para cada execução — a nova métrica pôde ser recalculada sobre os resultados existentes sem a necessidade de repetir nenhum experimento. As implicações dessa mudança de critério sobre o vencedor de cada sinal e sobre a leitura dos resultados da Fase 00 são discutidas no Capítulo 4.

2.1.6 Fala imaginada

A fala imaginada é um fenômeno em que uma pessoa "ouve"a si mesma falando internamente, sem produzir som. As regiões cerebrais usadas na fala imaginada são similares às que estão envolvidas na fala verbal, englobando áreas do córtex motor e pré-motor, além de outras áreas associadas à linguagem e a fala.

A ativação cerebral durante processos linguísticos (incluindo a fala imaginada) envolve as seguintes regiões (PINTO, 2012), (VANDERAH, 2020), (VASKOVIĆ ALEXANDRA OSIKA, 2024):

- **área de Broca**: Localizada no lobo frontal dominante, é crucial para a produção da fala.
- **área de Wernicke**: Situada no lobo temporal e parietal esquerdo, é fundamental para a compreensão da linguagem.
- **córtex motor e pré-motor**: Envolvidos na execução e planejamento dos movimentos necessários para a articulação da fala.
- **giro supramarginal**: Associado à integração sensorial e ao planejamento motor da fala.
- **giro supratemporal e médio**: Responsáveis pelo processamento da compreensão linguística e pela articulação durante a repetição de sílabas.

Na maioria dos casos, o lobo dominante é o esquerdo, em pessoas canhotas a probabilidade de ser o direito, embora baixa, é maior.

A localização de tais áreas serão mostradas na subseção 2.1.7.

2.1.7 Interfaces Humano-Máquina e EEG

Entre os métodos de Interface Cérebro-Computador (BCI), o Eletroencefalograma (EEG) se destaca como o sistema mais econômico e simples de implementar. O EEG registra as atividades elétricas do cérebro captando-as através de eletrodos colocados ao longo do couro cabeludo. No entanto possui algumas peculiaridades como alta sensibilidade a interferência eletromagnética e dificuldade em capturar o sinal devido a posicionamentos subótimos dos eletrodos no couro cabeludo. Portanto, um dos aspectos fundamentais que qualquer sistema de processamento de EEG deve ter é a tolerância ao ruído (BIDGOLY; BIDGOLY; AREZOUMAND, 2020).

As atividades elétricas surgem dos fluxos de corrente iônica induzidos pela ativação sináptica sincronizada dos neurônios do cérebro. Elas se manifestam como flutuações de voltagem rítmicas com amplitude variando de 5 a 100μV e com frequência entre 0,5 e 40 Hz (BIDGOLY; BIDGOLY; AREZOUMAND, 2020; MECARELLI, 2019; MASTRANGELO BARBARA SCELSAM, 2019). Para cada tipo de ação realizada, o cérebro trabalha segundo as bandas listadas logo abaixo (SANEI; CHAMBERS, 2021).

Para captação de sinais de EEG usam-se eletrodos que podem ser do tipo úmido ou seco. Os úmidos são colocados usando gel condutivo e são menos propensos a gerar artefatos provocados por movimentos no sinal captado. Artefatos são interferências eletromagnéticas causadas, por exemplo, pelos músculos ao piscar dos olhos. Os eletrodos secos não precisam do gel, mas são mais sensíveis a tais ruídos.

- **Delta (0,5–4Hz)**: A onda mais lenta e geralmente a de maior amplitude. A banda Delta é observada em bebês e durante o sono profundo em adultos com amplitude registrada de até 350 μV.

- **Theta (4–8Hz)**: Observada em crianças, adultos sonolentos e durante a recordação de memórias. A amplitude da onda Theta é tipicamente inferior a 100μV.

- **Alpha (8–12Hz)**: Geralmente a banda de frequência dominante, aparecendo durante a consciência relaxada ou quando os olhos estão fechados. Essas ondas têm amplitudes normalmente inferiores a 50 μV.

- **Beta (12–25Hz)**: Associada ao pensamento, concentração ativa e atenção focada. A amplitude da onda Beta é normalmente inferior a 30 μV.

- **Gamma (acima de 25Hz)**: Observada durante o processamento sensorial múltiplo. Os padrões Gamma têm a menor amplitude, no entanto, até o presente momento não foram encontradas fontes consensuais em relação a tais valores.

As amplitudes das bandas Delta e Gamma não foram encontradas quando procuradas nas principais bases de dados.

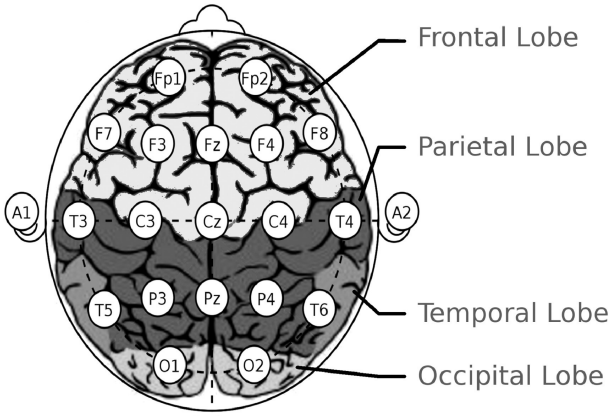
De acordo com (BIDGOLY; BIDGOLY; AREZOUMAND, 2020) e (VICENTE, 2023), para a maioria das tarefas realizadas pelo cérebro, existem regiões associadas a elas, conforme pode ser visto na Tabela 7.

2.1.8 Sistema 10-20 e as áreas do cérebro

Tabela 7 – Tarefas cerebrais e suas regiões correspondentes

Região	Canais	Tarefas
Lobo frontal	Fp1, Fp2, Fpz, Pz, F3, F7, F4, F8	Memória, concentração, emoções.
Lobo parietal	P3, P4, Pz	Resolução de problemas, atenção, sentido do tato.
Lobo temporal	T3, T5, T4, T6	Memória, reconhecimento de faces, audição, palavras e percepção social.
Lobo occipital	O1, O2, Oz	Leitura, visão.
Cerebelo		Controle motor, equilíbrio.
Córtex senso-motor	C3, C4, Cz	Atenção, processamento mental, controle motor fino, integração sensorial.

Figura 13 – Posicionamento dos eletrodos de acordo com o padrão 10-20. Números ímpares são atribuídos aos eletrodos no hemisfério esquerdo, e números pares são atribuídos aos eletrodos no hemisfério direito.

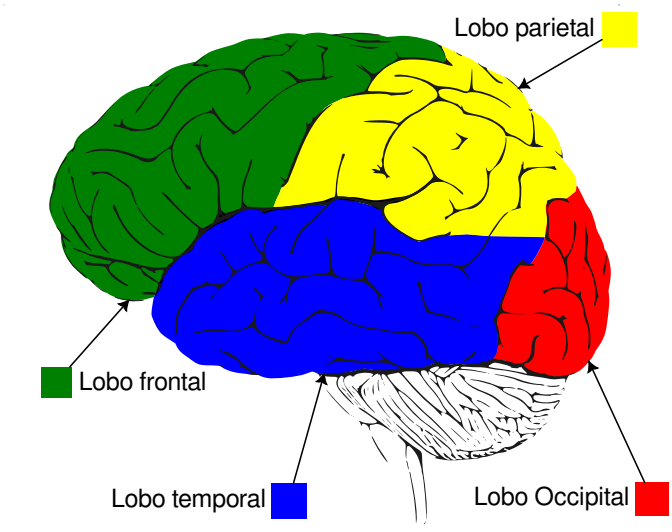


Fonte: (BIDGOLY; BIDGOLY; AREZOUMAND, 2020)

Segundo (ABDULGHANI; WALTERS; ABED, 2023a) e (PINTO, 2012), em se tratando da produção e articulação da fala, a área de Wernicke é responsável por garantir que a mesma faça sentido enquanto que a área de Broca garante que seja produzida de forma fluente portanto, já que a investigação desse trabalho também envolve a fala fonada, uma atenção especial deve ser dada a essas regiões. De acordo com a Figura 16, Figura 17 e a Figura 13 os eletrodos frontais e temporais esquerdos F7 e T5 podem estar mais próximos da área de Wernicke e os eletrodos Fp1, F3 e F7 da área de Broca.

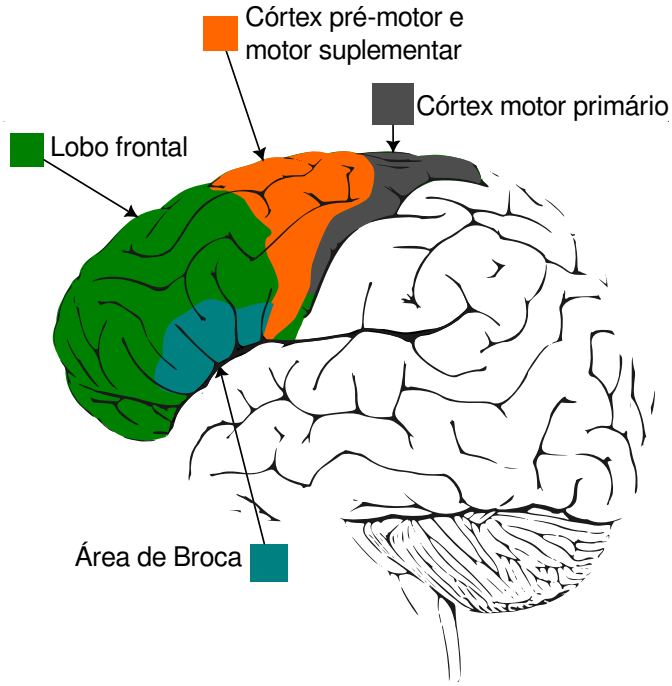
No entanto, é importante ressaltar que a fala imaginada, diferentemente da fonada, não mobiliza os córtex motores de forma intensa, ativando de modo mais distribuído as regiões corticais associadas à linguagem e ao processamento fonológico. De acordo com (FLINKER *et al.*, 2015) e (DEWITT; RAUSCHECKER, 2013), tais ativações se estendem por regiões temporais e parietais bilaterais, não se restringindo ao hemisfério dominante como ocorre na maioria dos processos de fala fonada.

Figura 14 – Principais regiões do cérebro: Lobos Frontal, Parietal, Occipital e Temporal



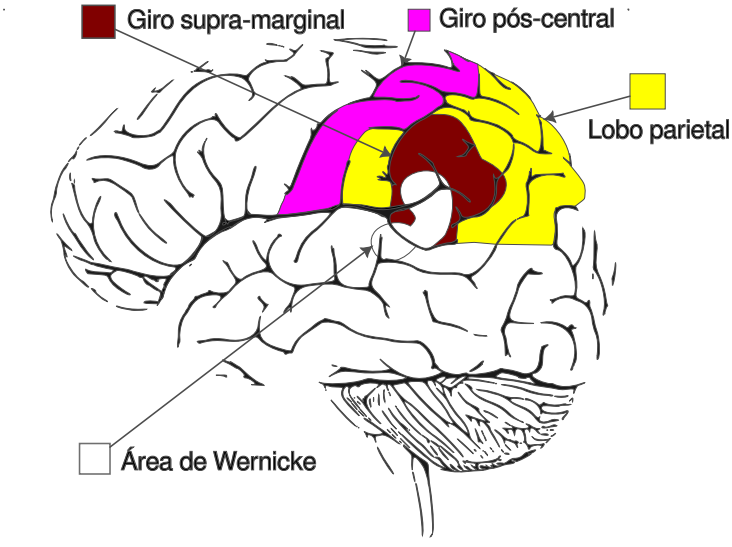
Fonte: Elaborado pelo autor, 2024.

Figura 15 – Áreas relacionadas a fala no lobo frontal: A área de Broca geralmente se encontra no hemisfério esquerdo.



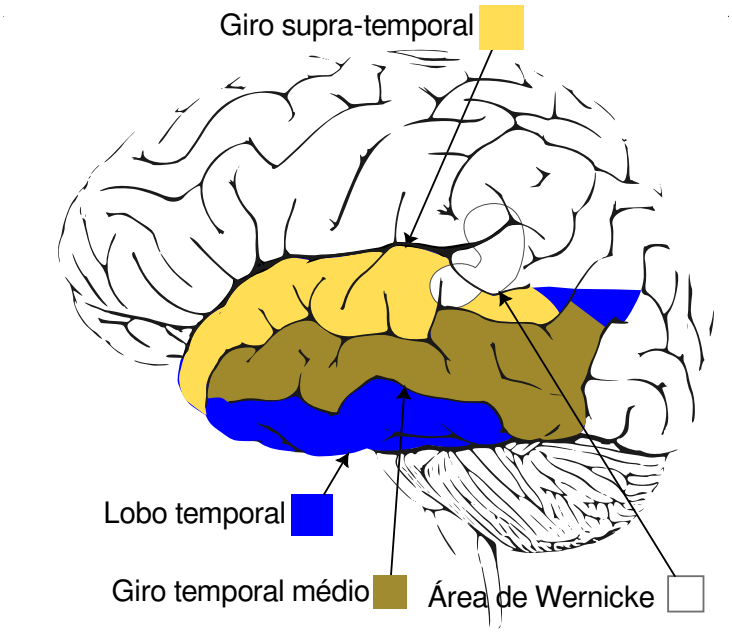
Fonte: Elaborado pelo autor, 2024.

Figura 16 – Áreas relacionadas a fala no lobo parietal: A área de Wernicke também ocupa parte do lobo temporal e geralmente se encontra no hemisfério esquerdo.



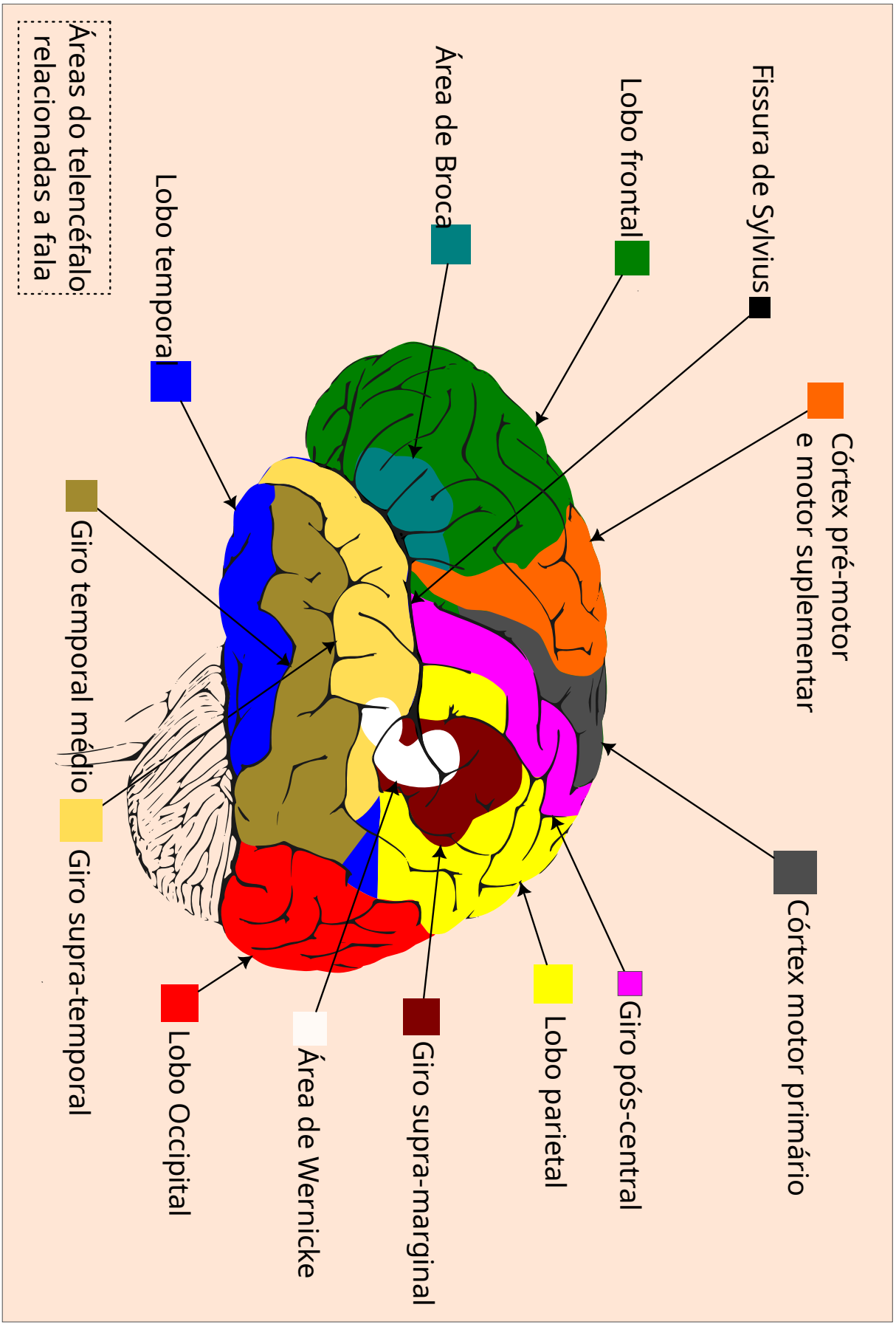
Fonte: Elaborado pelo autor, 2024.

Figura 17 – Áreas relacionadas a fala no lobo temporal: A área de Wernicke também ocupa parte do lobo parietal e geralmente se encontra no hemisfério esquerdo.



Fonte: Elaborado pelo autor, 2024.

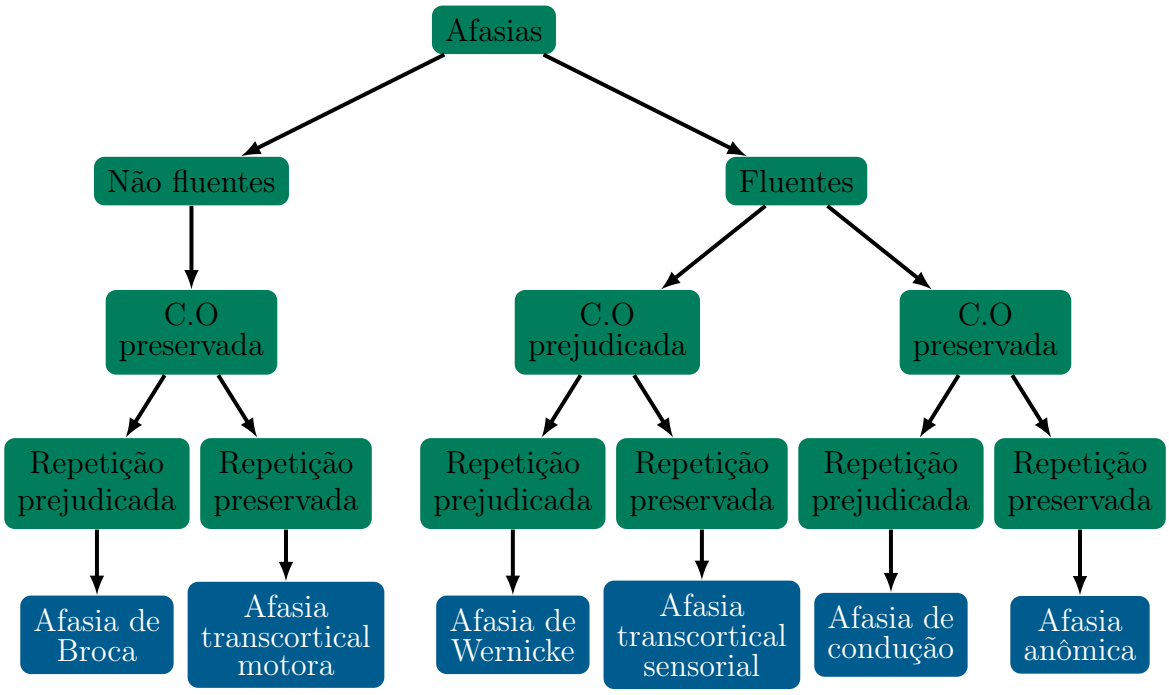
Figura 18 – Visão geral do telencéfalo e as áreas relacionadas à fala, tais regiões avizinham-se da fissura de Sylvius, sendo então chamadas de perissilvianas. O lobo occipital não contêm regiões relacionadas a fala (PINTO, 2012)



Fonte: Elaborado pelo autor, 2024.

O processamento fonológico se dá na região perissilviana do hemisfério dominante, já o semântico inclui áreas corticais de ambos os hemisférios para dar significado às palavras, nas áreas frontais, temporais e parietais, a articulação das palavras acontece. Quando se dá uma lesão em áreas como a de Wernicke, Broca ou em suas vizinhanças à condição resultante se dá o nome de **afasia** que podem ser fluentes (fala com ritmo normal porém incoerente) ou não-fluentes (dificuldade e esforço na produção da fala mas, preservam a compreensão) (PINTO, 2012). Portanto a fala tanto fonada quanto imaginada se realiza em quase todo o telencéfalo tendo como diferença a mobilização dos córtex motores quando o locutor ou locutora emite algum som.

Figura 19 – Afasias e seus sintomas: Sendo C.O. a compreensão oral. As afasias são resultado de lesões: Na área de Broca (Afasia de Broca), nas regiões superiores ou anteriores a área de Broca (Afasia transcortical motora), na área de Wernicke (Afasia de Wernicke) e finalmente ao redor da área de Wernicke (Afasia transcortical sensorial).



Fonte: Adaptado de (PINTO, 2012).

2.1.9 Normalização de Características de Entrada

Não é possível somar metros com quilogramas sem antes convertê-los a uma unidade comum; uma rede neural tem o mesmo problema com características de entrada em escalas diferentes. Se uma característica varia entre 0 e 1 e outra entre 0 e 10 000, o gradiente da segunda domina a atualização dos pesos, e a rede aprende com ela, ignorando a primeira. Reescalonar as entradas para uma faixa comparável, com média próxima de zero, remove essa assimetria e estabiliza e acelera o treinamento por gradiente (LECUN *et al.*, 1998).

Para alimentar os classificadores, este trabalho usa duas estratégias de padronização (*z-score*), uma para o áudio e outra para o EEG, porque as duas modalidades têm comportamentos estatísticos opostos, detalhados a seguir. Uma terceira normalização — a escala mín-máx para o intervalo $[0, 1]$ — é usada em contexto distinto: a engenharia paraconsistente de características (seção 2.1) exige que todos os componentes dos vetores estejam em $[0, 1]$ para que os coeficientes α e β fiquem bem definidos; essa escala é aplicada apenas na etapa de avaliação paraconsistente, não na entrada dos classificadores.

Antes de apresentar as fórmulas das duas padronizações *z-score*, definem-se os símbolos comuns:

- $x_{i,j}$ — valor bruto (não normalizado) na amostra/linha i e característica/coluna j;
- $x'_{i,j}$ — valor normalizado correspondente;
- ε — constante pequena de estabilidade numérica, que evita divisão por zero quando a dispersão calculada é nula (padrão 10^{-6}).

2.1.9.1 Normalização por característica (áudio)

Em um vetor de características de áudio, cada coluna tem significado físico fixo. Como o significado da coluna não muda, sua escala também não muda de amostra para amostra: é uma propriedade da característica, não da amostra.

A estratégia decorrente é: medir, uma vez, a média e o desvio-padrão de cada coluna usando os dados de treino, e reutilizar essas duas constantes para normalizar qualquer amostra, de treino ou de teste. Sejam μ_j a média e σ_j o desvio-padrão da característica (coluna) j, calculados sobre M amostras de treino:

$$x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}, \qquad \mu_j = \frac{1}{M}\sum_{i=1}^{M} x_{i,j}, \qquad \sigma_j^2 = \frac{1}{M}\sum_{i=1}^{M}(x_{i,j} - \mu_j)^2, \tag{2.20}$$

μ_j e σ_j são calculados uma vez, a partir do treino, e depois reaplicados sem recálculo a qualquer amostra nova — o porquê está na subseção 2.1.9.3. A técnica é conhecida na literatura de reconhecimento de fala como **normalização de média e variância no domínio cepstral** (*Cepstral Mean and Variance Normalization*, CMVN), usada para tornar sistemas de reconhecimento robustos a variações de canal e ruído (VIIKKI; LAURILA, 1998).

2.1.9.2 Normalização por janela (EEG)

O EEG tem o problema oposto. A amplitude de um sinal de EEG não é uma propriedade estável do canal: ela muda de sessão para sessão e de sujeito para sujeito,

por impedância variável dos eletrodos, ganho diferente do amplificador e deslocamento da linha de base (LOTTE *et al.*, 2018). Duas gravações do mesmo canal, em sessões diferentes, podem ter amplitudes diferentes mesmo representando o mesmo tipo de atividade cerebral.

A estratégia do áudio não se aplica aqui: um par (μ, σ) fixo, calculado no treino, não prevê nem corrige o deslocamento de uma sessão nova. A solução inverte a lógica — ao invés de normalizar **por coluna** usando estatísticas fixas do treino, normaliza-se **por linha** (por janela), recalculando a média e o desvio-padrão a cada janela, no momento em que ela é processada. Seja F o número de pontos (características) dentro da janela i:

$$x'_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}, \qquad \mu_i = \frac{1}{F}\sum_{j=1}^{F} x_{i,j}, \qquad \sigma_i^2 = \frac{1}{F}\sum_{j=1}^{F}(x_{i,j} - \mu_i)^2, \tag{2.21}$$

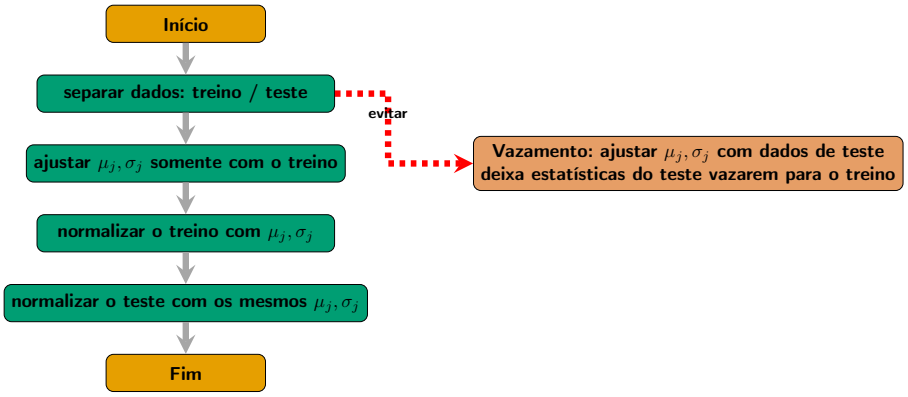
Cada janela corrige seu próprio nível DC e sua própria escala, o que torna o resultado robusto a variações entre sessões, ao custo da amplitude absoluta do sinal — informação pouco confiável no EEG bruto (LOTTE *et al.*, 2018).

2.1.9.3 Evitando vazamento de dados

Imagine estudar para uma prova já sabendo as respostas certas: o desempenho na prova deixaria de refletir o quanto se aprendeu. É isso que acontece quando as estatísticas de normalização são calculadas usando dados que, depois, servirão para avaliar o modelo — **vazamento de dados** (*data leakage*): informação do conjunto de teste influenciando, ainda que indiretamente, o modelo treinado, o que infla as métricas de validação (KAUFMAN *et al.*, 2012).

Por isso, na normalização por característica (Equação 2.20), μ_j e σ_j são calculados exclusivamente com dados de treino e depois aplicados, sem reajuste, a amostras de teste. A Figura 20 ilustra o fluxo correto e o erro a evitar. A normalização por janela do EEG (Equação 2.21) está livre desse risco por construção: como nenhuma estatística global é ajustada a partir do treino, cada janela — de treino ou de teste — é normalizada de forma independente e idêntica.

Figura 20 – Fluxograma de ajuste (*fit*) e aplicação (*transform*) da normalização por característica: as estatísticas μ_j, σ_j são calculadas exclusivamente com o conjunto de treino e depois aplicadas, sem reajuste, tanto ao treino quanto ao teste. Ajustar as estatísticas usando dados de teste (caminho tracejado) causa vazamento de dados.



Fonte: Elaborado pelo autor, 2026.

2.1.9.4 Exemplo numérico

Áudio (por característica). Seja a característica j observada em $M = 4$ amostras de treino: $x_{\cdot,j} = [2,\ 4,\ 4,\ 6]$. Tem-se $\mu_j = (2+4+4+6)/4 = 4$ e $\sigma_j^2 = \left[(2-4)^2+(4-4)^2+(4-4)^2+(6-4)^2\right]/4 = 8/4 = 2$, logo $\sqrt{\sigma_j^2+\varepsilon} \approx 1{,}4142$. Uma amostra de teste com $x_{i,j} = 5$ (nunca vista no ajuste) é normalizada com essas mesmas constantes: $x'_{i,j} = (5-4)/1{,}4142 \approx 0{,}7071$.

EEG (por janela). Seja uma janela com $F = 4$ pontos: $x_{i,\cdot} = [10,\ 12,\ 14,\ 16]\,\mu V$. Tem-se $\mu_i = 13$ e $\sigma_i^2 = \left[(10-13)^2+(12-13)^2+(14-13)^2+(16-13)^2\right]/4 = 20/4 = 5$, logo $\sqrt{\sigma_i^2+\varepsilon} \approx 2{,}2361$. Cada ponto é normalizado com as estatísticas **da própria janela**: $x'_{i,1} = (10-13)/2{,}2361 \approx -1{,}3416$, e assim por diante — se a próxima janela chegar com um deslocamento de linha de base de $+50\,\mu V$ (artefato comum entre sessões), ela recalcula sua própria μ_i e o resultado normalizado permanece na mesma escala, sem estatística herdada da janela anterior.

2.1.10 Redes Neurais de Pulso (Spiking Neural Networks)

Redes Neurais (RNs), conforme definido aqui como *uma rede multicamadas, totalmente conectada, com ou sem camadas recorrentes ou convolucionais*, exigem que todos os neurônios sejam ativados tanto na fase de *feed-forward* quanto na de *backpropagation*. Isso implica que cada unidade na rede deve processar alguns dados, resultando em consumo de energia (ESHRAGHIAN *et al.*, 2023).

Em contrapartida o neurônio biológico dispara apenas quando um certo nível de sinais excitatórios (voltagem) se acumula acima de um limiar em seu citoplasma, per-

manecendo inativo quando não há sinal, portanto, esse tipo de processamento é muito eficiente em termos de consumo de energia.

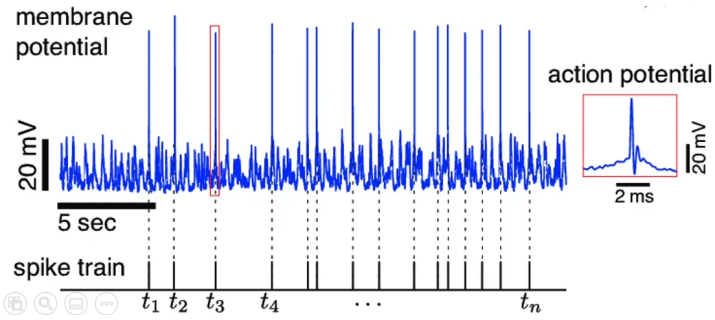
O sistema sensorial dos sistemas neurológicos biológicos converte dados externos, como luz, odores, toque, sabores e outros, em pulsos. Um pulso é uma alteração na voltagem que é propagada transmitindo informações (KASABOV, 2019). Esses pulsos são então transmitidos ao longo da cadeia neuronal sendo processados, gerando uma resposta ao ambiente.

Para obter as vantagens mencionadas, as RNP[4] assim como seus referenciais biológicos, em vez de empregar valores de ativação contínuos, como as RNs, utilizam **pulsos** em suas camadas. As RNPs também podem ter entradas contínuas e manter suas propriedades.

Uma RNP **não é** uma simulação um-para-um de neurônios. Em vez disso, ela aproxima certas capacidades computacionais de propriedades biológicas específicas que serão explicadas mais a diante. Caso haja interesse em algo do tipo existem modelos mais biologicamente precisos como o **neurônio Hodgkin-Huxley** (GERSTNER *et al.*, 2014) ou ainda outros que exploram a não linearidade dos dendritos e outras características neurais (JONES; KORDING, 2020) criando modelos muito mais próximos aos neurônios naturais, obtendo resultados bastante interessantes.

Como pode ser visto na Figura 21, os neurônios das RNPs, com os parâmetros corretos, são muito tolerantes a ruídos porque atuam como um **filtro passa-baixa**. Eles geram pulsos mesmo quando um nível considerável de interferência está presente. Tais neurônios são muito sensíveis ao tempo, sendo ótimos para processar fluxos de dados (ESHRAGHIAN *et al.*, 2023).

Figura 21 – Sinal ruidoso e os pulsos produzidos por uma **simulação** de neurônio de pulso (RNP/LIF) em resposta a esse sinal, ilustrando o comportamento de disparo por limiar discutido no texto. Note que é necessário um acúmulo de sinal suficiente antes que os pulsos comecem a ser exibidos



Fonte: (GOODMAN *et al.*, 2022)

[4] Redes Neurais de Pulso

2.1.10.1 Neurônio de Pulso

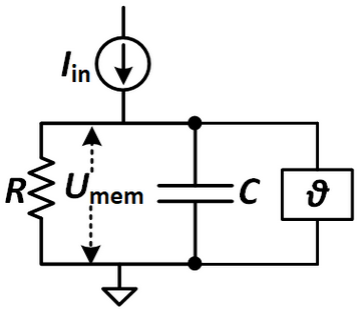
O foco deste trabalho é nos neurônios LIF[5] porque são mais simples, mais eficientes e atualmente generalizam melhor para a maioria dos problemas (GOODMAN *et al.*, 2022).

2.1.10.2 Entendendo e modelando o LIF

O LIF é um dos modelos de neurônios mais simples em RNPs, ainda assim, pode ser aplicado com sucesso na maioria dos problemas em que as RNPs podem ser usadas, tal estrutura, assim como um neurônio de RN, recebe a soma das entradas ponderadas, mas, em vez de passá-lo diretamente para sua função de ativação, algum *vazamento* é aplicado, diminuindo em algum grau o valor soma com o passar do tempo.

O LIF se assemelha com circuitos RC[6], como pode ser visto na Figura 22. Aqui, R é a resistência ao vazamento da corrente, I_{in} é a corrente de entrada, C é a capacitância, U_{mem} representa o potencial acumulado e ϑ é um interruptor que permite que o capacitor se descarregue (ou seja, emita um pulso) quando um determinado limiar de potencial é alcançado.

Figura 22 – O modelo RC



Fonte: (ESHRAGHIAN *et al.*, 2023)

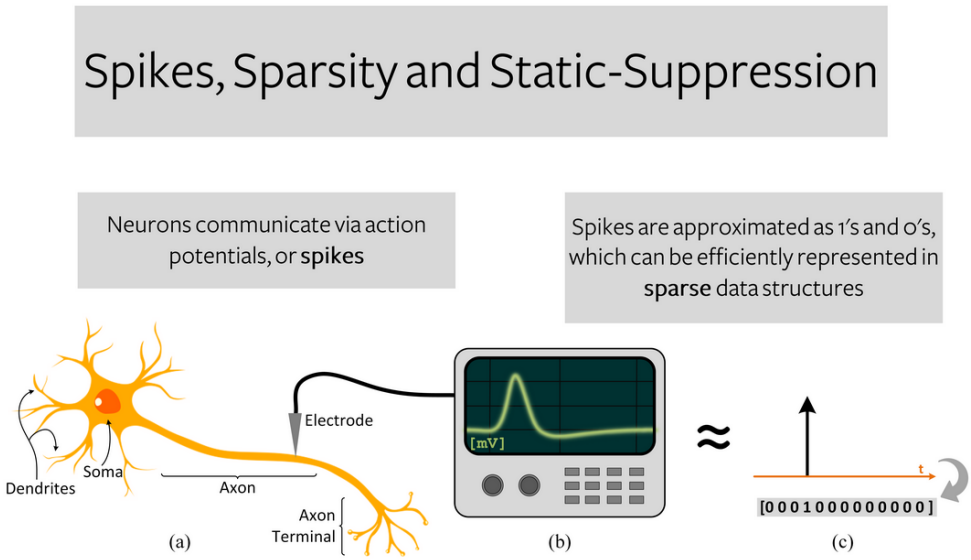
Ao contrário do neurônio Hodgkin-Huxley, os pulsos são representados como **uns** dispersamente distribuídos em uma sequência de **zeros**, como ilustrado na Figura 23 e Figura 24. Essa abordagem simplifica os modelos e reduz a potência computacional necessária em hardware convencional e neuromórfico além de requerer menos armazenamento.

Como resultado do mencionado acima, em RNP, a informação é codificada no formato de *tempo* e/ou *taxa* de pulsos, proporcionando consequentemente grandes capacidades de processamento de fluxos de dados, mas limitando o processamento de dados estáticos.

O modelo LIF é governado pelas equações abaixo (ESHRAGHIAN *et al.*, 2023).

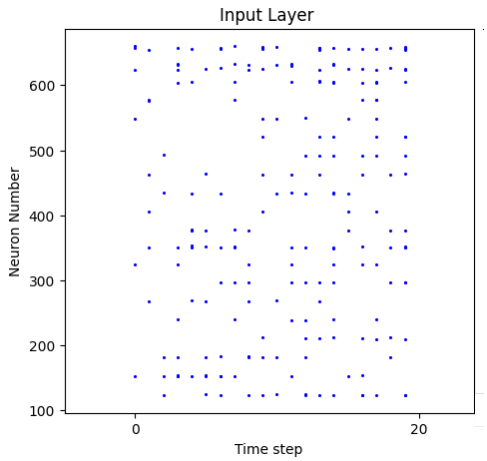
[5] *Leaky Integrate and Fire* (Integra-e-Dispara com Vazamento)
[6] Resistor-Capacitor

Figura 23 – Dispersão em Redes Neurais de Pulsos.



Fonte: (ESHRAGHIAN *et al.*, 2023)

Figura 24 – Atividade dispersa de uma simulação RNP: O eixo horizontal representa o momento no qual os dados estão sendo processados e o vertical representa o número do neurônio (índice) na RNP. Note que, na maior parte do tempo, muito poucos neurônios são ativados.



Fonte: Elaborado pelo autor, 2023.

Considerando que Q é uma medida de carga elétrica e $V_{\text{mem}}(t)$ é a diferença de potencial na membrana em um determinado tempo t, então a capacitância do neurônio C é dada pela Equação 2.22.

$$C = \frac{Q}{V_{mem}(t)} \tag{2.22}$$

Portanto, a carga do neurônio pode ser expressa pela Equação 2.23.

$$Q = C.V_{mem}(t) \tag{2.23}$$

Para saber como essa carga muda ao longo do tempo (ou seja, medir a corrente), podemos derivar Q como na Equação 2.24. Essa expressão representa a corrente I_C na parte capacitiva do neurônio.

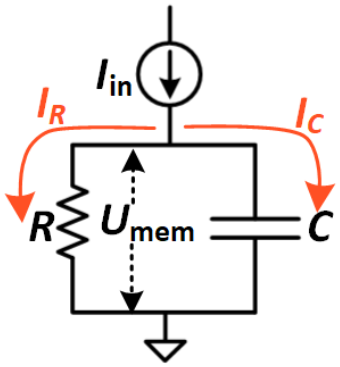
$$I_C = \frac{dQ}{dt} = C.\frac{dV_{mem}(t)}{dt} \tag{2.24}$$

Para calcular a corrente total passando pela parte resistiva do circuito, podemos usar a lei de Ohm:

$$V_{mem}(t) = R.I_R \implies I_R = \frac{V_{mem}(t)}{R} \tag{2.25}$$

Então, considerando que a corrente total não muda, como pode ser visto na Figura 25, temos a corrente total de entrada I_{in} do neurônio como na Equação 2.26.

Figura 25 – Modelo RC para correntes: $I_{in} = I_R + I_C$



Fonte: (ESHRAGHIAN *et al.*, 2023)

$$I_{in}(t) = I_R + I_C \implies I_{in}(t) = \frac{V_{mem}(t)}{R} + C.\frac{dV_{mem}(t)}{dt} \tag{2.26}$$

Portanto, para descrever a variação de potencial da membrana passiva, temos a Equação 2.27.

$$\begin{gathered} I_{in}(t) = \frac{V_{mem}(t)}{R} + C.\frac{dV_{mem}(t)}{dt} \implies \\ I_{in}(t) - \frac{V_{mem}(t)}{R} = C.\frac{dV_{mem}(t)}{dt} \implies \\ \boxed{R.I_{in}(t) - V_{mem}(t) = R.C.\frac{dV_{mem}(t)}{dt}} \end{gathered} \tag{2.27}$$

Já que o lado direito da equação $\left(\frac{dV_{mem}(t)}{dt}\right)$ expressa uma grandeza em $voltagem/s$ e o esquerdo $(R.I_{in}(t) - V_{mem}(t))$ em $voltagem$ apenas, conclui-se pela igualdade, que $R.C$ é uma unidade de tempo. Definindo $\tau = R.C$ como a **constante de tempo da membrana**, obtemos tensões em ambos os lados chegando na Equação 2.28 que **descreve o circuito RC**.

$$\begin{aligned} R.I_{in}(t) - V_{mem}(t) &= R.C.\frac{dV_{mem}(t)}{dt} \implies \\ R.I_{in}(t) - V_{mem}(t) &= \tau.\frac{dV_{mem}(t)}{dt} \implies \\ \boxed{\tau.\frac{dV_{mem}(t)}{dt} &= R.I_{in}(t) - V_{mem}(t)} \end{aligned} \tag{2.28}$$

Exemplo numérico. Para $R = 5\,\Omega$ e $C = 1\,F$ (os mesmos valores usados nas implementações a seguir), a constante de tempo é $\tau = R.C = 5 \times 1 = 5$.

Quando não há corrente de entrada no neurônio $I_{in} = 0$ ocorre um decaimento em sua voltagem que pode ser modelado, além disso, R e C não mudam e podem ser resumidos em $\tau = R.C$ que é uma constante. Portanto, atribuindo um potencial inicial $V_{mem}(0)$, se obtêm uma curva exponencial, como visto na Equação 2.29.

$$\begin{aligned} \tau.\frac{dV_{mem}(t)}{dt} &= \cancel{R.I_{in}(t)} - V_{mem}(t) \implies \\ \tau.\frac{dV_{mem}(t)}{dt} &= -V_{mem}(t) = \\ \frac{dV_{mem}(t)}{dt} &= \frac{-V_{mem}(t)}{\tau} = \\ \frac{dV_{mem}(t)}{V_{mem}(t)} &= \frac{-dt}{\tau} = \\ \int \frac{1}{V_{mem}(t)}.dV_{mem}(t) &= \int \frac{-1}{\tau}.dt = \\ \ln(|V_{mem}(t)|) &= -\frac{1}{\tau}.\int dt = \\ \ln(|V_{mem}(t)|) &= -\frac{1}{\tau}.t = \\ e^{\ln(|V_{mem}(t)|)} &= e^{-\frac{t}{\tau}} = \\ V_{mem}(t) &= e^{-\frac{t}{\tau}} \implies \end{aligned} \tag{2.29}$$

Considerando um valor inicial de $V_{mem}(t) = V_{mem}(0)$ em $t = 0 \implies$

$$\boxed{V_{mem}(t) = V_{mem}(0).e^{-\frac{t}{\tau}}}$$

Exemplo numérico. Com $\tau = 5$ (do exemplo acima) e $V_{mem}(0) = 1$, o potencial em $t = 1$ é $V_{mem}(1) = 1 \times e^{-1/5} = e^{-0,2} \approx 0{,}8187$. Repetindo a operação

implementada no Algoritmo 2.1, a discretização de Euler com $dt = 1$ dá $V_{mem}(1) = V_{mem}(0) + \frac{dt}{\tau}(-V_{mem}(0)) = 1 + \frac{1}{5}(-1) = 0{,}8$, próximo do valor exato 0,8187.

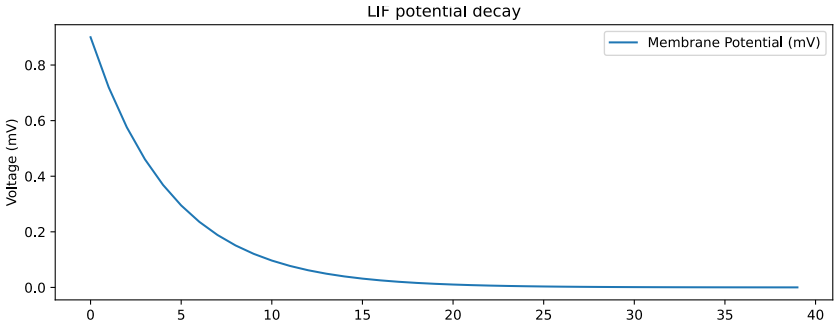
Então, pode-se dizer que: Na ausência de uma entrada I_{in}, o potencial da membrana decai exponencialmente, como ilustrado na Figura 26 e implementado no Algoritmo 2.1.

```
double lif(double V_mem, double dt = 1, double I_in = 0, double R = 5,
    double C = 1) {

  double tau = R * C;
  V_mem = V_mem + (dt / tau) * (-V_mem + I_in * R);
  return V_mem;
}
```

Algoritmo 2.1 – Implementação em C++ da decaimento do potencial de ação de um LIF: $I_{in} = 0$

Figura 26 – Decaimento do potencial da membrana.



Fonte: Elaborado pelo autor, 2023.

Com os resultados da Equação 2.28, é possível calcular o aumento da voltagem, na presença de uma corrente de entrada, conforme mostrado na Equação 2.30.

$$\tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) = \frac{dV_{mem}(t)}{dt} + \frac{V_{mem}(t)}{\tau} = \frac{R.I_{in}(t)}{\tau} \tag{2.30}$$

A Equação 2.30 se encaixa na categoria de equação diferencial ordinária (EDO) de primeira ordem, portanto, para resolvê-la se determina o fator de integração f definido na Equação 2.31:

$$
\begin{aligned}
f = e^{\int P(x)dx} \mid \frac{dy}{dx} + P(x).y = Q(x) \therefore \\
x = t \\
y = V_{mem}(t) \\
P(x) = \frac{1}{\tau} \\
Q(x) = \frac{R.I_{in}}{\tau} \implies \\
e^{\int \frac{1}{\tau}dx} = e^{\frac{1}{\tau}.\int dt} = \boxed{e^{\frac{t}{\tau}}}
\end{aligned}
\tag{2.31}
$$

Fazendo a substituição do fator na Equação 2.32:

$$
\begin{aligned}
\text{Substituindo em: } (f.y)' = f.Q(x) \implies \\
(e^{\frac{t}{\tau}}.V_{mem}(t))' = \frac{R.I_{in}(t)}{\tau}.e^{\frac{t}{\tau}} \implies \\
\int (e^{\frac{t}{\tau}}.V_{mem}(t))' = \int \frac{R.I_{in}(t)}{\tau}.e^{\frac{t}{\tau}}dt \implies \\
\boxed{e^{\frac{t}{\tau}}.V_{mem}(t) = \frac{R.I_{in}(t)}{\tau}.\int e^{\frac{t}{\tau}}dt}
\end{aligned}
\tag{2.32}
$$

Que resolvendo pelo método da substituição na Equação 2.33. É importante manter a constante de integração c_1 até o fim: descartá-la precocemente anula o termo exponencial e leva a um resultado que não decai com t, contradizendo o comportamento físico esperado.

$$
\begin{aligned}
&u = \frac{t}{\tau} \implies \frac{du}{dt} = \frac{1}{\tau} \implies dt = du.\tau \therefore \\
&\int e^u du.\tau = \tau.\int e^u du = \tau.e^u + c_1 = \tau.e^{\frac{t}{\tau}} + c_1 \implies \\
&e^{\frac{t}{\tau}}.V_{mem}(t) = \frac{R.I_{in}(t)}{\tau}\left(\tau.e^{\frac{t}{\tau}} + c_1\right) = R.I_{in}(t).e^{\frac{t}{\tau}} + c_2 \implies \\
&V_{mem}(t) = R.I_{in}(t) + c_2.e^{-\frac{t}{\tau}} \implies \\
&\text{Considerando: } V_{mem}(t=0) = 0 \implies 0 = R.I_{in} + c_2 \implies c_2 = -R.I_{in} \implies \\
&\boxed{V_{mem}(t) = I_{in}(t).R\left(1 - e^{-\frac{t}{\tau}}\right)}
\end{aligned}
\tag{2.33}
$$

Exemplo numérico. Usando os mesmos valores do Algoritmo 2.2 ($R = 5$, $C = 1$, logo $\tau = R.C = 5$) com $I_{in} = 1$ e $V_{mem}(0) = 0$, o potencial em $t = 1$ pela solução exata é

$$V_{mem}(1) = 1 \times 5 \times \left(1 - e^{-1/5}\right) = 5 \times (1 - 0{,}8187) = 5 \times 0{,}1813 \approx 0{,}9066.$$

Já a iteração discreta do Algoritmo 2.2, com passo $dt = 1$, calcula $V_{mem}(1) = V_{mem}(0) + \frac{dt}{\tau}\big(-V_{mem}(0) + I_{in}.R\big) = 0 + \frac{1}{5}(0 + 5) = 1{,}0$: um passo de Euler tão grosseiro ($dt = \tau$) já

se aproxima da assíntota $R.I_{in} = 5$, superestimando ligeiramente o valor exato 0,9066 — a diferença entre as duas soluções é retomada na Equação 2.35.

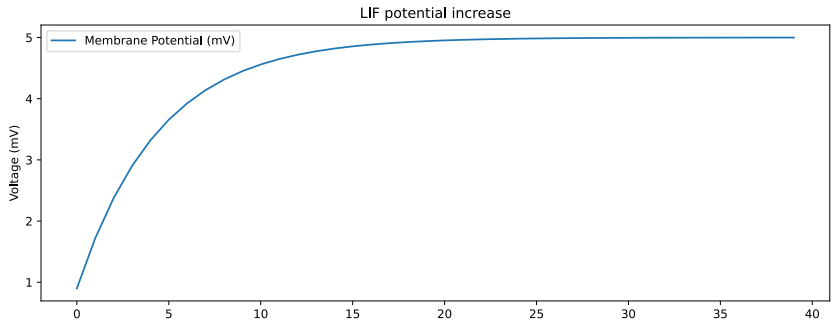
Note que quando os potenciais de ação aumentam, ainda há um comportamento exponencial, como se pode notar na Figura 27 e no Algoritmo 2.2.

```
double lif(double V_mem, double dt = 1, double I_in = 1, double R = 5,
    double C = 1) {

  double tau = R * C;
  V_mem = V_mem + (dt / tau) * (-V_mem + I_in * R);
  return V_mem;
}
```

Algoritmo 2.2 – Implementação em C++ do potencial de ação decrescente de um LIF: $I_{in} = 1$

Figura 27 – Aumento do potencial da membrana.



Fonte: Elaborado pelo autor, 2023.

Levando em consideração um certo **limiar** que determina um *reset* na voltagem do neurônio e dois tipos de *resets* (para zero e subtração do limiar), finalmente é possível modelar o comportamento completo do LIF, conforme ilustrado na Figura 28 e implementado no Algoritmo 2.3:

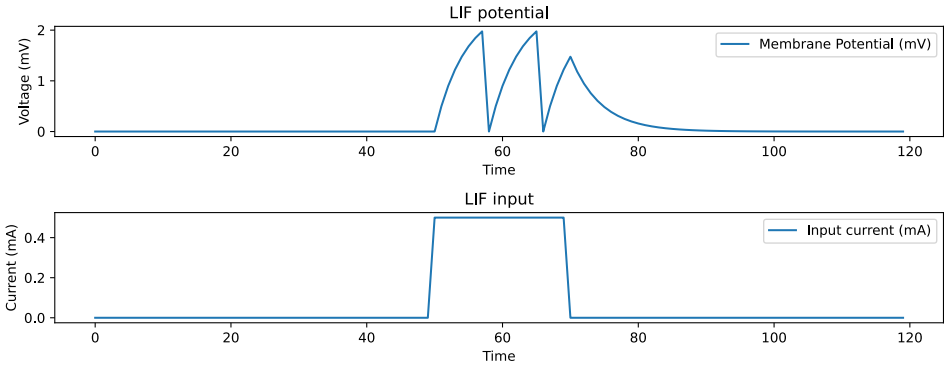
```
double lif(double V_mem, double dt = 1, double I_in = 1, double R = 5,
    double C = 1, double V_thresh = 2, bool reset_zero = true) {

  double tau = R * C;
  V_mem = V_mem + (dt / tau) * (-V_mem + I_in * R);
  if (V_mem > V_thresh) {
    if (reset_zero) {
      V_mem = 0;
    } else {
      V_mem = V_mem - V_thresh;
    }
  }
```

```
12   return V_mem;
13 }
```

Algoritmo 2.3 – Implementação em C++ da simulação completa do potencial de ação de um LIF: $I_{in} = 1$, $V_{thresh} = 2$ is threshold

Figura 28 – Gráfico do LIF simulado completo: Foram fornecidos 0.5 mA de corrente no intervalo de tempo de 51 a 70.



Fonte: Elaborado pelo autor, 2023.

2.1.10.3 Outra interpretação do LIF

Começando com a Equação 2.28 e usando o Método de Euler para resolver o modelo LIF:

$$\tau . \frac{dV_{\text{mem}}(t)}{dt} = R.I_{\text{in}}(t) - V_{\text{mem}}(t) \tag{2.34}$$

Resolvendo a derivada na Equação 2.35, obtemos o potencial da membrana para qualquer tempo $t + \Delta t$ no futuro:

$$\begin{aligned} \tau . \frac{V_{\text{mem}}(t+\Delta t) - V_{\text{mem}}(t)}{\Delta t} &= R.I_{\text{in}}(t) - V_{\text{mem}}(t) = \\ V_{\text{mem}}(t+\Delta t) - V_{\text{mem}}(t) &= \frac{\Delta t}{\tau} . (R.I_{\text{in}}(t) - V_{\text{mem}}(t)) = \\ \boxed{V_{\text{mem}}(t+\Delta t) &= V_{\text{mem}}(t) + \frac{\Delta t}{\tau} . (R.I_{\text{in}}(t) - V_{\text{mem}}(t))} \end{aligned} \tag{2.35}$$

Exemplo numérico. Com $R = 5$, $\tau = 5$, $I_{\text{in}} = 1$, $\Delta t = 1$ e partindo de $V_{\text{mem}}(0) = 0$: $V_{\text{mem}}(1) = 0 + \frac{1}{5}(5 \times 1 - 0) = 1{,}0$. Este é exatamente o passo de Euler já calculado ao final da seção anterior — a Equação 2.35 é a regra geral por trás daquela iteração, e do Algoritmo 2.3.

Nota de implementação. A Equação 2.35 e o Algoritmo 2.3 têm propósito didático: expõem o circuito RC subjacente e sua discretização por Euler explícito, na

qual a corrente de entrada é escalada por R. A implementação do neurônio LIF de fato usada para gerar todos os resultados experimentais deste trabalho — tanto na forma de passo único quanto na variante desenrolada no tempo usada para BPTT (próxima seção) — adota, em vez disso, a recorrência exata do decaimento exponencial por passo:

$$V_{\text{mem}}(t + \Delta t) = \beta \cdot V_{\text{mem}}(t) + I_{\text{in}}(t), \qquad \beta = e^{-\Delta t/\tau}, \qquad \tau = R.C. \tag{2.36}$$

Essa forma é a solução exata da parte homogênea (o decaimento) da Equação 2.28 a cada passo — mais precisa que o passo de Euler linear $(1 - \Delta t/\tau)$ da Equação 2.35 — e corresponde à convenção usual da literatura de RNPs treináveis, também adotada pela biblioteca *snnTorch* (ESHRAGHIAN *et al.*, 2023), usada como referência de corretude para essa camada durante o desenvolvimento deste trabalho. Diferentemente da Equação 2.35, R não escala $I_{\text{in}}(t)$ nessa recorrência — R (junto com C) influencia **apenas** a taxa de decaimento β via $\tau = R.C$; a corrente de entrada é somada diretamente ao potencial já decaído. Isso é uma simplificação deliberada em relação ao circuito RC ideal (cuja solução exata seria $V_{\text{mem}}(t + \Delta t) = \beta \cdot V_{\text{mem}}(t) + R.I_{\text{in}}(t).(1 - \beta)$), não um erro de discretização: por não reintroduzir o termo $R.(1 - \beta)$, o ganho efetivo do sinal de entrada permanece constante e independente de τ, o que simplifica o treinamento de R, C e V_{th} como parâmetros aprendíveis sem acoplar sua otimização à escala da entrada.

2.1.10.4 Treinamento

Como as RNPs são treinadas? Esta ainda é uma questão com múltiplas respostas. Um neurônio RNP tem o comportamento de sua função de ativação parecido com uma **função de passo**. Portanto, em princípio, não podemos usar soluções baseadas em descida de gradiente porque esse tipo de função **não é** diferenciável (KASABOV, 2019).

Mas existem algumas ideias que podem lançar alguma luz sobre este assunto: algumas observações *in vivo/in vitro* mostram que os cérebros, em geral, aprendem fortalecendo/enfraquecendo e adicionando/removendo sinapses, criando novos neurônios ou adotando até mesmo outros mecanismos como pacotes de RNA; contudo, segundo (KASABOV, 2019), há outras formas empregadas:

- **STDP**[7]: Se um neurônio pré-sináptico dispara **antes** do pós-sináptico, há um fortalecimento na conexão, mas se o neurônio pós-sináptico disparar antes, então há um enfraquecimento.
- **Descida de Gradiente Emprestada**: Aproxima a função de passo usando outra função matemática, que é diferenciável (como uma sigmoide), para treinar a rede. Essas aproximações são usadas apenas na fase de *backpropagation*, enquanto mantêm o comportamento original na fase do *feed-forward*.

[7] Plasticidade Dependente do Tempo de Pulsos (*Spike-Timing-Dependent Plasticity*)

- **Algoritmos Evolutivos**: Usam a seleção dos hiperparâmetros mais aptos ao longo de muitas gerações de redes.

- **Reservatório/Computação Dinâmica**: estratégia que evita treinar a rede recorrente mantendo o **reservatório** — uma rede recorrente com pesos e conectividade fixos — e treinando apenas a camada de saída (KASABOV, 2019).

- **ESN e LSM**: as **Redes de estado de eco** (*Echo State Networks* — ESN) usam neurônios de saída contínua no reservatório; as **Máquinas de estado líquido** (*Liquid State Machines* — LSM) usam neurônios pulsantes no reservatório e uma camada de saída simples que lê apenas o estado atual (MAASS; NATSCHLäGER; MARKRAM, 2002).

2.1.10.5 BPTT em RNPs

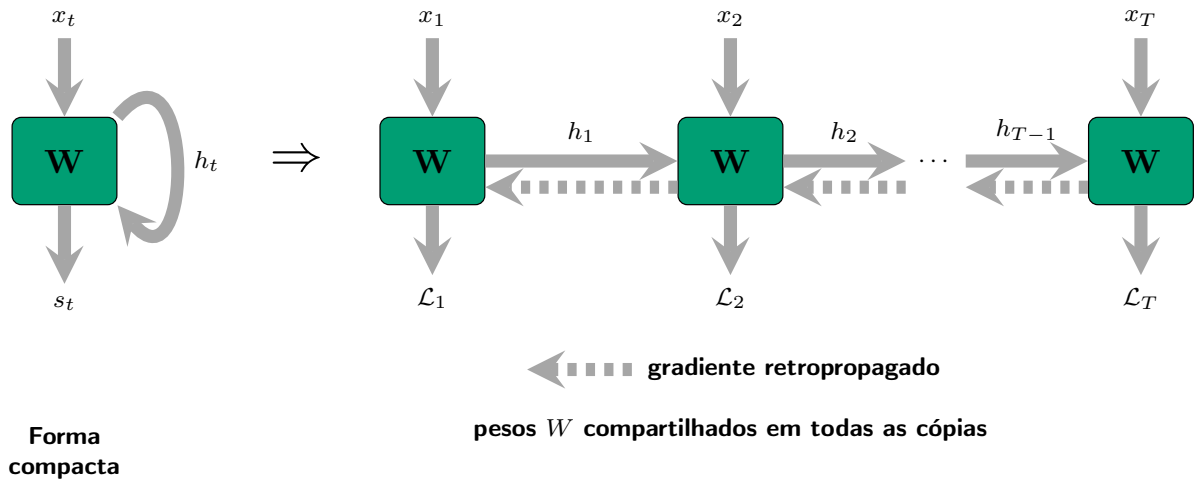
Redes recorrentes processam sequências: a saída em cada instante t depende da entrada atual x_t e do estado interno h_{t-1} da etapa anterior. Isso cria dependências temporais que a retropropagação padrão não trata.

O BPTT resolve isso *desenrolando* a rede no tempo, como ilustrado na Figura 29: a rede é replicada T vezes, sendo T o passo temporal, com todas essas cópias compartilhando os mesmos pesos W. Na propagação direta (*forward*), o estado interno avança da esquerda para a direita: $h_t = f(h_{t-1}, x_t; W)$. Seja $\mathcal{L}$ a perda total da sequência e $\mathcal{L}_t$ a contribuição do passo t para essa perda. Na retropropagação (*backward*), o gradiente percorre o caminho inverso (da última cópia até a primeira) acumulando a contribuição de cada passo (ESHRAGHIAN *et al.*, 2023):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}} \tag{2.37}$$

Exemplo numérico. Para uma sequência com $T = 3$ passos e gradientes por passo $\partial\mathcal{L}_1/\partial W = 0{,}2$, $\partial\mathcal{L}_2/\partial W = -0{,}5$ e $\partial\mathcal{L}_3/\partial W = 0{,}9$, o gradiente total acumulado é $\partial\mathcal{L}/\partial W = 0{,}2 - 0{,}5 + 0{,}9 = 0{,}6$.

Figura 29 – Retropropagação Através do Tempo: à esquerda, rede recorrente na forma compacta com conexão recorrente h_t; à direita, a mesma rede *desenrolada* em T cópias. Setas azuis: propagação direta; setas vermelhas tracejadas: retropropagação do gradiente. Os pesos W são compartilhados em todas as cópias.



Fonte: Elaborado pelo autor, 2025.

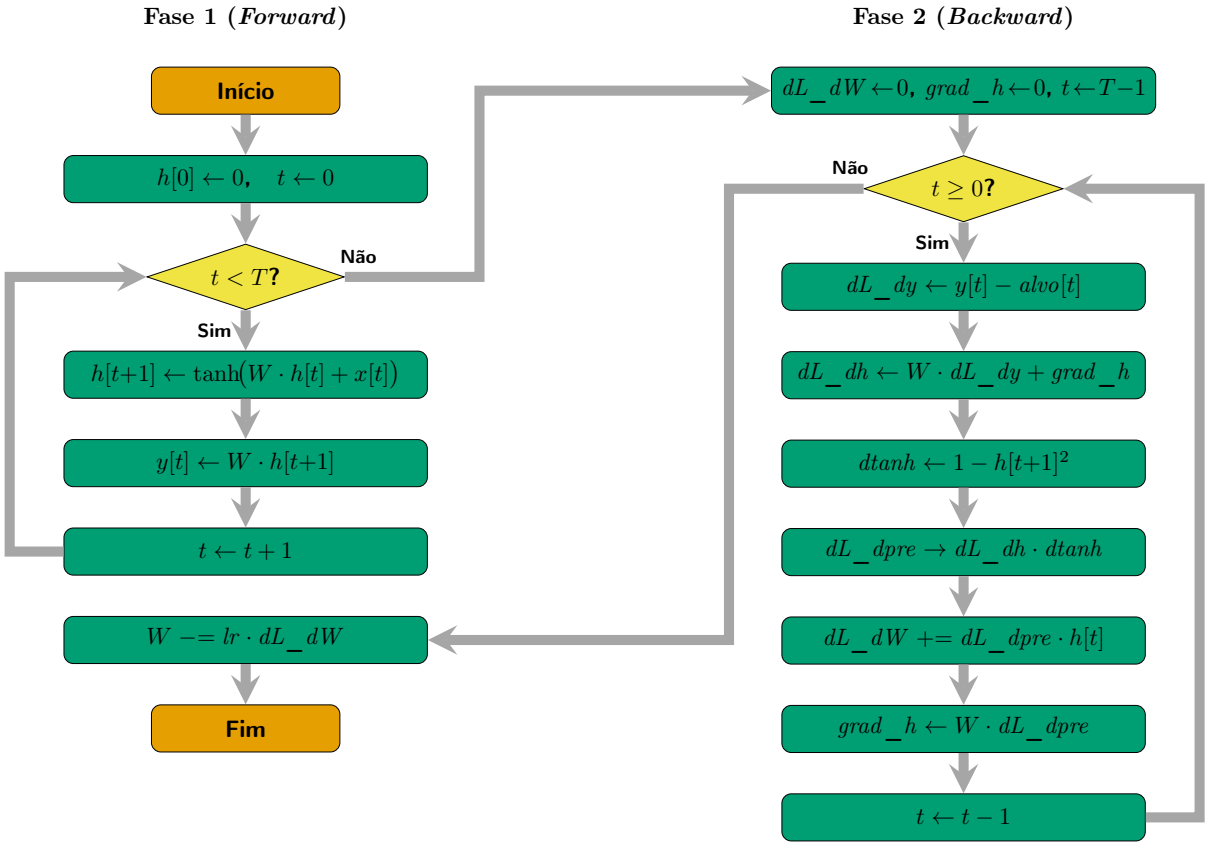
O Algoritmo 2.4 apresenta o algoritmo simplificado: o loop *forward* armazena os estados $h[t]$ na memória; o loop *backward* percorre o tempo ao contrário acumulando o gradiente ∇_W. Neste exemplo, W denota os pesos da rede, $y[t] = W \cdot h[t{+}1]$ a saída da rede no instante t, $alvo[t]$ o valor alvo correspondente e lr a taxa de aprendizagem. A Figura 30 detalha o fluxo de execução completo com as duas fases e todas as variáveis envolvidas.

```
/* ---- Forward: armazena estados ---- */
float h[T+1];
h[0] = 0.0f;
for (int t = 0; t < T; t++) {
    h[t+1] = tanh(W * h[t] + x[t]); /* recorrencia  */
    y[t]   = W *h[t+1];             /* saida        */
}
/* ---- Backward: percorre ao contrario ---- */
float dL_dW = 0.0f;
float grad_h = 0.0f;  /* gradiente acumulado da etapa seguinte */
for (int t = T-1; t >= 0; t--) {
    float dL_dy   = y[t] - alvo[t];        /* erro na saida       */
    float dL_dh   = W *dL_dy + grad_h;    /* output + recorrencia*/
    float dtanh   = 1.0f - h[t+1] * h[t+1];/* derivada de tanh    */
    float dL_dpre = dL_dh * dtanh;
    dL_dW  += dL_dpre * h[t];  /* acumula gradiente de W              */
    grad_h  = W * dL_dpre;     /* propaga para a etapa anterior (t-1) */
}
```

```
19 W -= lr * dL_dW;   /* atualiza pesos */
```

Algoritmo 2.4 – BPTT simplificado: propagação direta seguida de retropropagação no tempo.

Figura 30 – Fluxograma do algoritmo BPTT simplificado. Fase 1 (*forward*): computa e armazena $h[t]$ e $y[t]$ para cada passo t. Fase 2 (*backward*): percorre os passos em ordem inversa, acumula o gradiente $\partial\mathcal{L}/\partial W$ e propaga $grad_h$ para a etapa anterior.



Fonte: Elaborado pelo autor, 2025.

2.1.10.6 Gradiente substituto

Antes da fórmula, definem-se os símbolos: $\tilde{S}$ é o pulso aproximado (a versão suave e diferenciável da função de disparo, usada apenas no *backward*); V_{mem} é o potencial de membrana já apresentado; V_{th} é o limiar de disparo do neurônio; e $\beta > 0$ é um hiperparâmetro que controla a inclinação da aproximação em torno de V_{th}. O gradiente substituto exponencial utilizado neste trabalho é definido como (ESHRAGHIAN *et al.*, 2023):

$$\frac{\partial \tilde{S}}{\partial V_{mem}} = \beta \cdot e^{-\beta |V_{mem} - V_{th}|} \tag{2.38}$$

Exemplo numérico. Com $\beta = 5$ e $V_{th} = 1{,}0$: exatamente no limiar ($V_{mem} = 1{,}0$) o gradiente vale seu máximo, $5 \times e^{0} = 5{,}0$; a 0,2 do limiar ($V_{mem} = 1{,}2$) ele cai para $5 \times e^{-5\times 0{,}2} = 5 \times e^{-1} \approx 5 \times 0{,}3679 = 1{,}8394$ — ilustrando o decaimento rápido da influência do gradiente à medida que o potencial se afasta do limiar de disparo.

Valores maiores de β produzem uma função mais próxima da derivada da função em degrau, enquanto valores menores resultam em uma aproximação mais suave e numericamente estável. A função é centrada em V_{th} e decai exponencialmente ao se afastar do ponto de disparo, ponderando mais os gradientes próximos ao limiar do neurônio e atenuando contribuições distantes do mesmo.

2.1.10.7 Normalização em Lote Dependente do Limiar (tdBN)

A **tdBN**[8] é uma camada de normalização proposta por (ZHENG *et al.*, 2021) para viabilizar o treinamento direto de RNPs profundas. Trata-se de uma adaptação da BN[9] (IOFFE; SZEGEDY, 2015) ao domínio pulsante.

Motivação. A BN clássica normaliza uma pré-ativação para média zero e variância unitária, $N(0, 1)$, considerando apenas a dimensão do lote. Em uma RNP, contudo, a corrente de membrana de um neurônio é uma distribuição sobre as amostras **e** sobre os T passos de tempo, e o disparo depende da comparação dessa corrente com o limiar V_{th}. Normalizar para variância unitária é, portanto, cego ao limiar: se $V_{th} \neq 1$, a entrada normalizada ou raramente cruza o limiar (poucos disparos) ou o satura (disparos em excesso) (ZHENG *et al.*, 2021). Além disso, ao empilhar muitas camadas pulsantes, o potencial de membrana — e o gradiente retropropagado — tende a desaparecer ou explodir, o que impede o treinamento de redes profundas e pode levar ao **problema da ausência de disparos** (*No-Spike Problem*), em que uma camada inteira deixa de disparar e, como o gradiente substituto só é não-nulo em torno do limiar, deixa de ser ajustada (ZHENG *et al.*, 2021).

Formulação matemática. Seja a pré-ativação em formato *tempo-maior* $X \in \mathbb{R}^{(T\cdot B)\times F}$, em que T é o número de passos de tempo, B é o tamanho do lote e F é o número de canais (características). Para o canal k, sejam $X_{k,i}$ os seus $N = T \cdot B$ valores agrupados sobre o lote e o tempo. Definem-se a média e a variância **por canal**, agrupadas sobre lote e tempo:

$$\mu_k = \frac{1}{N}\sum_{i=1}^{N} X_{k,i}, \qquad \sigma_k^2 = \frac{1}{N}\sum_{i=1}^{N} \left(X_{k,i} - \mu_k\right)^2, \tag{2.39}$$

onde μ_k é a média do canal k e σ_k^2 a sua variância. A normalização e a transformação afim dependente do limiar são

[8] *Threshold-Dependent Batch Normalization* (Normalização em Lote Dependente do Limiar)
[9] *Batch Normalization* (Normalização em Lote)

$$\hat{X}_{k,i} = \frac{X_{k,i} - \mu_k}{\sqrt{\sigma_k^2 + \varepsilon}}, \qquad Y_{k,i} = \gamma_k \left(\alpha\, V_{th}\, \hat{X}_{k,i}\right) + \beta_k, \tag{2.40}$$

em que $\hat{X}_{k,i}$ é o valor padronizado (média zero, variância unitária); ε é uma constante de estabilidade numérica (padrão 10^{-5}); V_{th} é o limiar de disparo do neurônio LIF a jusante; α é um hiperparâmetro que define a dispersão alvo (padrão $\alpha = 1$); e γ_k e β_k são, respectivamente, a escala e o deslocamento aprendíveis do canal k. Note-se que o fator αV_{th} multiplica **apenas** o termo normalizado — o deslocamento β_k não é escalonado. Com $\gamma_k = 1$ e $\beta_k = 0$, a saída segue a distribuição $N\big(0, (\alpha V_{th})^2\big)$, propriedade que define a tdBN (ZHENG *et al.*, 2021).

Derivação. A tdBN decorre de duas modificações sobre a BN. Primeiro, as estatísticas (Equação 2.39) são calculadas sobre $N = T \cdot B$ valores — lote e tempo agrupados — pois todos eles influenciam decisões de disparo; isso distingue a tdBN de esquemas por passo, como a BNTT (KIM; PANDA, 2021). Segundo, deseja-se que uma fração fixa de neurônios dispare, independentemente da profundidade. Se $\hat{X} \sim N(0,1)$, então $\alpha V_{th}\hat{X} \sim N(0, (\alpha V_{th})^2)$ e a probabilidade de ultrapassar o limiar é

$$P\big(\alpha V_{th}\, \hat{X} > V_{th}\big) = P\Big(\hat{X} > \tfrac{1}{\alpha}\Big) = 1 - \Phi\big(\tfrac{1}{\alpha}\big)\,, \tag{2.41}$$

onde Φ é a função de distribuição acumulada da normal padrão. Essa fração depende **somente** de α — não da profundidade, de T ou da escala da entrada. Para $\alpha = 1$ obtém-se $1 - \Phi(1) \approx 15{,}9\%$ de neurônios acima do limiar, um regime de disparo saudável e não-saturado. (ZHENG *et al.*, 2021) demonstram ainda, via teoria da *Block Dynamical Isometry*, que esse fator αV_{th} mantém a norma do gradiente de cada bloco próxima da unidade, evitando o seu desaparecimento ou explosão e permitindo o treinamento de redes com dezenas de camadas.

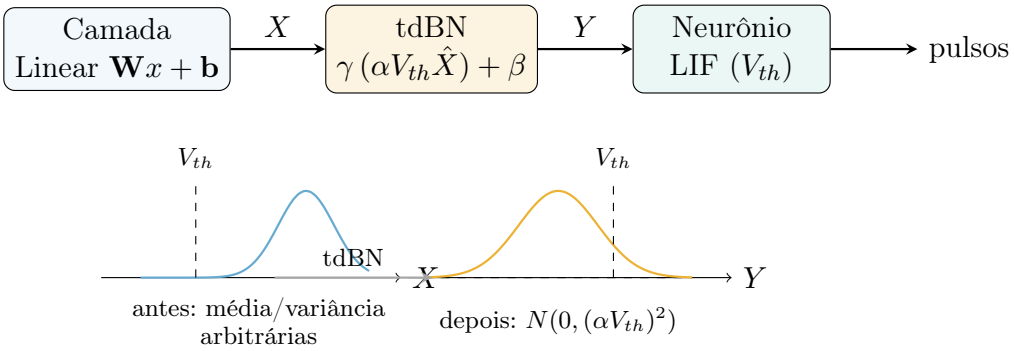
Exemplo numérico. Considere-se um único canal ($F = 1$), com $T = 2$, $B = 2$ (logo $N = 4$), $\gamma = 1$, $\beta = 0$ e $\varepsilon = 10^{-5}$. Sejam as entradas $X = [\,0,\ 2,\ 4,\ 6\,]$ (cujos dois passos de tempo têm médias distintas, 1 e 5, ilustrando o agrupamento sobre o tempo). Tem-se:

$$\begin{aligned} \mu &= (0+2+4+6)/4 = 3, \\ \sigma^2 &= \big[(0-3)^2 + (2-3)^2 + (4-3)^2 + (6-3)^2\big]/4 = 20/4 = 5, \\ 1/\sqrt{\sigma^2+\varepsilon} &\approx 0{,}447214, \\ \hat{X} &= (X-3)\cdot 0{,}447214 = [-1{,}3416,\ -0{,}4472,\ 0{,}4472,\ 1{,}3416]. \end{aligned} \tag{2.42}$$

Aplicando a escala dependente do limiar com $\alpha = 1$: para $V_{th} = 1$ (escala 1) obtém-se $Y = [-1{,}3416,\ -0{,}4472,\ 0{,}4472,\ 1{,}3416]$, ou seja, $N(0,1)$; para $V_{th} = 2$ (escala

2) obtém-se $Y = [-2{,}6833,\ -0{,}8944,\ 0{,}8944,\ 2{,}6833]$, com desvio-padrão exatamente igual a $\alpha V_{th} = 2$, isto é, $N(0,4) = N(0,(\alpha V_{th})^2)$. A Figura 31 ilustra esse efeito: a tdBN recentra e reescala a corrente de modo que uma fração controlada da distribuição ultrapasse o limiar.

Figura 31 – Normalização em Lote Dependente do Limiar (tdBN): inserida entre a camada Linear e o neurônio LIF, ela agrupa as estatísticas sobre lote e tempo e reescala a corrente para a distribuição $N(0,(\alpha V_{th})^2)$, mantendo uma fração controlada de neurônios acima do limiar V_{th} a cada passo.



Fonte: Elaborado pelo autor, 2026.

Vantagens. Por compartilhar γ e β entre os passos de tempo, a transformação afim da tdBN pode ser **incorporada aos pesos** da camada Linear anterior em tempo de inferência, sem custo computacional adicional — ao contrário de esquemas com parâmetros por passo, como BNTT (KIM; PANDA, 2021) e TEBN (DUAN *et al.*, 2022) (ZHENG *et al.*, 2021). Além disso, ao fixar a dispersão em αV_{th}, a tdBN preserva a norma do gradiente em profundidade e mitiga estruturalmente o problema da ausência de disparos.

Limitações. Um único par escala/deslocamento por canal é menos expressivo do que esquemas que atribuem parâmetros distintos a cada passo de tempo; para valores muito pequenos de T, a TEBN (DUAN *et al.*, 2022) pode superá-la. A tdBN também pressupõe uma população de lote e tempo suficientemente grande para estimativas estáveis de média e variância. Variantes posteriores, como a MPBN (GUO *et al.*, 2023), normalizam o potencial de membrana após a sua atualização, antes da função de disparo.

2.1.10.8 Regularização da taxa de disparo

Enquanto a tdBN (subseção 2.1.10.7) ataca o problema da ausência de disparos **estruturalmente** — reescalando a corrente para que uma fração fixa de neurônios ultrapasse o limiar, independentemente da profundidade — uma segunda estratégia, complementar, atua sobre a **função de perda**: penalizar a taxa média de disparo de cada camada pulsante quando ela sai de uma faixa-alvo $[r_{min}, r_{max}]$. Diferente da tdBN, essa penalidade não garante estruturalmente uma taxa de disparo saudável — ela apenas

empurra o gradiente nessa direção — mas dispensa qualquer camada de normalização adicional e se aplica mesmo quando a rede pulsante já está construída sem tdBN, como é o caso do *autoencoder* de RNP deste trabalho (subseção 2.1.11.5).

Formulação. Seja $S_l \in \mathbb{R}^{N_l}$ o tensor de pulsos (0 ou 1) emitido pela camada pulsante l ao longo do lote e do tempo, com N_l elementos. A taxa de disparo dessa camada é a sua média:

$$r_l = \frac{1}{N_l} \sum_{i=1}^{N_l} S_{l,i}. \tag{2.43}$$

A penalidade, somada à perda de reconstrução (ou de classificação), é uma função de banda quadrática — nula dentro da faixa-alvo, crescendo quadraticamente fora dela:

$$\mathcal{L}_{fr} = \lambda \sum_{l} \Big[\max(0,\, r_{min} - r_l)^2 + \max(0,\, r_l - r_{max})^2 \Big], \tag{2.44}$$

em que $\lambda \geq 0$ é o peso da regularização (inerte quando $\lambda = 0$), r_{min} é a borda inferior da faixa (guarda contra neurônios mortos) e r_{max} é a borda superior (guarda contra disparo excessivo, que satura a seletividade do neurônio). Sua derivada em relação a cada pulso individual é constante dentro de uma mesma camada:

$$\frac{\partial \mathcal{L}_{fr}}{\partial S_{l,i}} = \frac{2\lambda\big(r_l - \mathrm{clamp}(r_l,\, r_{min},\, r_{max})\big)}{N_l}, \qquad \forall i, \tag{2.45}$$

ou seja, o efeito da regularização sobre o *backward* é somar essa mesma constante a todo elemento do gradiente que chega na saída da camada pulsante l, imediatamente antes de retropropagá-lo através do neurônio LIF. Quando r_l já está dentro da faixa, o gradiente adicional é zero e a regularização não interfere no treinamento.

Exemplo numérico. Considere $\lambda = 0{,}1$, faixa $[r_{min}, r_{max}] = [0{,}05,\ 0{,}80]$ e uma camada com $N_l = 1024$ elementos disparando a uma taxa $r_l = 0{,}02$ (abaixo da borda inferior, sinal de colapso). O termo de clamp vale $\mathrm{clamp}(0{,}02;\, 0{,}05;\, 0{,}80) = 0{,}05$, e o gradiente adicional é

$$\frac{2 \times 0{,}1 \times (0{,}02 - 0{,}05)}{1024} = \frac{2 \times 0{,}1 \times (-0{,}03)}{1024} \approx -5{,}86 \times 10^{-6}, \tag{2.46}$$

um empurrão negativo (uniforme, em todo elemento do gradiente) que incentiva o otimizador a aumentar a corrente de entrada da camada, elevando a taxa de disparo até que ela reentre na faixa-alvo.

Limitação conhecida. A penalidade opera sobre a taxa **média** da camada, um sinal agregado. Uma camada pode satisfazer $r_l \in [r_{min}, r_{max}]$ mesmo com uma distribuição desigual entre neurônios — alguns disparando muito, outros nunca — sem que a média o

revele. A eficácia depende também de r_{min} estar acima do ponto de equilíbrio para o qual o treinamento convergeria de outra forma: se a rede já converge, sem regularização, para uma taxa dentro da faixa (o que ocorre, por exemplo, com a taxa observada na inicialização de He/Kaiming — subseção 2.1.11.5), a penalidade permanece inerte durante boa parte do treinamento e só atua quando a taxa realmente cruza a borda.

2.1.10.9 Inicialização de Pesos

Antes do treinamento, os pesos de cada camada precisam receber valores iniciais. Essa escolha não é um detalhe: se os pesos começam grandes demais, os sinais crescem a cada camada até saturar (explosão); se começam pequenos demais, os sinais encolhem até desaparecer (desvanecimento). Em ambos os casos o gradiente também degenera e a rede não aprende (GLOROT; BENGIO, 2010). Uma boa inicialização mantém a variância do sinal aproximadamente constante ao longo da profundidade.

Antes de apresentar a fórmula, definem-se os símbolos utilizados:

- W — matriz de pesos de uma camada, com elemento W_{ij};
- n_{in} — número de entradas de cada neurônio (o *fan-in*, isto é, o número de colunas de W);
- $\mathcal{U}(a, b)$ — distribuição uniforme contínua no intervalo $[a, b]$, da qual cada peso é sorteado;
- ℓ — o limite que delimita esse intervalo;
- b — o vetor de vieses (*bias*) da camada.

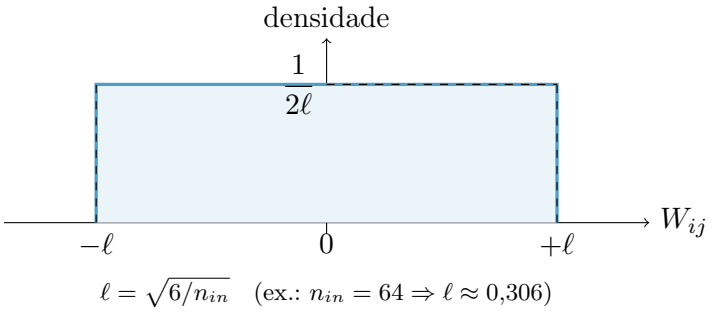
Este trabalho utiliza a inicialização de **He** (também chamada de **Kaiming**), na sua forma uniforme, indicada para ativações do tipo retificadora e para os neurônios de pulso (HE *et al.*, 2015b). Cada peso é sorteado de forma independente e os vieses começam em zero:

$$\ell = \sqrt{\frac{6}{n_{in}}}, \qquad W_{ij} \sim \mathcal{U}(-\ell,\, +\ell), \qquad b = 0. \tag{2.47}$$

A escolha de $\ell = \sqrt{6/n_{in}}$ não é arbitrária. Para a distribuição uniforme em $[-\ell, \ell]$, a variância de cada peso é $\ell^2/3 = 2/n_{in}$. Esse valor é justamente o que (HE *et al.*, 2015b) demonstram manter a variância das ativações estável entre camadas quando a não-linearidade descarta cerca de metade dos sinais (como o retificador ou o disparo do neurônio). Assim, quanto mais entradas um neurônio possui (maior n_{in}), menor o intervalo de sorteio — evitando que a soma de muitas entradas cresça sem controle.

Exemplo numérico. Para uma camada com $n_{in} = 64$ entradas, tem-se $\ell = \sqrt{6/64} = \sqrt{0{,}09375} \approx 0{,}306$. Portanto cada peso é sorteado uniformemente no intervalo $[-0{,}306,\ +0{,}306]$, como ilustra a Figura 32, e todos os vieses iniciam em 0.

Figura 32 – Distribuição uniforme dos pesos na inicialização de He/Kaiming: cada peso é igualmente provável dentro de $[-\ell, +\ell]$, com $\ell = \sqrt{6/n_{in}}$, e densidade constante igual a $1/(2\ell)$.



Fonte: Elaborado pelo autor, 2026.

Reprodutibilidade. O sorteio dos pesos usa um gerador de números pseudoaleatórios. Se esse gerador for iniciado a partir de uma **semente** fixa, a mesma sequência de pesos é obtida a cada execução, tornando os experimentos reprodutíveis. Neste trabalho, ambos os classificadores — a rede residual e a Rede Neural de Pulso — semeiam o gerador com a semente do experimento, de modo que uma mesma configuração produz sempre os mesmos pesos iniciais. O Algoritmo 2.5 descreve o procedimento em C simplificado, e a Figura 33 apresenta o mesmo procedimento como fluxograma.

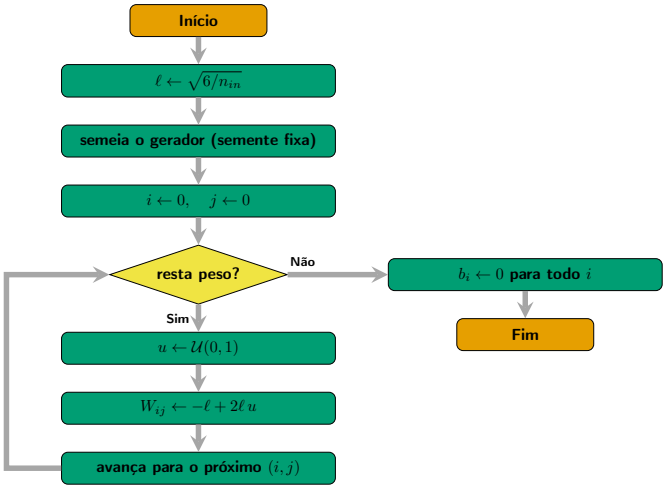
```c
/* n_in   : numero de entradas do neuronio (fan-in)   */
/* n_out  : numero de neuronios da camada             */
/* semente: valor fixo -> mesmos pesos a cada execucao */
void inicializa_kaiming(float W[n_out][n_in], float b[n_out],
                        int n_in, int n_out, unsigned int semente)
{
    float limite = sqrtf(6.0f / (float) n_in); /* l = raiz(6 / n_in) */
    semeia_gerador(semente);                   /* torna o sorteio
   determinista */

    for (int i = 0; i < n_out; i++) {
        for (int j = 0; j < n_in; j++) {
            float u = sorteia_uniforme_0_1();   /* u em [0, 1)
   */
            W[i][j] = -limite + 2.0f * limite * u; /* u em [-l, +l]
   */
        }
        b[i] = 0.0f;                           /* vies comeca em zero
   */
    }
```

```
}
```

Algoritmo 2.5 – Inicialização de He/Kaiming uniforme de uma camada, com gerador semeado para reprodutibilidade.

Figura 33 – Fluxograma da inicialização de He/Kaiming: calcula-se o limite ℓ, semeia-se o gerador, percorrem-se todos os pesos sorteando cada um em $[-\ell, +\ell]$ e, ao final, zeram-se os vieses.



Fonte: Elaborado pelo autor, 2026.

Vale registrar que a inicialização de He é uma evolução da inicialização de **Xavier** (ou Glorot), que usa $\ell = \sqrt{6/(n_{in} + n_{out})}$ e foi concebida para ativações simétricas como a tangente hiperbólica (GLOROT; BENGIO, 2010). A forma de He ajusta o fator para não-linearidades retificadoras, sendo por isso a mais adequada às redes deste trabalho.

2.1.10.10 Taxa de aprendizado por grupo de parâmetros

Uma RNP treinada por retropropagação tem dois tipos de parâmetro qualitativamente distintos: os pesos e vieses das camadas Lineares, de um lado, e os parâmetros biofísicos do neurônio LIF (R, C e V_{th}), do outro. Os primeiros só aparecem multiplicando ou somando o sinal; os segundos entram na constante de tempo $\tau = R \cdot C$ (Equação 2.28) e no próprio limiar de disparo — uma atualização grande demais pode levar τ a um valor absurdo ou deslocar V_{th} para fora da faixa em que o neurônio dispara de forma útil. Por essa razão, este trabalho usa uma taxa de aprendizado menor para o segundo grupo: um fator de escala interno de 0,1 aplicado sobre a taxa global, isto é, $\eta_{\text{biofísico}} \approx 10^{-4}$ quando $\eta_{\text{global}} = 10^{-3}$. Registra-se aqui, por honestidade, que esse fator 0,1 é uma escolha de engenharia deste projeto — não um valor extraído de um estudo específico da literatura; uma citação anteriormente associada a essa escolha não pôde ser verificada e foi removida da documentação do código (Wiki, `Core/Optimizers.md`).

Mecanismo. O otimizador Adam implementado aceita um multiplicador de taxa de aprendizado por parâmetro: dado o vetor achatado de parâmetros do modelo $\theta_1, \ldots, \theta_n$ e um vetor de escalas $s_1, \ldots, s_n$, a taxa efetiva do parâmetro i é

$$\eta_i = \eta_{\text{global}} \times s_i \,, \tag{2.48}$$

de modo que $s_i = 1$ mantém a taxa global e $s_i = 0{,}1$ aplica a redução — em ambos os casos o restante da atualização de Adam (médias móveis dos gradientes, correção de viés) segue idêntico. A intenção de projeto era popular esse vetor com $s_i = 0{,}1$ exatamente nas posições correspondentes a R, C e V_{th}, e $s_i = 1$ em todas as demais.

O defeito e sua descoberta. A implementação original não distinguia as posições do vetor θ: preenchia s_i com o mesmo fator para **todos** os parâmetros do modelo, sempre que o fator fosse diferente de 1. O resultado não era uma taxa de aprendizado por grupo — era um multiplicador global disfarçado, reduzindo em 10× a taxa de aprendizado de *toda* a rede, pesos incluídos, sempre que a redução biofísica estava habilitada. O defeito só veio à tona durante a investigação do latente colapsado descrita em subseção 2.1.5.1: ao rastrear por que uma variante do *autoencoder* de pulso convergia para uma saída constante mesmo com uma taxa de aprendizado nominalmente razoável, ficou claro que a taxa *efetivamente aplicada aos pesos* era dez vezes menor que a declarada no perfil do experimento — o que os dados publicados descreviam não era o que, de fato, havia sido executado.

O alcance do defeito se estende além do ramo de *autoencoders* pulsantes desta tese: o mesmo mecanismo de escala por grupo é usado pelo experimento de comparação SNN-*vs*-LSTM (artigo de congresso, ainda em rascunho), cujos perfis de SNN também habilitam a redução biofísica. Nesse experimento, a rede LSTM de comparação — sem parâmetros biofísicos — treinava com a taxa de aprendizado declarada por completo, enquanto a rede de pulso, pelo mesmo defeito, treinava **todos os seus pesos** a um décimo dela — uma desvantagem estrutural não-intencional contra a própria contribuição do artigo, introduzida não pelo método proposto, mas por uma falha de implementação do mecanismo de escala.

Correção. Rotular explicitamente quais posições do vetor achatado de parâmetros são biofísicas exigiria alterar a interface comum de todas as camadas da base de código. Em vez disso, a correção aproveita uma regularidade já verdadeira na implementação: R, C e V_{th} são, em todo neurônio LIF do projeto, tensores escalares 1×1, ao passo que nenhum peso ou viés jamais o é — o mesmo critério de tamanho que a implementação de decaimento de peso (*weight decay* desacoplado) já usava para excluir vieses e parâmetros biofísicos do próprio decaimento. A construção do vetor de escalas passou a testar o tamanho de cada parâmetro:

$$s_i = \begin{cases} 0{,}1 & \text{se } \theta_i \text{ tem exatamente 1 elemento} \\ 1{,}0 & \text{caso contrário} \end{cases} \tag{2.49}$$

sem qualquer mudança na interface comum das camadas. Um teste de regressão foi acrescentado à suíte de testes do treinador genérico: um modelo sintético com um parâmetro 1×1 e um parâmetro 2×2, ambos recebendo o mesmo gradiente fixo a cada passo, confirma que, após um passo do Adam, o parâmetro 1×1 se desloca por $\eta_{\text{global}} \times 0{,}1$ e o 2×2 pelo η_{global} completo — exatamente a distinção que o defeito original violava.

Situação no momento da escrita. A correção está no código e validada pelo teste de regressão acima, mas os resultados já publicados — tanto os 300 *rankings* da Fase 00 desta tese quanto as tabelas do artigo de comparação SNN-*vs*-LSTM — foram produzidos **antes** da correção, sob o comportamento antigo. Reexecutá-los para refletir a taxa de aprendizado corrigida é uma decisão em aberto, não tomada até o fechamento deste texto: trata-se de experimentos longos (horas, por configuração), e o artigo, ainda em rascunho, é o caso de maior urgência — sua tabela comparativa atual favorece indevidamente a rede LSTM.

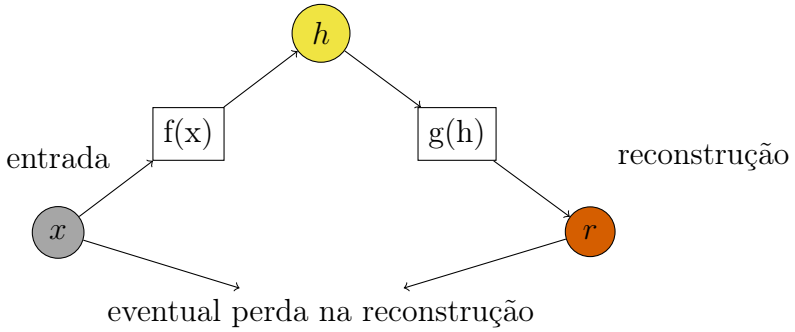
2.1.11 *Autoencoders*

Como ilustrado na Figura 34 *autoencoders* são redes neurais treinadas para reconstruir seus dados de entrada. Elas consistem respectivamente em: Uma função codificadora $f(x)$ que gera um estado h, denotada como $h = f(x)$ e uma função decodificadora $g(x)$ que produz uma reconstrução r, denotada como $r = g(h)$. O estado h representa um **código**, uma representação da entrada que pode ser comprimida ou não (GOODFELLOW; BENGIO; COURVILLE, 2016). Tais funções se traduzem em camadas como ilustrado na Figura 35 cujos neurônios são todos unidades ativas com ou sem funções de ativação. É importante destacar que um *autoencoder* pode ser tão raso como o mostrado na Figura 34.

O principal objetivo de um *autoencoder* é aprender uma representação dos dados de entrada na camada de **código** e, em seguida, reconstruir os dados de entrada com a maior precisão possível usando o decodificador.

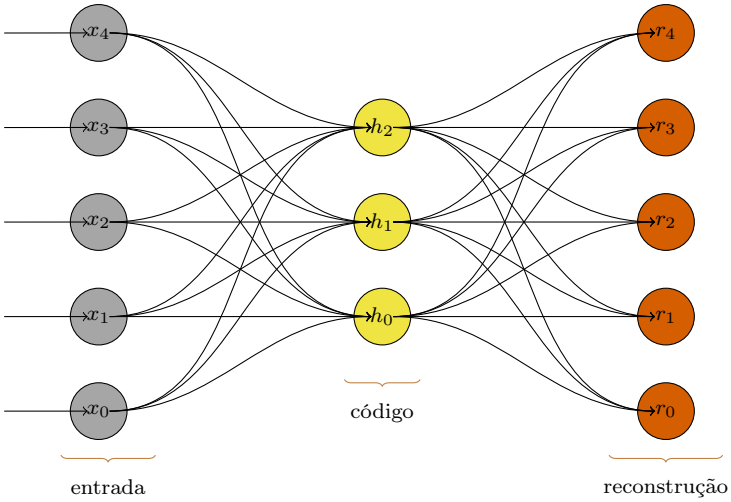
Considerando-se que a reconstrução r seja razoável, isso significa que o estado h contém dados suficientes para representar a informação em sua essência, sendo assim, *autoencoders* são ótimos produtores de vetores de características. Hipoteticamente é possível criar *autoencoders* cuja camada de código seja apenas um número inteiro caso tanto o codificador quanto o decodificador sejam grandes e complexos o suficiente, mas isso se mostraria inútil caso se necessitasse algum tipo de representação da informação original que fosse significativa, ademais, excluindo-se casos triviais, segundo (GOODFELLOW; BENGIO; COURVILLE, 2016) isso ainda não se verificou na prática.

Figura 34 – Representação funcional de um *autoencoder*: Sendo o vetor x uma entrada, então o vetor h é estado resultante da aplicação de uma função codificadora f sobre x: $h = f(x)$, portanto r é a reconstrução de x a partir h através de uma função decodificadora g: $r = g(h)$. Tal processo pode ou não implicar em uma perda de informação.



Fonte: Elaborado pelo autor, 2024.

Figura 35 – Exemplo esquemático de um *autoencoder* sub-completo ou clássico: Os nós de entrada estão em cinza, a camada de código em amarelo contêm as características codificadas e finalmente a camada de reconstrução em vermelho contêm uma cópia aproximada da entrada.



Fonte: Elaborado pelo autor, 2024.

Em se tratando de treinamento, as técnicas já conhecidas e testadas para redes neurais em geral (como, por exemplo, *gradient descent* com *minibatch* ou estocástico) podem ser usadas (GOODFELLOW; BENGIO; COURVILLE, 2016) incluindo as formas de regularização como será exposto mais adiante.

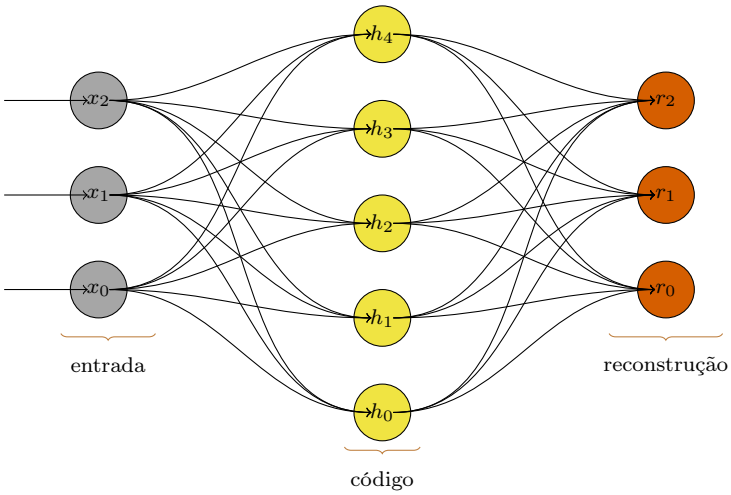
2.1.11.1 Autoencoders sub-completos ou clássicos

Esse tipo de rede neural, como ilustrado na Figura 35, tem sua camada de código limitada em dimensionalidade para apenas aproximar a entrada e priorizar certos aspectos dos dados extraindo dessa forma os componentes mais cruciais e significativos de

um conjunto de dados (GOODFELLOW; BENGIO; COURVILLE, 2016). Um autoencoder clássico age de forma muito semelhante ao algoritmo *Principal Component Analysis* (PCA) (BENGIO; COURVILLE; VINCENT, 2014).

2.1.11.2 Autoencoders supra-completos

Figura 36 – Exemplo esquemático de um *autoencoder* supra-completo.



Fonte:Elaborado pelo autor, 2024.

Como mostrado na Figura 36, ao contrário dos autoencoders sub-completos, os supra-completos têm sua camada de código com uma **dimensão maior** do que os dados de entrada. Dessa forma as características dos dados de entrada podem ser codificadas mais de uma vez e uma reconstrução perfeita é atingida. Até o momento estruturas assim são consideradas inúteis já que não conseguem extrair características significativas a não ser que sejam aplicadas técnicas de regularização (BENGIO; COURVILLE; VINCENT, 2014).

2.1.11.3 *Autoencoders* regularizados

Segundo (BENGIO; COURVILLE; VINCENT, 2014) uma das formas de regularização é justamente a limitação da dimensão da camada de código, a mesma aplicada aos *autoencoders* sub-completos, como estes já foram explicados apresentar-se-ão a seguir outras formas de regularização.

Considerando que *autoencoders* devem ser treinados para minimizar uma função de erro E que compara a entrada original x com a reconstrução r então um autoencoder **regularizado** deve ter adicionado a essa função um termo de regularizador ϕ como a mostrado na Equação 2.50 que é um termo que pode corresponder a uma regularização L1, L2, $\Omega(h)$ (discutidas na seção A.1).

$$E(x, r) + \phi \tag{2.50}$$

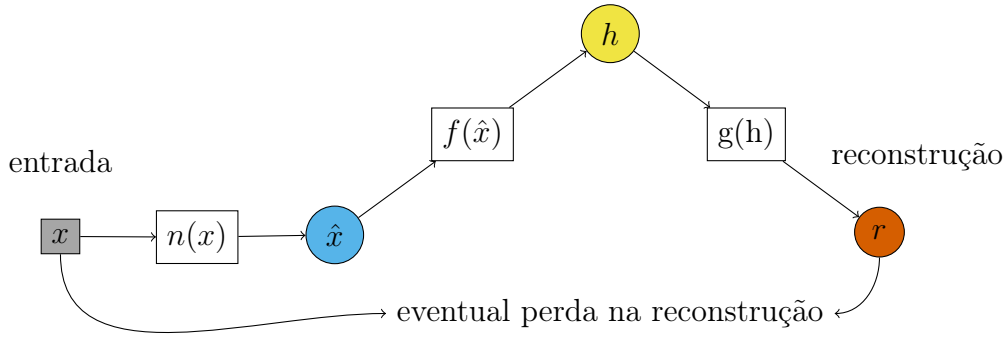
Segundo (GOODFELLOW; BENGIO; COURVILLE, 2016) usando técnicas de regularização é possível até mesmo para um *autoencoder* supra-completo que o mesmo extraia características úteis para uso. Tais técnicas devem prover a seguintes propriedades para a rede:

- dispersão: É a tendência de ativar apenas uma pequena quantidade das unidades (também conhecidas como dimensões) da camada de código para qualquer entrada dada criando uma representação mais compacta.
- pequena variação no gradiente: Garantir pouca amplitude nos valores dos gradientes em relação as variações ou ruídos de uma entrada evita que grandes flutuações ocorram na camada de código proporcionando um comportamento estável.

2.1.11.4 *Denoising autoencoders*

Tais redes também podem ser caracterizadas como *autoencoders* regularizados. Porém há diferenças: Aqui o foco não é criar funções de regularização e sim adicionar ruídos **apenas** na informação de entrada mas mantendo a intactos os dados que serão comparados na função de erro como mostra a Figura 37. Um *autoencoder* treinado dessa forma, além de ser mais resiliente a ruídos, também adquire uma funcionalidade de preencher falhas nos dados de entrada quando o componente adicionador de ruído é removido e os dados são diretamente apresentados a rede.

Figura 37 – Representação funcional de um *denoising autoencoder*: Sendo o vetor x uma entrada e n um função que adiciona um ruído aleatório então a informação com ruído $\hat{x}$ é definida como $\hat{x} = n(x)$, portanto o vetor h é resultado da aplicação de uma função codificadora f sobre x: $h = f(x)$, finalmente r é a reconstrução de x a partir h através de uma função decodificadora g: $r = g(h)$. É importante notar que r é comparada com x e não com $\hat{x}$.



Fonte: Elaborado pelo autor, 2024.

2.1.11.5 Colapso do latente no autoencoder pulsante e regularização da taxa de disparo

Esta subseção descreve um problema encontrado no *autoencoder* de RNP deste trabalho (o ramo `snn-ae` da Fase 00, subseção 2.1.5.1) e a correção implementada: a regularização da taxa de disparo (subseção 2.1.10.8) aplicada ao codificador.

Investigação da causa-raiz. O latente morto identificado em subseção 2.1.5.1 ($\alpha = \beta = 1$, saída constante) motivou uma investigação sobre por quê o codificador do `snn-ae` deixa de aprender. A arquitetura declarada para os perfis da Fase 00 — `linear:16:leaky, linear:8:identity` — resolve-se em Linear$(256, 16) \to$ LIF$(V_{th}) \to$ Linear$(16, 8) \to$ identidade: o token `identity` é um *no-op* literal, de modo que o latente de 8 dimensões é a saída **crua** da segunda camada Linear, sem qualquer não-linearidade adicional. Com a codificação `direct` ($T = 1$), a recorrência do LIF, $V_{mem} = \beta_{LIF} V_{mem}^{(t-1)} + I_{in}$, perde o termo de estado anterior (reiniciado a cada amostra) e se reduz a um teste de limiar puro sobre a saída da primeira Linear — ou seja, todo o poder de discriminação do codificador depende de quantos, dentre os 16 neurônios dessa única camada pulsante, efetivamente disparam.

Um teste diagnóstico instrumentado diretamente sobre essa camada (com pesos inicializados por He/Kaiming, subseção 2.1.10.9, e entradas uniformes em $[0, 1]$) mediu a taxa de disparo média em função da escala dos pesos:

Escala dos pesos	1×	2×	5×	10×	20×
Taxa de disparo	8,30%	24,51%	35,45%	37,11%	39,06%

Na escala nativa (1×, a produzida pela inicialização de He/Kaiming sem reescalonamento), apenas 8,3% dos disparos ocorrem (85 de 1024 combinações neurônio×amostra testadas), e a taxa cresce monotonicamente — mas de forma decrescentemente marginal — com a magnitude dos pesos. Esse é exatamente o **problema da ausência de disparos** descrito em subseção 2.1.10.7: pré-ativação predominantemente abaixo do limiar $\Rightarrow$ poucos disparos $\Rightarrow$ gradiente substituto (subseção 2.1.10.7, antes da fórmula de tdBN) próximo de zero para a maioria dos neurônios $\Rightarrow$ esses neurônios deixam de ser ajustados e permanecem mortos. Como a única camada pulsante do encoder já nasce com disparo escasso, e o restante da rede (a segunda Linear, a `identity`) não introduz nenhuma outra dinâmica de disparo, o codificador inteiro herda essa fragilidade.

O framework já possuía, antes desta investigação, duas defesas contra o problema da ausência de disparos — a tdBN (subseção 2.1.10.7) e a regularização de taxa de disparo (subseção 2.1.10.8) — mas ambas estavam conectadas apenas ao classificador DSNN, nunca ao *autoencoder*. Sem nenhuma delas, um *autoencoder* treinado por erro quadrático médio sobre um canal de informação faminto converge para a solução trivial: o decodificador aprende a ignorar o latente e emitir a média do conjunto de dados, já que essa é a

saída que minimiza o erro quadrático médio quando o latente carrega pouca ou nenhuma informação distintiva — mecanicamente forçando o vértice Ambiguidade ($\alpha = \beta = 1$) descrito em subseção 2.1.5.1.

Correção implementada. Portou-se a regularização de taxa de disparo (Equação 2.44–Equação 2.45) do classificador DSNN para o *autoencoder* pulsante, com a mesma matemática e os mesmos parâmetros (λ, r_{min}, r_{max}), agora aplicados à(s) camada(s) LIF do codificador do *autoencoder*. A diferença está na integração: o codificador do classificador é uma sequência de camadas nomeadas explicitamente, o que permite injetar o gradiente de regularização exatamente no ponto certo do *backward*; o codificador do *autoencoder*, por sua vez, é montado dinamicamente a partir da especificação declarativa do perfil (`encoder_layer_spec`) em um contêiner genérico de camadas, sem pontos de extensão para injeção de gradiente em posições internas. A solução adotada localiza as camadas LIF desse contêiner uma única vez, na construção do modelo, por introspecção de tipo, e substitui a chamada de *backward* do contêiner por um laço equivalente que replica sua ordem reversa, inserindo a regularização exatamente antes de retropropagar através de cada camada LIF identificada — sem alterar o contêiner genérico, que continua sendo usado, sem modificação, por todo o restante do framework.

Os parâmetros λ, r_{min} e r_{max} tornaram-se campos opcionais da configuração do *autoencoder*, com o mesmo padrão **desligado** ($\lambda = 0$) do classificador, de modo que nenhum perfil já executado da Fase 00 muda de comportamento — a regularização é estritamente opt-in.

Validação empírica. Executou-se o perfil `p00_ae_snn_direct_tiny_eeg` a uma taxa de aprendizado baixa o suficiente para produzir um latente quase colapsado ($\alpha =$ 0,875, $\beta \approx$ 0,9999, próximo do vértice Ambiguidade), com e sem a regularização. Sem regularização ($\lambda = 0$), o resultado permanece quase colapsado. Com $\lambda = 5{,}0$ e faixa-alvo $[r_{min}, r_{max}] = [0{,}10;\ 0{,}60]$ — acima, portanto, da taxa de equilíbrio nativa de 8,3% medida na inicialização (subseção 2.1.10.8, limitação conhecida) — α cai para 0,25: o latente deixa de ser quase-constante e passa a variar de fato entre amostras de classes diferentes, confirmando que o gradiente de regularização efetivamente alcança e altera o treinamento do codificador. Já β permanece próximo de 1 nesse perfil específico — esperado, pois β mede a sobreposição entre classes no sinal de EEG bruto, um problema de separabilidade dos dados diferente do colapso do latente, que a regularização de taxa de disparo não tem por objetivo resolver.

Valor recomendado. Os 18 perfis reais `snn-ae` da Fase 00 (e os 18 perfis `smoke` correspondentes) passaram a declarar $\lambda = 0{,}5$, $r_{min} = 0{,}10$ e $r_{max} = 0{,}80$. r_{max} mantém o padrão já usado pelo classificador DSNN; $r_{min} = 0{,}10$ foi colocado acima da taxa de disparo nativa medida na inicialização de He/Kaiming (8,3%, valor da tabela acima), garantindo que a penalidade realmente entre em ação em vez de ficar inerte; $\lambda = 0{,}5$ foi

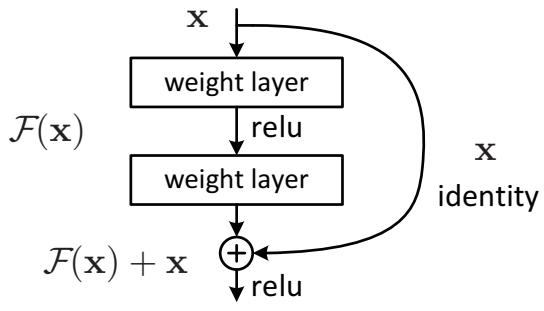
validado na codificação `direct` sob a taxa de aprendizado real do perfil (não artificialmente reduzida): sem regularização $\alpha = 0{,}5$, e com $\lambda = 0{,}5$ o mesmo experimento produz $\alpha = 0{,}375$, confirmando engajamento sem recorrer ao valor extremo ($\lambda = 5$) usado apenas no teste de colapso forçado descrito acima. Uma tentativa de repetir essa validação nas codificações `poisson` e `latency` ($T = 16$ passos de tempo) foi interrompida após quase duas horas sem terminar — ficou evidente que essas duas codificações, por si só, constituem um experimento caro nessa escala, e uma bateria de comparação nelas exige planejamento e confirmação explícita à parte.

Situação em aberto. Os 300 resultados armazenados da Fase 00 foram gerados **antes** desta mudança e não refletem mais a configuração atual dos 18 perfis `snn-ae` (agora com $\lambda = 0{,}5$). Reexecutá-los e reclassificá-los pela métrica $D_{\text{penalizado}}$ (subseção 2.1.5.1) é, pela evidência acima, uma operação de custo real (as variantes `poisson`/`latency` sozinhas ultrapassam duas horas cada) e permanece pendente no momento da escrita deste texto.

2.1.12 Redes neurais residuais (ResNets)

Segundo (HE *et al.*, 2015a) a ideia-chave por trás das *ResNets* é a inclusão de conexões de salto como ilustrado na Figura 38, também conhecidas como mapeamentos de identidade, permitindo que a saída de uma camada seja adicionada elemento-a-elemento diretamente à entrada de uma camada subsequente. Para que isso aconteça é importante que tanto a saída quanto a entrada tenham a mesma dimensão.

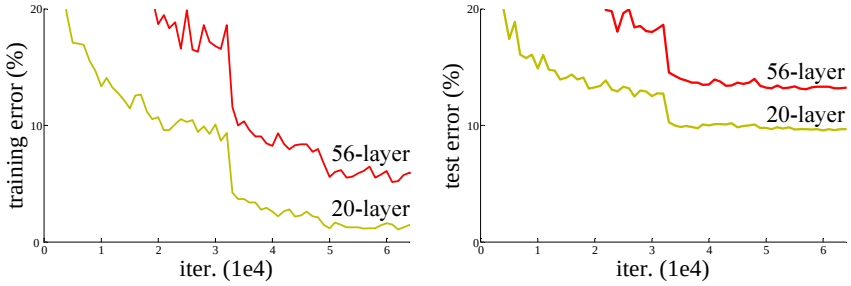
Figura 38 – Bloco Residual: x contorna as camada intermediárias $F(x)$ via uma função identidade somando-se a própria $F(x)$ ao final do bloco.



Fonte: (HE *et al.*, 2015a)

Quando uma rede chega a um certo limite de acurácia, se pode pensar que adicionando mais camadas o desempenho da mesma melhore, no entanto, na prática isso não se verifica. O que se observa em redes mais profundas pode ser exatamente o contrário como mostrado na Figura 39.

Figura 39 – Desempenho de duas redes neurais não residuais com 56 e 20 camadas. A esquerda a taxa de erro durante o treinamento, a direita a taxa de erro em relação ao conjunto de testes.



Fonte: (HE *et al.*, 2015a)

Redes neurais muito profundas podem sofrer de problemas como o desaparecimento ou explosão do gradiente, onde os gradientes (ou seja, as direções para ajustar os pesos da rede) se tornam muito pequenos ou grandes à medida que são retro-propagados. Isso pode dificultar o treinamento eficaz das camadas mais profundas.

Então já que, empiricamente, as redes com menos camadas estão expostas a menos problemas de otimização, é possível juntar várias dessas formando assim uma rede mais profunda e com mais hiperparâmetros que podem ser ajustados. Isso, aliado as técnicas de regularização, diminui a ocorrência do desaparecimento ou explosão de gradientes. (HE *et al.*, 2015a).

Em vez de aprender a mapear diretamente de x para y, as camadas da rede aprendem a mapear de x para a diferença entre x e y, ou seja, para o **resíduo**. Isso simplifica o treinamento, pois a rede precisa apenas ajustar os resíduos, em vez de aprender a mapear completamente as entradas para as saídas.

Além das características já expostas, as redes residuais não adicionam complexidade computacional além da adição elemento-a-elemento. Isso facilita a comparação com redes não residuais com o mesmo número de hiperparâmetros.

2.1.12.1 Aprendizado residual

Considerando então x como a entrada de uma rede, $f(\cdot)$ como uma série de camadas cujas sucessivas aproximações, via treinamento, resultam em um desejado estado intermediário na camada escondida h, então o que se convenciona chamar de resíduo r é dado na Equação 2.51 considerando que h e x tenham a mesma dimensão. Esta é uma reformulação didática, feita pelo autor deste trabalho, da ideia central de (HE *et al.*, 2015a) — o artigo original expressa o mesmo conceito através da função $y = F(x) + x$, em que $F(x)$ é o mapeamento residual aprendido pelas camadas.

$$r = h - x \tag{2.51}$$

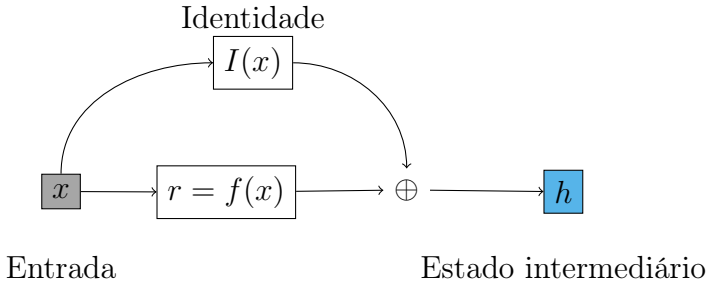
Considerando o resíduo, então se reformula o estado h como mostrado na Equação 2.52 demonstrando que, para se mapear x em h basta que $f(\cdot)$ atue sobre r, ou seja, que as aproximações sucessivas sejam feitas sobre o resíduo r.

$$h = r + x \tag{2.52}$$

Exemplo numérico. Sejam $x = [0{,}80,\ 0{,}50]$ a entrada e $h = [0{,}85,\ 0{,}40]$ o estado intermediário desejado. Pela Equação 2.51, o resíduo é $r = h - x = [0{,}05,\ -0{,}10]$. Aplicando a Equação 2.52, $h = r + x = [0{,}05 + 0{,}80,\ -0{,}10 + 0{,}50] = [0{,}85,\ 0{,}40]$, recuperando exatamente h: a camada só precisa aprender o pequeno ajuste r, não o estado h inteiro.

Se à rede não profunda forem adicionadas camadas cuja única função é receber uma entrada e devolvê-la sem modificações (função identidade) então é trivial que uma rede possa atingir grandes profundidades. No entanto segundo (HE *et al.*, 2015a) em redes profundas os otimizadores tem dificuldade em aprender tais funções identidade levando a degradação do desempenho como mostrado na Figura 39. Nas redes residuais se os mapeamentos identidade (também chamados de conexões de atalho) forem ótimos os otimizadores podem simplesmente zerar todos os pesos das camadas intermediárias de um bloco residual.

Figura 40 – Obtenção do resíduo: Se x é um vetor de entrada na rede e $I(\cdot)$ é uma função identidade cujo valor é diretamente somado $\oplus$ com o resultado produzido pelas várias camadas representadas por $f(\cdot)$, h é um vetor representando um estado intermediário de uma rede neural.



Fonte: Elaborado pelo autor, 2024.

O bloco residual mostrado na Figura 40 torna mais fácil para a rede se aproximar de funções identidade pois basta a mesma zerar os pesos constituintes das camadas representadas por $f(\cdot)$. $f(\cdot)$ é flexível e portanto pode ter n camadas constituintes, essa estrutura pode ser usada múltiplas vezes sozinha ou em conjunto com outras estruturas em uma rede.

2.2 Trabalhos mais recentes

A fim de entender melhor o campo de pesquisa, além de formar uma boa base para compreensão do assunto, foram levantados os métodos de autenticação por fala e/ou análise de sinais cerebrais.

2.2.1 Protocolo de coleta

A estratégia de busca dos artigos foi guiada segundo as base de dados, palavras-chaves e critérios constantes na subseção 2.2.1.1 e subseção 2.2.1.2 a seguir.

2.2.1.1 Critérios de inclusão e exclusão

- publicados entre 30/03/2019 e 30/03/2024 além de outros trabalhos relevantes para construção da base teórica.
- que correspondam às palavras-chave de pesquisa constantes na listagem constante na subseção 2.2.1.2
- cujo *abstract* contenha as bases de dados usadas, os resultados e a metodologia brevemente descritos.
- que tratam de autenticação por EEG e/ou voz, e, quando da falta dos mesmos, que pelo menos auxiliem no tema estudado

2.2.1.2 Base de dados e palavras-chaves

- Web of Science [10]:
  - palavras chave: voice *AND* authentication
  - filtros aplicados: *Publication date*: 2019-03-30 *to* 2024-03-30
- Google Scholar [11]:
  - palavras chave: +voice +authentication
  - filtros aplicados: *Período específico...* 2019 - 2024; *Incluir citações*;
- Athena [12]:
  - palavras chave: *Qualquer campo contém* Imagined Speech; *E qualquer campo contém* EEG; *E qualquer campo contém* wavelet

[10] https://www.webofscience.com/ (Acesso restrito por VPN)
[11] https://scholar.google.com
[12] https://unesp.primo.exlibrisgroup.com/discovery/search?vid=55UNESP_INST:UNESP (Necessita autenticação da Central de Acessos da UNESP)

  - filtros aplicados: *Data inicial:* 30/03/2019; *Data final:* 30/03/2024; *Idiomas:* Excluir Coreano e Japonês

- Athena:

  - palavras chave: *Qualquer campo contém* authentication; *E qualquer campo contém* voice
  - filtros aplicados: *Data inicial:* 30/03/2019; *Data final:* 30/03/2024; *Idiomas:* Excluir Coreano e Japonês; *Excluir artigos de newsletter*

O primeiro estudo consultado (PARK; LEE, 2023) aborda o uso de EEG para captação de fala imaginada usando as técnicas de decomposição de sinal NA-MEMD (*multivariate empirical mode decomposition*) e DTWPT e a arquitetura de redes neurais MRF-CNN. A NA-MEMD é uma extensão do EMD (*Empirical mode decomposition*) para sinais multivariados permitindo a decomposição de sinais. O MRF-CNN é uma arquitetura de rede neural convolucional profunda que utiliza características estatísticas extraídas dos sinais decompostos como entrada para o classificador. Foram utilizadas técnicas de validação cruzada e divisão de dados para treinamento e teste do classificador. Os dados EEG foram coletados dos nove participantes saudáveis, estes tiveram que imaginar as vogais "a", "e", "i", "o", "u". O estudo atingiu uma média de 80,41% de classificações corretas.

Em (ABDULGHANI; WALTERS; ABED, 2023b) usa *Wavelet Scattering Transform* (WST), técnica que aplica sobre o sinal transformadas wavelet combinadas com operações não lineares, para extração de características para posterior classificação por uma Rede Neural LSTM(*Long-Short term memory*) recorrente. Os quatro participantes imaginaram as seguintes expressões: *"up", "down", "left"e "right"*. Em testes a acurácia alcançada foi 92,50%.

Com o fim de remover ruídos e artefatos causados por movimentos musculares relacionados à região da cabeça em (MAHAPATRA; BHUYAN, 2023) foi usada a DTWT[13] para processar os sinais EEG, que foram obtidos a partir das bases de dados EPOC e MUSES (VIVANCOS, 2023). Depois da filtragem e geração de características, variações de Redes Neurais Recorrentes Bidirecionais foram empregadas na classificação das classes compostas pelos dígitos de 0 a 9. Foram alcançadas acurácias máximas de 98,18% na base MUSE e 71,60% na EPOC.

Nesta revisão (SHAH *et al.*, 2022) que foi feita segundo as orientações constantes no protocolo PRISMA-ScR foram selecionados 34 trabalhos que analisaram palavras e expressões imaginadas, nestes, a técnica mais usada para extração de características foram os filtros Wavelet e os algoritmos de classificação mais proeminentes foram as *Support*

[13] Transformada Wavelet em Tempo Discreto

Vector Machines(SVM) e as Redes Neurais Convolucionais. É reportado que os estudos utilizam poucos tipos de métricas limitando-se na maioria dos casos a acurácia, a revisão sugere que também sejam usadas métricas como: Precisão, *Recall*, Taxa de erro de palavras (Word Error Rate), AUC (Área Under the Curve), Sensitividade, Especificidade e Média Quadrática a fim de se oferecer uma visão mais geral dos resultados.

Uma combinação de redes neurais convolucionais temporais (TCN) e convolucionais tradicionais (CNN) foi usada no estudo (MAHAPATRA; BHUYAN, 2022) para classificar sinais de EEG pré-processados pelo algoritmo de Análise Independente de Componentes (*FastICA*) a fim de proporcionar uma melhor separação dos canais e por DTWT para remoção de artefatos e criação dos vetores de características. Os sinais foram obtidos da base (CORETTO; GAREIS; RUFINER, 2017) que é composta por sinais produzidos por 15 indivíduos falantes de espanhol com as vogais "a", "e", "i", "o", "u"e com os comandos *"arriba", "abajo", "derecha"e "isquierda"*. O estudo alcançou uma acurácia de 96,49%.

Para classificar as letras do alfabeto imaginadas em inglês por 13 indivíduos o estudo (AGARWAL; KUMAR, 2022) usou a DTWT para remoção ruídos e separação de bandas de frequência do sinal a ser classificado por dois algoritmos simultaneamente: Árvores Aleatórias (ou *Random Forrest*)(RF) e SVM. O estudo alcançou uma acurácia média entre as classificações de todos os elementos do alfabeto de 77,97%. Neste trabalho também foi utilizado o método *Common Average Referecing*(CAR) que, via a subtração de uma média dos sinais captados em um certo intervalo de tempo, procura remover interferências induzidas em todos os eletrodos por variáveis ambientais aumentando a relação sinal/interferência (*Signal-to-noise ratio*)(SNR). Uma interessante constatação desse trabalho é de que as ondas cerebrais α, β e θ, quando consideradas sozinhas, melhoram o desempenho dos classificadores.

Aqui (HERNÁNDEZ-DEL-TORO; REYES-GARCÍA; PINEDA, 2021) usa os seguintes métodos de extração de características: DTWT (para obtenção da energia de bandas), EMD, dimensões fractais e características extraídas segundo metodos da teoria do caos. Os dados foram obtidos a fim de se criar 3 bases de dados contendo as palavras em espanhol *"arriba", "abajo", "derecha"e "isquierda"*, a primeira continha informações de 27 pessoas, a segunda também 27 e a terceira 20. Na primeira e segunda bases, a taxa de captura foi de 128Hz e na terceira 500Hz, esta foi subamostrada para 128Hz a posteriori. Para remoção de ruído o método CAR foi usado. Os classificadores usados foram RF, *K-nearest-neighbors*(KNN), SVM e Regressão Logística. Para avaliação as métricas Pontuação F1 (PF1), Precisão e *Recall* foram usadas. Os melhores resultados obtidos nas bases 1, 2 e 3 foram respectivamente: 0,73, 0,79 e 0,68 para PF1, 0,66, 0,70, 0.65 para Precisão e 0,87, 0,93 e 0,83 para *Recall*.

Constante nas referências de (HERNÁNDEZ-DEL-TORO; REYES-GARCÍA; PI-

NEDA, 2021) um dos poucos trabalhos que discorreram sobre a identificação de pessoas (MOCTEZUMA *et al.*, 2019) usou CAR para melhorar a SNR dos sinais armazenados e amostrados a 128Hz correspondentes as palavras em espanhol *"arriba", "abajo", "derecha"e "isquierda"* imaginadas por 27 voluntários. A geração do vetor de características foi feita usando DTWT em 4 níveis e para cada nível foi calculada a Energia de *Teager* ou *Teager Energy Operator* (TEO). RF foi usada como classificador. Os resultados variaram entre 85% quando todos os 27 indivíduos foram incluídos até 97% quando apenas 5 foram considerados.

Dada a pouca quantidade de dados e bases existentes quando se trata de EEG, o artigo (PANACHAKEL; RAMAKRISHNAN, 2021) usou uma técnica de expansão de informação baseada em "janelas deslizantes"ao mesmo tempo em que, em vez de considerar separadamente cada sensor, agrupou os dados dos mesmos em matrizes tridimensionais semelhantes a imagens que foram processadas por uma rede Resnet50 (HE *et al.*, 2015a) adaptada usando a técnica de transferência de aprendizado. A base de dados usada (NGUYEN; KARAVAS; ARTEMIADIS, 2018) é composta por EEG capturado de 64 canais a 1000Hz para as seguintes falas imaginadas por 11 pessoas do sexo masculino e 4 do feminino: *"independent", "cooperate", "in", "out", "up", "a", "i", "u"*. As taxas de amostragem foram posteriormente diminuídas para 250Hz e filtradas para remoção de ruídos e artefatos. Os resultados variaram de um mínimo de 79,7% de acurácia para classificação de vogais até um máximo de 95,5% para palavra curtas e longas.

(TAMM; MUHAMMAD; MUHAMMAD, 2020) usou uma base de dados composta por 15 voluntários imaginando as letras *"a", "e", "i", "o", "u"* e mais seis diferentes palavras (CORETTO; GAREIS; RUFINER, 2017) e simplificou a estrutura de uma CNN criada em outro estudo usando uma técnica de transferência de aprendizado enquanto tentou manter a mesma acurácia. Os dados iniciais foram submetidos a sub-amostragem para que se chegasse a uma taxa de amostragem de 128Hz e dispostos em uma matriz de duas dimensões. O classificador simplificado atingiu uma acurácia média de 23,98%.

(PANACHAKEL; RAMAKRISHNAN; ANANTHAPADMANABHA, 2019) utilizou 11 palavras da base de dados (ZHAO; RUDZICZ, 2015), extraindo de cada canal características baseadas em transformadas *Wavelet* de nível 7, alcançando uma acurácia média de 57,15%. A inovação aqui foi tratar separadamente os vetores de características dos 11 canais, inserindo-os, um por vez, em uma rede neural profunda com 2 camadas ocultas. A classificação foi realizada por votação de maioria simples após o processamento dos 11 sinais da fala imaginada a ser classificada. A base de dados é composta por 7 fonemas e 4 palavras imaginadas, com uma taxa de amostragem de 1KHz. Os dados foram filtrados em uma banda de 1 a 50Hz.

Usando uma rede neural profunda de 4 camadas ocultas como classificador para duas palavras (*"in"* e *"cooperate"*) presentes no banco de dados referenciado em

(DASALLA *et al.*, 2009), (PANACHAKEL; RAMAKRISHNAN; ANANTHAPADMANABHA, 2020) criou vetores de características usando a transformada wavelet de 4 níveis, extraindo características como variância, entropia e Root Mean Square (RMS). Esses vetores foram então classificados usando um sistema de votação, onde cada vetor de características de cada canal é avaliado individualmente e a classe é determinada por maioria simples (similar a (PANACHAKEL; RAMAKRISHNAN; ANANTHAPADMANABHA, 2019)). O banco de dados é composto por amostras coletadas de 15 indivíduos com taxa de amostragem de 1KHz. O sistema obteve uma acurácia média de 71,8%.

A revisão (SEN *et al.*, 2023), cujo objetivo foi analisar artigos cuja temática fosse a escrita e fala imaginadas a partir de sinais captados por técnicas de EEG invasivas ou não, possibilitou uma visão geral do estudo da fala imaginada e as respectivas técnicas de pré-processamento e classificação já usadas. A revisão coletou trabalhos de 2014 até 2022 e identificou que os métodos usados no intervalo de tempo considerado foram os clássicos SVM, RF, *Hidden Markov Model* (HMM) e *Gaussian Mixed Models* (GMM), além desses, as técnicas de aprendizado profundo usadas foram CNN, Rede Neurais Recorrentes (RNN), *Gate Recurrent Unit* (GRU) e *Long Short Term Memory* (LSTM), sendo as técnicas de aprendizado profundo começando a serem usadas a partir do ano de 2020, ou seja, relativamente recente. Quanto a extração de características o MFCC e a (Transformada de Fourier Discreta) DFT foram as mais utilizadas seguidas por PCA, ICA, DTWT e Média Quadrática (RMS).

Apesar da autenticação por voz não ser um tema novo, há poucos trabalhos recentes que tratam do assunto, em sua maioria os artigos relacionados ao reconhecimento de voz focam mais em detecção de patologias. No que concerne a locutores disfônicos arrisca-se dizer que não há trabalhos recentes relacionados. Dito isso (WANG *et al.*, 2023) combinou a arquitetura Res2Net (GAO *et al.*, 2021a) com *Perceptual Wavelet Packet Entropy* (PWPE) e um bloco *Efficient Channel Attention* (ECA) (WANG *et al.*, 2019) para reconhecer indivíduos cujas vozes constam da base de dados (FAN *et al.*, 2020) composta por vozes de celebridades chinesas em várias situações (gravação em estúdio, palestras, gravações em ambientes ruidosos, etc.). PWPE consiste em "podar"os nós da árvore de decomposição *wavelet package* segundo seus "centros de frequência"e energia definidos anteriormente de acordo com a amplitude de audição da espécie (no caso: Humanos), dessa forma apenas as frequências pertencentes à escala auditiva humana permanecem. Em seguida é aplicado o operador não normalizado de entropia de Shannon, gerando assim os vetores de características. Nesse artigo o objetivo foi verificar como os modelos se comportariam em termos de EER se fossem treinados em um contexto e depois aplicados a outros com, por exemplo, treinar em uma palestra em um auditório e testar com dados vindos de ambientes externos. Os resultados de classificação foram, em média, de menos do que 8% de EER para contextos de médio e alto ruído.

No estudo de (ALI *et al.*, 2022), embora o foco não fosse a identificação dos locutores, os classificadores concentraram-se na classificação do gênero das pessoas envolvidas. Para a criação dos vetores de características, foram utilizados MFCC e *Linear Prediction Coefficients* (LPC). Os dados utilizados foram coletados de uma base de dados com 93 locutores (HILLENBRAND *et al.*, 1995). Como classificadores, foram empregadas Análises Discriminantes e redes neurais artificiais. Os melhores resultados foram alcançados pela combinação de uma rede neural com MFCC, obtendo uma acurácia de 97,07%.

Dos estudos apresentados até o momento todos avaliaram a fala (imaginada ou não) do locutor ao pronunciar uma certa palavra, frase ou sílaba, nesses trabalhos o acesso ao sistema deve ser feito no esquema de um porteiro que libera ou não o acesso sem uma verificação contínua da identidade do locutor. Em (MENG; ALTAF; JUANG, 2020) implementou-se o *active voice authentication* (AVA) ou sistema de autenticação ativa que consiste na verificação constante e em tempo real da identidade de quem fala. O sistema extrai os dados coletando sinais com um esquema de janela móvel que acumula 1 segundo de dados e, usando essa amostra, calcula coeficientes MFCC para gerar os vetores de características que alimentam uma HMM para treinamento e classificação. Os dados usados no trabalho foram coletados de 25 voluntários (14 mulheres e 11 homens) a uma taxa 8Khz contendo um serie de locuções de textos longos, curtos, alguma senha escolhida pelo locutor e pares de dígitos totalizando um total de 2 horas e meia para todos os locutores. Como medida de desempenho uma versão adaptada do ERR foi criado: O *Window-based equal error rate* (WEER) alcançando uma taxa média de 3% a 4%.

Captando a fonação distorcida pelos tecidos e ossos do usuário através de microfones instalados em fones de ouvido, o estudo (GAO *et al.*, 2021b) teve como objetivo aproveitar a anatomia individual para gerar uma impressão digital. Essa impressão digital é obtida comparando o áudio captado por outro dispositivo, permitindo assim a identificação do locutor. Para a realização do trabalho, uma base de dados amostrada a 48 kHz com 23 participantes (5 mulheres e 18 homens) entre 24 e 30 anos foi criada. Os dados foram submetidos a um filtro para remover frequências acima de 4 kHz. Dois classificadores (GMM e Floresta de Árvores de Decisão) utilizando um sistema de votação ponderada foram usados para autenticar os participantes. Em termos de características, foram extraídos 20 coeficientes MFCC. O resultado foi um EER de 3,64

Na mesma linha de aproveitar características outras que não só a fonação, (JIANG *et al.*, 2023) usa os *"sons de estalos"* (*pop noises*) emitidos durante a fala para prevenção de *voice spoofing*, tais sons são típicos de quando uma certa pressão do ar se acumula no aparelho fonador sendo liberado em ondas de maior pressão, geralmente durante a pronúncia de palavras com consoantes, tais sons são específicos de cada locutor ou locutora e é de difícil reprodução já que a captação de som deve ser feita a, no máximo, 12 centímetros de distância. Para desenvolver o trabalho foram coletadas por cinco vezes

senhas faladas por 30 participantes a uma taxa de 44.1kHz, tais senhas continham de 3 a 10 palavras. Para classificação foi empregado um modelo de regressão logística que avalia um vetor de características de 3 dimensões $f = \{pro1, pro2, pro3\}$ sendo $pro1$ a medida de *"estalos"* nos fonemas vozeados (geralmente vogais), $pro2$ a medida de *"estalos"* nos fonemas não vozeados (geralmente consoantes) e $pro3$ que é a medida de similaridade entre os *"estalos"* da atual tentativa de autenticação com os dados de cadastro já armazenado em uma fase anterior.

O estudo (FURLAN, 2021) usou dois métodos principais para classificação: um baseado no cálculo de distâncias Euclidiana e Manhattan entre pontos de dados e outro usando uma SVM. Para extrair características úteis dos sinais de fala, primeiro aplicou-se transformadas usando várias wavelets para destacar informações de tempo e frequência. Em seguida, esses sinais transformados foram agrupados de acordo com as escalas MEL ou BARK, que refletem a percepção humana do som, e a energia dentro de cada grupo foi calculada para criar vetores de características. Tais combinações de escala e wavelet foram testadas usando a engenharia paraconsistente de características a fim de encontrar qual geraria o melhor vetor de características. Os dados vieram de gravações de 21 pessoas falando dígitos de 0 a 9 em inglês e português em ambientes com diferentes níveis de ruído. Essas gravações foram categorizadas em grupos genuínos e falsificados. O sistema diferenciou corretamente as falas autênticas das falsificadas com mais de 99% de precisão e um EER de 0,024390 usando uma combinação específica da escala BARK e wavelet Haar.

Conclusões: Os estudos consultados forneceram ideias para a construção do protocolo para coleta de dados que será explicado no Capítulo 3, além disso, trouxeram a importância da filtragem das frequências próprias da rede elétrica local (MAHAPATRA; BHUYAN, 2023) a fim de que tais interferências não sejam consideradas no vetor de características. Ainda no tópico de captação dos sinais (MAHAPATRA; BHUYAN, 2022) usou o algoritmo *FastICA* a fim de garantir uma melhor separabilidade entre os vários canais EEG. Na revisão (SHAH *et al.*, 2022) se demonstrou que, em se tratando de processamento de sinais, a fase de extração de características é de grande importância pois esta impacta de forma sensível no desempenho dos algoritmos de classificação. Também em (SHAH *et al.*, 2022) foram destacadas várias métricas que contribuirão para uma melhor avaliação dos resultados nos procedimentos futuros, além disso, (AGARWAL; KUMAR, 2022) lembrou a importância de separar indivíduos por sexo, destros e canhotos além de constatar que as ondas cerebrais α, β e θ quando consideradas sozinhas melhoram o desempenho dos classificadores. Em se tratando de tratamento dos sinais além da DTWT, CAR pareceu uma escolha bem comum entre os trabalhos (AGARWAL; KUMAR, 2022), (HERNÁNDEZ-DEL-TORO; REYES-GARCÍA; PINEDA, 2021), (MOCTEZUMA *et al.*, 2019). Fez muito sentido quem em (PANACHAKEL; RAMAKRISHNAN, 2021) os sinais foram analisados de forma conjunta criando matrizes tridimensionais, pois a fala imagi-

nada acontece em todo o cérebro, sendo assim, analisar de forma isolada cada canal pode prejudicar o aprendizado das correlações entre os sinais. Em (TAMM; MUHAMMAD; MUHAMMAD, 2020) foi possível constatar que nem sempre redes neurais mais profundas terão um desempenho muito melhor do redes mais rasas. Indo na contramão da maioria dos estudos (PANACHAKEL; RAMAKRISHNAN; ANANTHAPADMANABHA, 2019) e (PANACHAKEL; RAMAKRISHNAN; ANANTHAPADMANABHA, 2020) trouxeram uma ideia interessante: Como os sinais da fala imaginada são altamente correlacionados, uma forma de aumentar a quantidade de dados fornecidos ao classificador é considerar separadamente cada um dos canais de EEG como uma entrada à rede de forma que, ao fim da classificação de todos os canais, a classe mais votada seja o resultado da classificação. A revisão (SEN *et al.*, 2023) além de de trazer referências e informações interessantes foi importante ao estimular um entendimento fundamental para essa pesquisa: O de que, apesar da fala imaginada ocorrer em todas as regiões cerebrais monitoradas por EEG, como a autenticação dos locutores será feita também pela fonação (por mais disfônica que a mesma seja), provavelmente as áreas como o córtex motor primário, Broca e Wernicke, responsáveis respectivamente pela mobilização muscular necessária para a produção da fala, criação dos sons e articulação da fala, também estarão envolvidos. Em se tratando de DTWT fica claro que o uso dessa técnica, ao menos na área de estudo em questão, se popularizou recentemente já que apenas 1 trabalho da revisão a utilizou em contraste com as várias implementações vistas após esse período. Ainda no campo das transformadas Wavelet (WANG *et al.*, 2023) trouxe uma ideia interessante: Selecionar os nós de uma árvore de decomposição *Wavelet-Packet* segundo uma amplitude de frequências desejadas, no caso, aquelas pertencentes a voz humana, é interessante destacar que tal técnica pode ser aplicada a qualquer sinal, o que pode ser muito útil no momento do tratamento do sinal tanto de voz quando de EEG. Seguindo uma abordagem mais tradicional (ALI *et al.*, 2022) confirma a tendência do uso do MFCC para criação de vetores de características. Em (MENG; ALTAF; JUANG, 2020) o conceito de sistema de autenticação ativa mostrou que é possível aumentar em muito a precisão de uma avaliação estatística (dentro de um contexto de autenticação independente de fala) avaliando pequenos intervalos do sinal fornecido criando assim uma grande massa de resultados que podem ser avaliados. Em (GAO *et al.*, 2021b) aumentou a resistência a ataques de *voice spoofing* captando a locução através dos tecidos do próprio locutor. Em (FURLAN, 2021) e (JIANG *et al.*, 2023) apesar de não tratarem de autenticação de locutores os mesmos reafirmam algo muito importante na classificação de sinais: A construção de vetores/matrizes características melhores para que os classificadores possam ser mais simples e, ainda assim, eficazes.

Por fim, uma ocorrência comum entre os estudos é que quanto maior a quantidade de pessoas a serem autenticadas menor a precisão do modelo o que levanta uma questão se realmente é prático manter dentro do espaço latente dos modelos citados quando há a necessidade de autenticar uma grande quantidade de usuários.

3 Abordagem proposta

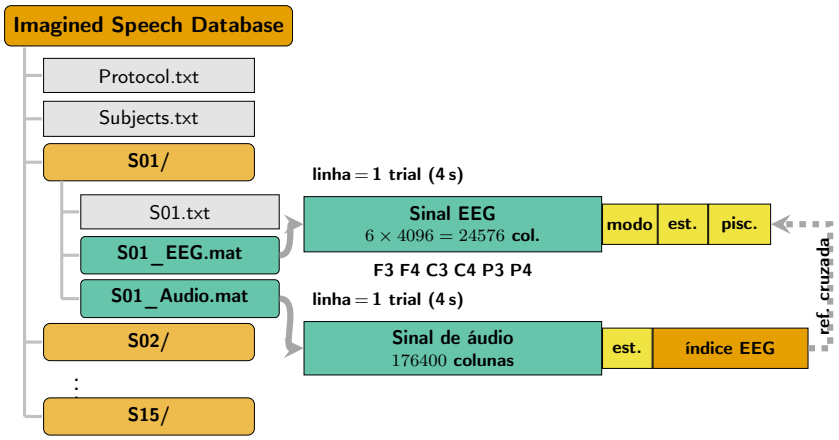
3.1 A Base de sinais

Os experimentos deste trabalho utilizam a base pública de sinais descrita em (CORETTO; GAREIS; RUFINER, 2017), que reúne registros simultâneos de voz e eletroencefalograma (EEG) de 15 voluntários falantes nativos de espanhol. Cada participante produziu onze classes de enunciados: cinco vogais (/a/, /e/, /i/, /o/, /u/) e seis comandos direcionais (*arriba*, *abajo*, *adelante*, *atras*, *derecha*, *izquierda*), em duas modalidades — fala fonatória e fala imaginada.

Os sinais de voz foram gravados a 44100 Hz com canal único. Os sinais de EEG foram capturados com seis eletrodos posicionados em F3, F4, C3, C4, P3 e P4, conforme o sistema internacional 10-20, com taxa de amostragem de 1024 Hz. Cada trial corresponde a uma janela de 4 segundos da fase de imaginar/pronunciar, resultando em 4096 amostras por canal de EEG e 176400 amostras de áudio por trial.

Os dados são organizados em arquivos MATLAB (`.mat`) por sujeito, com dois arquivos por participante: `S##_EEG.mat` e `S##_Audio.mat`. Cada arquivo contém uma matriz cujas linhas representam trials individuais. No arquivo de EEG, cada linha possui 24576 colunas de sinal (6 canais $\times$ 4096 amostras, concatenados) seguidas de 3 colunas de metadados: modalidade (imaginada ou fonatória), código do estímulo e indicador de artefato de piscada ocular. No arquivo de áudio, cada linha contém 176400 amostras seguidas de 2 colunas: código do estímulo e índice da linha correspondente na matriz de EEG, garantindo o alinhamento temporal entre as duas modalidades. A Figura 41 ilustra essa organização.

Figura 41 – Estrutura dos arquivos da base de sinais (CORETTO; GAREIS; RUFINER, 2017)



Fonte: Elaborado pelo autor, 2025.

3.2 Estrutura da estratégia proposta

Primeiramente, os sinais são decompostos pela DTWPT; em seguida, realizam-se os agrupamentos energéticos em três esquemas de particionamento espectral — bandas lineares, Mel e Bark — gerando, na Categoria 1, os vetores de *Linear-band energies*, *Mel-band energies* e *Bark-band energies*, respectivamente. Para a Categoria 2, aplica-se adicionalmente o logaritmo e a DCT sobre cada um desses vetores, produzindo os coeficientes LFCC, MFCC e BFCC. Por fim, conduz-se a análise paraconsistente de características a fim de selecionar as melhores combinações entre agrupamento energético e *wavelet*, visando gerar características as mais disjuntas possíveis.

Em segundo lugar, tanto os dados de voz quanto os de EEG são submetidos a *autoencoders* para extração de características — a Categoria 3 da taxonomia introduzida na seção 2.1. Ao lado deles entra a PCA, extrator de características clássico, como referência linear da extração automatizada (justificada adiante). As três categorias e a PCA são, portanto, os extratores comparados nesta tese.

As características extraídas, tanto automatizadas quanto as manualmente obtidas, servem de entrada para uma rede neural residual profunda de pulsos (DSNN), que determina as classes de cada um dos sinais. A arquitetura dessa rede e o protocolo de avaliação são detalhados na subseção 3.2.1.2.

Como o tema central deste trabalho é a autenticação biométrica combinando fala fonada e fala imaginada, cada configuração é avaliada em três modos, para medir a contribuição individual e conjunta das duas fontes:

- **apenas fala fonada**: somente o sinal de voz (áudio) alimenta o classificador;
- **apenas fala imaginada**: somente o sinal de EEG alimenta o classificador;

- **fonada + imaginada (fundida)**: as duas fontes são combinadas antes da classificação.

No modo fundido, o *instante* em que as duas fontes se combinam é, ele próprio, um eixo experimental, pois desloca a fronteira entre o que é modelado separadamente e o que é modelado em conjunto:

- **fusão precoce** (*early fusion*): os sinais brutos de voz e EEG são concatenados *antes* da extração de características, de modo que um único extrator opera sobre o sinal conjunto e pode capturar interações entre as modalidades. Em contrapartida, impõe uma taxa de amostragem comum às duas metades: adota-se a taxa da voz (44100 Hz), dominante em número de amostras, o que exige reamostrar o EEG de 1024 Hz para 44100 Hz — um fator de aproximadamente 43×. Essa reamostragem não cria informação: ela apenas interpola o sinal de EEG para que as duas metades possam ocupar um mesmo vetor, e o custo é que a metade de EEG do sinal concatenado passa a ser majoritariamente composta por amostras interpoladas;
- **fusão tardia** (*late fusion*): cada sinal é submetido *independentemente* à extração de características, e apenas os vetores resultantes são concatenados. Preserva a taxa nativa e o extrator próprio de cada fonte, ao custo de nunca observar as duas conjuntamente.

Ambas as estratégias são comparadas, uma vez que não há garantia a priori de qual delas produz características mais separáveis segundo a engenharia paraconsistente.

Como o modo fundido se desdobra nessas duas variantes, os *três* modos acima correspondem, no eixo experimental da Fase 01 (subseção 3.2.1.2), a *quatro* fontes distintas: apenas voz, apenas EEG, fundida-precoce e fundida-tardia.

Quanto à extração automatizada, comparam-se dois *autoencoders* de Categoria 3 — um de pulsos (SNN[1]-AE) e um convencional (ANN[2]-AE) — e um extrator linear de referência, a PCA, e os três são confrontados com a extração manual (*handcrafted*), usando a engenharia paraconsistente de características como critério de comparação (seção 2.1).

Por que a PCA entra na comparação. A PCA é um dos extratores de características mais consolidados: projeta cada amostra nos k eixos ao longo dos quais os dados mais variam, obtidos em forma fechada a partir da matriz de covariância, sem treino iterativo e sem outro hiperparâmetro além de k (JOLLIFFE, 2002). Nesta tese, ela recebe exatamente a entrada dos *autoencoders* — 256 valores normalizados por amostra — e k assume os mesmos tamanhos dos seus latentes (8, 16 e 32). Serve, assim, de régua para a pergunta que os *autoencoders* precisam responder para se justificar: o treino de um

[1] *Spiking Neural Network* (Rede Neural de Pulso)
[2] *Artificial Neural Network* (Rede Neural Artificial)

modelo não linear acrescenta algo a uma projeção linear do mesmo tamanho? A régua não é retórica. Em um estudo preliminar do autor (FURLAN, 2026), com validação cruzada agrupada por locutor ou sujeito em quatro bases (duas de fala, duas de EEG), a PCA com k igual à dimensão latente reconstruiu melhor que o ANN-AE e que o SNN-AE em todas elas. Há ainda uma razão própria desta tese: por não ter regime de treino, a PCA permite separar as duas leituras possíveis do colapso dos latentes observado na Fase 00 (seção 4.1). Se o latente da PCA também apresentar $\bar{\beta}$ próximo de 1, a sobreposição entre classes é da representação de dimensão k desses sinais, e não de um treino insuficiente dos *autoencoders*.

[PENDENTE] *A PCA já está implementada no pipeline da Fase 00 (perfis de voz e de EEG com $k \in \{8, 16, 32\}$, ranqueados junto dos demais extratores e tabelados em separado), mas ainda não foi executada; a comparação acima aguarda esses resultados.*

Por que não *autoencoders* de sequência. LSTM[3]-AE, GRU[4]-AE e Transformer-AE foram pré-validados no mesmo estudo preliminar (FURLAN, 2026), em uma base de fala (FSDD) e uma de EEG (EEGMMIDB), e descartados. O motivo principal é a qualidade: nenhum deles reconstruiu melhor que a PCA, e o LSTM-AE ficou próximo de um preditor que apenas devolve a média da janela, cujo erro médio quadrático é de cerca de 1 em janelas padronizadas. Seu erro de teste foi 0,967 no FSDD, contra 0,675 da PCA, e $0{,}821 \pm 0{,}255$ no EEGMMIDB, contra 0,192 — um desvio que indica que o treino ora convergia, ora não. O custo reforça o descarte: pelo modelo de energia do estudo, o LSTM-AE consumiu de 257 a 338 vezes a energia por janela do ANN-AE, e cada *fold* levou de 30 a 55 vezes o tempo de parede do *fold* do SNN-AE (18,1 e 16,9 horas). O descarte vale no escopo em que foi medido — orçamento de busca reduzido, apenas os *folds* de desenvolvimento, critério de reconstrução e janelas curtas de 256 amostras, justamente o regime em que a memória longa de uma recorrência tem menos a oferecer — e não estabelece que modelos de sequência sejam piores extratores de identidade em locuções longas. A dinâmica temporal não fica, contudo, sem representante: o SNN-AE carrega estado no potencial de membrana ao longo dos passos, e a DTWPT descreve explicitamente a estrutura tempo-frequência. O LSTM-AE permanece implementado no *framework* (`autoencoder.model = "lstm-ae"`) e pode ser reavaliado nessas condições.

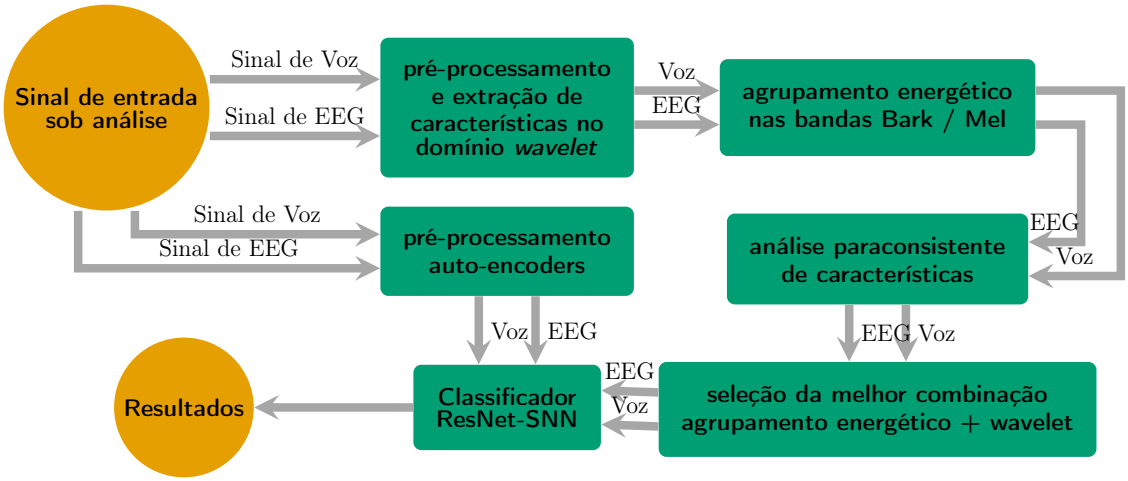
Uma visão geral do processo é mostrada na Figura 42.

3.2.1 Protocolo experimental em duas fases

O experimento é dividido em duas fases independentes, executadas em sequência. A **Fase 00** constrói o vetor de características e escolhe, por sinal, o melhor extrator segundo a engenharia paraconsistente. A **Fase 01** recebe apenas o vetor vencedor e treina

[3] *Long Short-Term Memory* (Memória de Longo e Curto Prazo)
[4] *Gated Recurrent Unit* (Unidade Recorrente com Portas)

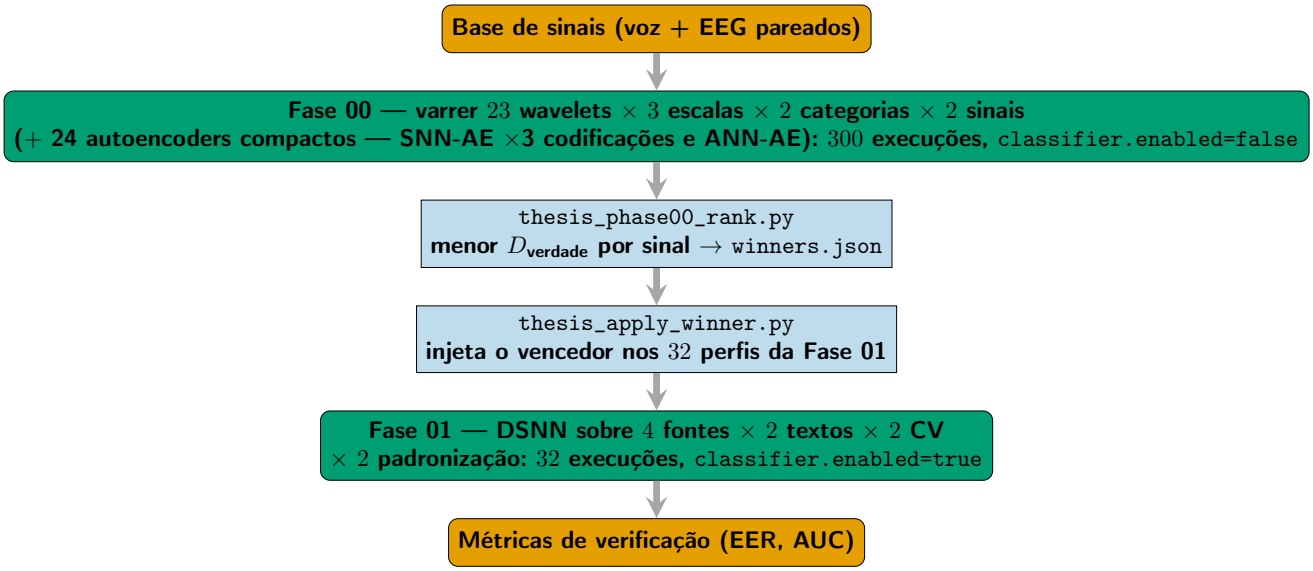
Figura 42 – Fluxograma da estratégia proposta



Fonte: Elaborado pelo autor, 2024.

o classificador para medir o desempenho de autenticação. A separação é controlada por um único parâmetro de configuração, `classifier.enabled`: quando falso, a execução para após o ranking (Fase 00); quando verdadeiro, o classificador é treinado (Fase 01). A Figura 43 mostra o encadeamento completo.

Figura 43 – Encadeamento das duas fases e dos *scripts* de transição



Fonte: Elaborado pelo autor, 2026.

3.2.1.1 Fase 00 — construção e seleção do vetor de características

A Fase 00 é aplicada separadamente à voz e ao EEG. Para o método manual, varrem-se duas escolhas: a **wavelet-mãe** da DTWPT (23 opções: `haar` e `daub4` a `daub46`) e a **escala** de agrupamento espectral (*bark*, *mel*, *lfcc*). Além disso, escolhe-se a **categoria** do descritor: Categoria 1 usa as energias de banda diretamente; Categoria 2 aplica

logaritmo e DCT sobre essas energias, gerando os coeficientes cepstrais LFCC, MFCC ou BFCC (para escala *lfcc*, *mel* ou *bark*). Para a **voz**, isso dá $23 \times 3 \times 2 = 138$ combinações manuais. Para o **EEG**, porém, o eixo da escala não se aplica: *bark* e *mel* são escalas cocleares — modelam a resolução em frequência da *audição* humana — e não há base fisiológica para aplicá-las a um sinal que não é som. Essa exclusão é também empiricamente inócua: sobre o agrupamento espectral usado aqui, as três escalas produzem para o EEG exatamente o mesmo particionamento (subseção 2.1.3.9), de modo que as três variantes eram numericamente idênticas. O EEG usa portanto apenas a escala linear (*lfcc*), dando $23 \times 1 \times 2 = 46$ combinações manuais. A ambos os sinais somam-se 12 *autoencoders* compactos, sem wavelet ou escala: 9 SNN-AE (3 tamanhos $\times$ 3 codificações temporais) e 3 ANN-AE (3 tamanhos) — e 3 projeções PCA, uma para cada dimensão latente ($k = 8$, 16 e 32), sobre a mesma entrada. Ambas as famílias de *autoencoders* consomem um vetor achatado de dimensão fixa (o sinal é reamostrado por *pooling* para 256 amostras, normalizado min-max para $[0, 1]$), usam uma única camada oculta (rasa, adequada a dispositivos de baixo poder) e razão de compressão $2:1$ entre a camada oculta e o gargalo latente (dimensões latentes 8, 16 e 32, tamanhos `tiny`/`small`/`base`). O latente é o próprio vetor de características, então um gargalo menor gera vetor menor, barateando o classificador a jusante — princípio da literatura de autoencoders leves para borda.

SNN-AE (codificador `Linear`→`LIF`, decodificador `Linear`→`LIF-integrador`) só carrega informação através de pulsos: um único vetor analógico apresentado em um só passo (equivalente a $T = 1$) deixa o neurônio LIF quase sempre abaixo do limiar de disparo, colapsando o latente a zero. Por isso cada amostra normalizada é expandida em T (padrão 16) quadros de pulso via três codificações temporais comparadas nesta tese — $18 = 3$ tamanhos $\times$ 3 codificações $\times$ 2 sinais:

- `poisson` — cada componente dispara com probabilidade de Bernoulli igual ao seu valor normalizado (codificação por taxa);
- `latency` — componentes de maior valor disparam mais cedo ($t_{\text{disparo}} = \text{round}((1 - v)(T - 1))$, mesmo esquema do Experimento 04);
- `direct` — passagem analógica direta, sem pulsos (linha de base não-temporal; $T = 1$).

O estado de membrana é *reiniciado uma única vez por amostra* e depois *integra ao longo dos T quadros* (sem reset entre quadros) — é essa integração temporal que permite a uma corrente sub-limiar por passo acumular até disparar. A característica de cada amostra é a *taxa de disparo média do latente sobre os T quadros*. O limiar de disparo do codificador (`voltage_threshold`) é reduzido para 0,2 (padrão LIF = 1,0) nas variantes `poisson`/`latency`, já que quadros de pulso normalizados têm amplitude baixa; `direct` mantém o limiar padrão 1,0, reproduzindo intencionalmente o regime não-temporal, que serve de linha de base para as duas codificações temporais.

3.2.1.1.1 Codificação temporal e função de perda do treinamento.

Embora as três codificações produzam padrões de pulso distintos na entrada do codificador, o SNN-AE é treinado com a **mesma** função de perda de reconstrução — Erro Médio Quadrático (EQM) — nas três variantes (`poisson`, `latency`, `direct`), pois o alvo da reconstrução é sempre o sinal analógico original de entrada, e não um trem de pulsos: a codificação temporal afeta apenas como a informação flui pelo codificador, não o que o decodificador precisa reproduzir. É importante notar que o *framework* desenvolvido para esta tese também implementa duas funções de perda casadas a codificações de pulso — `SpikeCountLoss` (EQM sobre a contagem total de disparos, adequada quando o próprio alvo supervisionado é uma taxa de disparo) e `SpikeTimeLoss` (EQM sobre o instante do primeiro disparo, adequada quando o alvo é um único evento temporal) — seguindo a convenção da codificação temporal usada por (COMŞA *et al.*, 2021) e a discussão de funções de perda sob medida para predição de pulsos de (MANNA *et al.*, 2024). Usar a função casada errada quebra o treino: `SpikeTimeLoss` sobre um alvo de taxa enxerga apenas o primeiro disparo e ignora a contagem; `SpikeCountLoss` sobre um alvo de latência trata a ausência de disparo como contagem zero, gerando gradiente incorreto para disparos tardios. Essa correspondência codificação–perda é, no entanto, uma capacidade geral do *framework* (`include/layers/losses/SpikeCountLoss.hpp`, `SpikeTimeLoss.hpp`), exercitada apenas nos testes unitários da biblioteca — o SNN-AE da Fase 00 não a utiliza, precisamente porque seu alvo de treino nunca é um trem de pulsos, e sim o sinal contínuo original.

3.2.1.1.2 Comparação entre as codificações temporais.

As três codificações foram executadas sobre a grade da Fase 00 (18 perfis SNN-AE — 3 codificações $\times$ 3 tamanhos $\times$ 2 sinais, 3 repetições cada). Uma versão anterior deste texto reportava valores de α e D_{verdade} medidos sob o defeito de escala da taxa de aprendizado descrito em subseção 2.1.10.10 — a redução destinada apenas aos parâmetros biofísicos era aplicada a *todos* os parâmetros, de modo que cada peso treinou a uma taxa efetiva dez vezes menor que a declarada no perfil. Aqueles números foram **retirados**, e não corrigidos: descreviam um experimento diferente do que os perfis declaravam. Corrigido o defeito, os resultados armazenados foram descartados e a Fase 00 foi integralmente reexecutada; os valores atuais estão nas Tabelas 8 e 9.

A reexecução confirma a ordem `direct` $\succ$ `latency` $\succ$ `poisson` em ambos os sinais, e de forma limpa: as três codificações formam blocos contíguos no ranking, com a `poisson` ocupando as três últimas posições e exibindo α praticamente nulo ($\alpha \leq 0{,}003$ nos seis perfis). **Isso não estabelece um resultado negativo contra a codificação temporal**, e a razão é o argumento desenvolvido a seguir: essa é exatamente a ordem que o piso de

ruído de cada codificação prevê, *antes de qualquer aprendizado*. O que a medição confirma é a previsão do mecanismo, não uma propriedade informacional dos códigos temporais.

O *mecanismo* que sustenta essa leitura é teórico, e prevê a ordem observada independentemente de qualquer execução — é por isso que a concordância entre previsão e medição, por si só, não decide a questão. Cada codificação injeta, **antes de qualquer aprendizado**, uma quantidade de ruído que se calcula em fechado. A `poisson` sorteia um Bernoulli independente a cada passo de tempo, de modo que a média sobre T quadros tem variância $v(1-v)/T$; para $v = 0{,}5$ (pior caso) e $T = 16$, isso dá desvio-padrão

$$\sigma_{\text{poisson}} = \sqrt{\frac{0{,}25}{16}} = 0{,}125\,, \tag{3.1}$$

isto é, 12,5% **da faixa total** $[0, 1]$ **da característica**, somados a cada amostra de forma independente e aleatória — e presentes ainda que o codificador fosse perfeito, pois nascem na entrada. A `latency`, ao contrário, é determinística: o mesmo v sempre produz o mesmo instante de disparo, e seu erro é de quantização, limitado a meio degrau, $\frac{1}{2} \cdot \frac{1}{T-1} \approx 0{,}033$ — quase quatro vezes menor e, sobretudo, idêntico entre amostras de mesmo valor. A `direct` não injeta ruído algum.

Esse escalonamento — ruído nulo, pequeno-determinístico, grande-aleatório — incide diretamente sobre o que α mede. Como α é definido a partir da *amplitude* (mínimo–máximo) intraclasse (seção 2.1), ele é maximamente sensível a valores extremos: basta que a estocasticidade produza poucas amostras extremas para que a amplitude de uma classe cubra todo o intervalo normalizado e $\alpha \to 0$. O piso de ruído de cada codificação mapeia-se, portanto, monotonicamente em α, e prevê a ordem `direct` $\succ$ `latency` $\succ$ `poisson` como **artefato conjunto da métrica e do piso estatístico do codificador**, não como evidência de que códigos temporais carreguem menos informação de locutor.

Daí a retificação metodológica: a afirmação de que a codificação temporal constitui um resultado negativo **não está estabelecida**, e não por mera incerteza — α é *estruturalmente incapaz* de distinguir "o codificador não aprendeu" de "esta codificação tem um piso estatístico irredutível em $T = 16$". Como esse piso decresce com $1/T$ (Equação 3.1), a questão é empiricamente testável: em $T = 64$ a variância cai por um fator de quatro, e σ_{poisson} vai a 0,0625. Esse teste ainda não foi executado. Registre-se também que este problema é **independente** da vulnerabilidade de $D_{1,0}$ tratada em subseção 2.1.5.1: aquela mostra que o critério *recompensa* indevidamente variância nula; esta, que ele pode *punir* indevidamente uma variância real que não decorre de pior aprendizado. Corrigir a primeira não corrige a segunda.

Cada combinação gera um vetor de características, avaliado pela distância D_{verdade} — a distância ao vértice *Verdade* no plano paraconsistente (seção 2.1); quanto menor, mais separáveis são as classes antes de qualquer classificador. O vencedor do sinal, porém, **não**

é a combinação de menor D_{verdade}, e sim a de menor $D_{\text{penalizado}} = D_{\text{verdade}} + \lambda|G_2|$, com $\lambda = 2 - \sqrt{2}$ (subseção 2.1.5.1): D_{verdade} isolado é vulnerável a extratores degenerados, que produzem uma saída praticamente constante e assim obtêm uma pontuação artificialmente boa sem carregar qualquer informação de classe. O termo $\lambda|G_2|$ penaliza exatamente essa degenerescência. A escolha altera o resultado na prática, e não apenas em princípio — as consequências sobre o vencedor de cada sinal são discutidas no Capítulo 4. O vencedor é escolhido entre *todas* as combinações do sinal, manuais e *autoencoders* conjuntamente. Nenhum classificador é treinado nesta fase. O Algoritmo 3.1 descreve o procedimento e a Figura 44 o resume.

```
/* WAVELETS: 23 wavelets-mae (haar, daub4, ..., daub46)          */
/* ESCALAS : 3 escalas espectrais (bark, mel, lfcc)              */
/* sinal   : vetor de amostras de um sinal (voz OU EEG)          */
/* Retorna a melhor combinacao (menor D_verdade).                */

Combinacao seleciona_vencedor(double sinal[], int n)
{
    Combinacao vencedor;
    vencedor.d_verdade = INFINITO;      /* comeca no pior valor  */

    /* 1) parte handcrafted: varre wavelet X escala             */
    for (int w = 0; w < 23; w++) {
        for (int e = 0; e < 3; e++) {
            double vetor[MAX_CARACT];
            int m = extrai_handcrafted(sinal, n, WAVELETS[w],
                                       ESCALAS[e], vetor);

            /* alpha, beta -> ponto no plano paraconsistente     */
            double d = calcula_d_verdade(vetor, m);

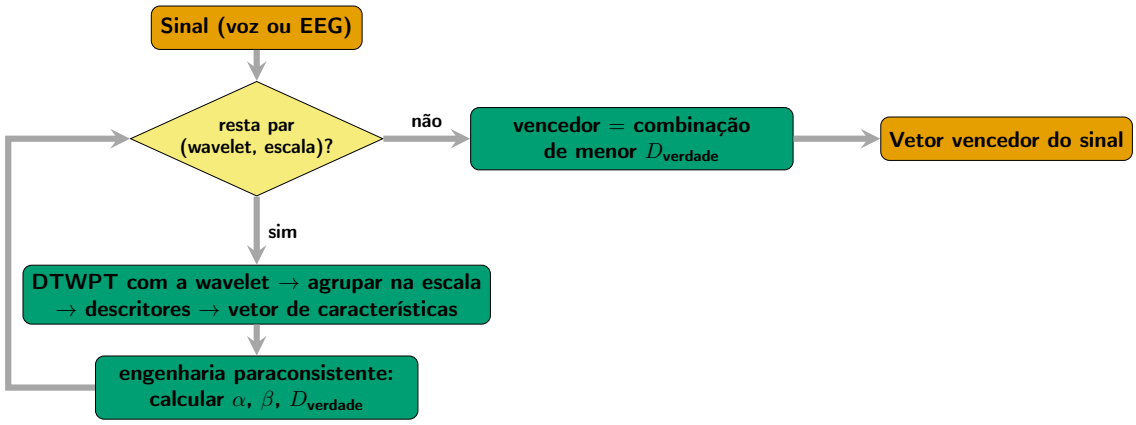
            if (d < vencedor.d_verdade) {   /* mais separavel    */
                vencedor.d_verdade = d;
                vencedor.estrategia = HANDCRAFTED;
                vencedor.wavelet = WAVELETS[w];
                vencedor.escala  = ESCALAS[e];
            }
        }
    }

    /* 2) parte automatizada: um autoencoder (sem wavelet/escala) */
    double vetor_ae[MAX_CARACT];
    int mae = extrai_autoencoder(sinal, n, vetor_ae);
    double d_ae = calcula_d_verdade(vetor_ae, mae);
    if (d_ae < vencedor.d_verdade) {
        vencedor.d_verdade = d_ae;
        vencedor.estrategia = AUTOENCODER;
```

```
37      }
38
39      return vencedor;                        /* NAO treina classificador */
40  }
```

Algoritmo 3.1 – Fase 00 — varredura de extratores e seleção do vencedor por sinal via engenharia paraconsistente. Executada uma vez para a voz e uma vez para o EEG.

Figura 44 – Seleção do vetor vencedor de um sinal na Fase 00



Fonte: Elaborado pelo autor, 2026.

Considerando os dois sinais, a Fase 00 tem $(138 + 12 + 3)$ execuções para a voz e $(46 + 12 + 3)$ para o EEG, isto é, $153 + 61 = 214$ execuções de ranking. O vetor fundido não é varrido aqui: ele é montado na Fase 01, a partir dos vencedores de cada sinal.

3.2.1.2 Fase 01 — autenticação com o vetor vencedor

A Fase 01 usa *apenas* a combinação vencedora da Fase 00 — o eixo do extrator não é mais variado, pois a engenharia paraconsistente já o fixou. Variam-se quatro eixos, detalhados adiante: a **fonte** do sinal, o **modo de texto**, o **esquema de validação cruzada** e a **padronização das características**. São $4 \times 2 \times 2 \times 2 = 32$ execuções, todas com o classificador DSNN. O Algoritmo 3.2 mostra a montagem da fonte e o laço de autenticação; a Figura 45 resume o fluxo.

3.2.1.2.1 Arquitetura do classificador DSNN.

O classificador é uma rede de pulsos residual profunda, construída a partir da especificação declarativa `["linear:128:relu", "residual:2", "linear:N:identity"]` do perfil, onde N é o número de locutores. Concretamente, ela é composta por:

- um **estágio de entrada** `Linear`$(d \to 128)$ seguido de um neurônio LIF, onde d é a dimensão do vetor de características vindo da Fase 00;

- **dois blocos residuais** idênticos, cada um `Linear`$(128 \to 128)$ seguido de LIF, envoltos por uma conexão de atalho identidade, $h_{\text{saída}} = \text{bloco}(h) + h$. Os atalhos não têm parâmetros e existem para que o gradiente alcance as camadas iniciais sem se dissipar através da não-linearidade de disparo — é o mesmo desenho de SNN residual profunda de (ZHENG *et al.*, 2021), para o qual a tdBN foi originalmente proposta. Os estágios de entrada e de saída não recebem atalho porque suas larguras diferem de 128, o que impediria a soma elemento a elemento;
- um **estágio de saída** `Linear`$(128 \to N)$, cuja saída é a média dos logits sobre os T passos de tempo.

O treino usa retropropagação através do tempo (BPTT) com gradiente substituto, e a regularização da taxa de disparo descrita adiante mantém cada camada pulsante fora dos regimes de neurônio morto e saturado.

3.2.1.2.2 Protocolo de verificação: como EER e AUC são calculados.

Esta é a parte do protocolo que a definição textual de EER[5] não revela, e sem a qual os números da Capítulo 4 não são interpretáveis. A camada de saída tem uma unidade por locutor de *treino*, mas ela **não** é usada como decisão de classe — sob o protocolo de verificação os locutores de teste nunca aparecem no treino, de modo que nenhuma dessas unidades corresponde ao indivíduo sendo verificado. O vetor de logits é usado, em vez disso, como um **vetor de representação** (*embedding*) do sinal. A partir dele:

1. cada par de amostras de teste forma uma **tentativa** (*trial*), rotulada *genuína* quando as duas amostras pertencem ao mesmo locutor e *impostora* caso contrário;
2. a pontuação de cada tentativa é a **similaridade de cosseno** entre os dois vetores de representação;
3. varre-se um limiar sobre essas pontuações; o EER é o ponto em que a taxa de falsa aceitação iguala a de falsa rejeição;
4. a AUC[6] é estimada pela estatística de Wilcoxon–Mann–Whitney, AUC $= P(\text{pontuação genuína} > \text{pontuação impostora})$, que equivale à área sob a curva ROC[7] das duas distribuições de pontuação.

É por isso que as métricas de conjunto fechado são `NaN` nos resultados: elas exigiriam que a identidade de teste estivesse entre as classes treinadas, o que este protocolo deliberadamente proíbe.

[5] Taxa de Erro Igual
[6] Área sob a Curva (*Area Under the Curve*)
[7] Característica de Operação do Receptor (*Receiver Operating Characteristic*)

3.2.1.2.3 Os quatro eixos experimentais.

Cada eixo é binário ou quaternário, e o experimento é o produto cartesiano completo:

- **Fonte** (4 níveis): apenas voz, apenas EEG, fundida-precoce e fundida-tardia — os três modos da seção 3.2, com o modo fundido desdobrado nas duas variantes de instante de fusão;
- **Modo de texto** (2 níveis): *dependente*, em que o mesmo enunciado aparece no treino e no teste, e *independente*, em que os enunciados de teste podem ser quaisquer outros. É o eixo da Questão 4 (Capítulo 1), e mede o quanto do desempenho vem da identidade do locutor e não do conteúdo falado;
- **Esquema de validação cruzada** (2 níveis). Em ambos os esquemas as partições são *disjuntas por locutor* (*group k-fold*, $k = 5$), o que é o que torna o protocolo de verificação válido. Na **plana**, cada partição é treinada e pontuada uma vez, sem conjunto de validação e sem parada antecipada. Na **aninhada**, um laço interno — também $k = 5$, com semente distinta — separa um conjunto de validação usado para selecionar o melhor modelo por parada antecipada, e só então o modelo escolhido é pontuado uma única vez na partição externa. A aninhada é o estimador não enviesado quando há seleção de modelo; a plana é mais barata e não descarta dados para validação. Os dois são comparados justamente porque a diferença entre eles mede o custo dessa seleção;
- **Padronização das características** (2 níveis): *z-score* por característica ligado ou desligado. Quando ligado, média e desvio padrão são estimados *apenas* sobre a partição de treino de cada *fold* e então aplicados à de teste, evitando vazamento de informação. É uma ablação do efeito da normalização.

3.2.1.2.4 Protocolo de treino e de repetição.

Todas as 32 execuções compartilham os mesmos hiperparâmetros: otimizador Adam com taxa de aprendizado 10^{-3}, 50 épocas, lotes de 32 amostras, e paciência de 10 épocas para a parada antecipada (ativa apenas na validação cruzada aninhada, já que a plana não reserva conjunto de validação). Cada configuração é executada **três vezes**, com sementes distintas derivadas da semente base 42; os valores reportados no Capítulo 4 são as médias sobre essas três repetições, e o desvio padrão ali tabelado é calculado entre elas. Como cada repetição percorre $k = 5$ partições, cada configuração contribui com 15 medidas de EER.

```c
/* v_voz, v_eeg: vencedores da Fase 00 (um por sinal)             */
/* fonte : VOZ | EEG | FUNDIDA_PRECOCE | FUNDIDA_TARDIA          */
/* Preenche X (vetores de caracteristicas) e devolve a dimensao. */

int monta_fonte(Amostra a, int fonte,
                Combinacao v_voz, Combinacao v_eeg, double X[])
{
    if (fonte == VOZ)
        return extrai(a.audio, v_voz, X);
    if (fonte == EEG)
        return extrai(a.eeg,   v_eeg, X);

    if (fonte == FUNDIDA_PRECOCE) {         /* funde sinal bruto */
        double s[MAX_AMOSTRAS];
        int n = concatena(a.audio, a.eeg, s);
        return extrai(s, v_voz, X);         /* uma extracao so   */
    }
    /* FUNDIDA_TARDIA: extrai cada sinal, concatena os vetores   */
    double xv[MAX_CARACT], xe[MAX_CARACT];
    int mv = extrai(a.audio, v_voz, xv);
    int me = extrai(a.eeg,   v_eeg, xe);
    return concatena_vetores(xv, mv, xe, me, X);
}

/* Loop de autenticacao: fonte X modo de texto X esquema de CV.  */
Metricas autentica(Amostra base[], int fonte,
                   int modo_texto, int cv_aninhada,
                   Combinacao v_voz, Combinacao v_eeg)
{
    double X[MAX_AMOSTRAS][MAX_CARACT];
    for (int i = 0; i < N_AMOSTRAS; i++)
        monta_fonte(base[i], fonte, v_voz, v_eeg, X[i]);

    /* grupos disjuntos por locutor (protocolo de verificacao)   */
    Folds f = divide_por_locutor(base, modo_texto, cv_aninhada);

    Metricas m = treina_e_avalia_dsnn(X, f);  /* LIF + BPTT       */
    return m;                                 /* EER e AUC        */
}
```

Algoritmo 3.2 – Fase 01 — autenticação com o vencedor da Fase 00. O extrator nao e mais variado: usa-se apenas a combinacao escolhida por sinal.

Figura 45 – Autenticação DSNN na Fase 01



Fonte: Elaborado pelo autor, 2026.

3.2.1.3 Organização dos perfis e transição entre fases

Cada execução é descrita por um *perfil* em formato JSON[8]. Os perfis ficam separados por fase: 214 perfis em `profiles/phase00/` (um por combinação varrida) e 32 em `profiles/phase01/` (fonte × texto × CV[9] × padronização). O vencedor da Fase 00 só é conhecido após a execução; por isso os perfis da Fase 01 nascem com um extrator provisório, substituído pelo vencedor antes de rodar.

Dois *scripts* automatizam a transição, evitando edição manual (Figura 43):

- `thesis_phase00_rank.py`: lê os resultados paraconsistentes das 214 execuções — exatamente os arquivos que cada perfil declara, recusando um conjunto incompleto —, calcula a média de $D_{\text{penalizado}}$ sobre as três repetições, escreve o vencedor de cada sinal em `winners.json` e sinaliza explicitamente eventuais empates exatos, para que uma escolha feita por desempate de ordenação não seja confundida com um resultado de mérito;

- `thesis_apply_winner.py`: injeta cada vencedor nos 32 perfis da Fase 01 — voz recebe o vencedor da voz, EEG o do EEG, e as fontes fundidas recebem o vencedor do sinal declarado na chamada; o *pipeline* declara a voz (sinal dominante em número de amostras).

3.2.2 Métricas

A autenticação segue o protocolo de **verificação** (*verification*), no qual os locutores de teste não aparecem no treino: os grupos de validação cruzada são disjuntos por locutor.

[8] *JavaScript Object Notation*
[9] Validação Cruzada

Nesse protocolo, as métricas primárias são a **EER** (o ponto em que a taxa de falsa aceitação iguala a de falsa rejeição) e a **área sob a curva ROC** (AUC). As métricas de classificação de conjunto fechado (acurácia, F1, precisão, recall, especificidade) só se aplicam quando as classes de teste também estão no treino; sob o protocolo de verificação elas não são reportadas — nos arquivos de resultado da Fase 01 elas aparecem literalmente como `NaN`, e não como zero, justamente para que a ausência de valor não seja confundida com um desempenho nulo. As seguintes métricas são adotadas, garantindo a comparação com outros estudos:

- **Taxa de Erro Igual (EER)** — primária, protocolo de verificação;
- **área sob a Curva ROC (AUC)** — primária, protocolo de verificação;
- acurácia, pontuação F1, precisão, recall, sensitividade, especificidade — reportadas apenas em avaliações de classificação de conjunto fechado;
- Erro Médio Quadrático (EQM) — usado na reconstrução dos *autoencoders*, não na classificação.

3.2.3 Desenvolvimento e Validação de Algoritmos

Quanto aos algoritmos de classificação, empregam-se redes neurais de pulso em uma arquitetura residual (a DSNN detalhada na subseção 3.2.1.2), com o objetivo de minimizar o gasto de energia, memória e processamento, adequando-se a dispositivos com baixo poder computacional para garantir portabilidade e eficiência energética.

Há, no entanto, um obstáculo prático: o número de locutores e de amostras por locutor é pequeno. Com poucos dados, a rede tende a "decorar" os exemplos de treino em vez de aprender padrões gerais — o chamado *overfitting*. Para combatê-lo, usam-se duas técnicas de regularização complementares, descritas em detalhe no Apêndice A.

A primeira é o **decaimento de peso L2**, que penaliza pesos muito grandes e, com isso, favorece soluções mais simples e que generalizam melhor. Nas redes de pulso, atua apenas sobre as matrizes de peso; os parâmetros biofísicos dos neurônios de pulso (R, C e V_{th}) são preservados, pois reduzi-los descaracterizaria o neurônio.

A segunda é a **regularização da taxa de disparo**, exclusiva das redes de pulso. Ela mantém a frequência média de disparos de cada camada dentro de uma faixa saudável, evitando dois extremos: neurônios mortos, que param de disparar e interrompem o aprendizado, e neurônios saturados, que disparam o tempo todo e perdem a capacidade de distinguir entradas. Como bônus, ao incentivar disparos comedidos, essa técnica reforça a própria razão de ser das redes de pulso: a eficiência energética.

4 Testes e Resultados

Este capítulo apresenta os resultados das duas fases do Experimento 05, na ordem em que foram executadas: a Fase 00, que constrói e ordena os vetores de características segundo o critério paraconsistente (seção 4.1), e a Fase 01, que treina o classificador de autenticação sobre a combinação vencedora e mede EER e AUC (seção 4.2). A seção 4.3 retoma, à luz desses resultados, as quatro questões de pesquisa formuladas no Capítulo 1. O protocolo experimental — eixos, arquitetura do classificador e forma de cálculo das métricas — está descrito no Capítulo 3.

4.1 Fase 00 — Ranking por grau de verdade paraconsistente

As tabelas a seguir apresentam todas as configurações avaliadas na fase 00 do Experimento 05, ordenadas de forma **crescente** pelo grau de verdade paraconsistente penalizado médio ($\bar{d}_{\text{pen}}$). Ambas as grandezas tabeladas são *distâncias* ao vértice *Verdade* do plano paraconsistente, de modo que **quanto menor, melhor**: a posição #1 de cada tabela é a melhor configuração daquela família, e não a pior.

Aqui, d denota o grau de verdade paraconsistente de uma única repetição — a mesma grandeza chamada D_{verdade} no Capítulo 3 e $D_{1,0}$ na Seção 2.1 — calculado a partir dos valores de α (grau de crença) e β (grau de descrença) obtidos nas repetições de cada configuração; $d_{\text{pen}} = d + \lambda|G_2|$ é a versão penalizada por contradição definida na Seção 2.1.5.1 (com $\lambda = 2 - \sqrt{2}$). $\bar{\alpha}$ e $\bar{\beta}$ são as médias de α e β nessas mesmas repetições, e σ_d é o desvio padrão populacional de d entre as três repetições, indicando a estabilidade da configuração.

O critério de ordenação é $\bar{d}_{\text{pen}}$, e não $\bar{d}$. Essa escolha é a discutida na Seção 2.1.5.1 e não é uma formalidade: as duas ordens de fato divergem. Para a voz, a configuração de menor $\bar{d}$ é a *Daubechies* 22 + LFCC (c1), mas o vencedor é a Haar + LFCC (c1), porque o termo $\lambda|G_2|$ penaliza a primeira por contradição. A divergência é ainda mais consequente entre famílias: pelo $\bar{d}$ isolado, o melhor *autoencoder* do EEG (1,5046) *superaria* o extrator manual vencedor (1,5143), e apenas a penalização por contradição — que reconhece o latente quase colapsado desses *autoencoders*, com $\bar{\beta}$ acima de 0,98 — inverte a comparação. Por isso as duas colunas são exibidas lado a lado.

O vencedor de cada sinal, marcado com †, é escolhido *entre as duas famílias* (manual e *autoencoder*), e não dentro de cada tabela: para ambos os sinais venceu a extração manual com Haar + LFCC (c1), de modo que a marca aparece apenas nas Tabelas 10 e 11. A posição #1 das Tabelas 8 e 9 é, portanto, o melhor *autoencoder*, não o vencedor

da fase.

As Tabelas 8 e 9 comparam as variantes de *autoencoder* (ANN-AE e SNN-AE nas codificações direta, de latência e de Poisson) para EEG e voz, respectivamente. As Tabelas 10 e 11 comparam as configurações de características manuais (família *wavelet* × característica espectral × categoria) para EEG e voz. Os dados das tabelas são gerados automaticamente pelo script `software/nn/scripts/pipeline/thesis/thesis_build_phase00_paraconsistent_tables.py` a partir dos arquivos `results/thesis/phase00/*_summary.json`, e devem ser regenerados sempre que a fase 00 for reexecutada; o script escreve apenas os arquivos `.csv` em `tables/`.

Tabela 8 – Configurações de autoencoder (EEG) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor)

#	Configuração	$\bar{d}_{\text{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
1	ANN-AE (tiny)	1.8531	1.5601	0.0598	0.5144	0.9859
2	ANN-AE (base)	1.8624	1.4924	0.0198	0.6404	0.9912
3	ANN-AE (small)	1.8624	1.4924	0.0198	0.6404	0.9912
4	SNN-AE direct (tiny)	1.8894	1.4989	0.0582	0.6667	1.0000
5	SNN-AE latency (tiny)	1.9348	1.8696	0.0086	0.1226	0.9888
6	SNN-AE latency (base)	1.9546	1.9186	0.0087	0.0719	0.9895
7	SNN-AE latency (small)	1.9546	1.9186	0.0087	0.0719	0.9895
8	SNN-AE direct (base)	1.9832	1.9588	0.0144	0.0417	0.9999
9	SNN-AE direct (small)	1.9832	1.9588	0.0144	0.0417	0.9999
10	SNN-AE poisson (base)	1.9958	1.9945	0.0015	0.0020	0.9965
11	SNN-AE poisson (small)	1.9958	1.9945	0.0015	0.0020	0.9965
12	SNN-AE poisson (tiny)	1.9998	1.9995	0.0004	0.0000	0.9995

Tabela 9 – Configurações de autoencoder (voz) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor)

#	Configuração	$\bar{d}_{\text{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
1	ANN-AE (base)	1.8364	1.5423	0.0222	0.5223	0.9797
2	ANN-AE (small)	1.8364	1.5423	0.0222	0.5223	0.9797
3	ANN-AE (tiny)	1.8383	1.5058	0.0420	0.5888	0.9788
4	SNN-AE direct (tiny)	1.9021	1.4874	0.0663	0.7083	0.9995
5	SNN-AE latency (tiny)	1.9074	1.8365	0.0114	0.1443	0.9767
6	SNN-AE direct (base)	1.9387	1.8414	0.0527	0.1667	0.9994
7	SNN-AE direct (small)	1.9387	1.8414	0.0527	0.1667	0.9994
8	SNN-AE latency (base)	1.9488	1.9157	0.0007	0.0708	0.9857
9	SNN-AE latency (small)	1.9488	1.9157	0.0007	0.0708	0.9857
10	SNN-AE poisson (base)	1.9820	1.9768	0.0043	0.0072	0.9840
11	SNN-AE poisson (small)	1.9820	1.9768	0.0043	0.0072	0.9840

#	Configuração	$\bar{d}_{\mathbf{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
12	SNN-AE poisson (tiny)	1.9950	1.9878	0.0003	0.0000	0.9877

Tabela 10 – Configurações de características manuais (EEG) ordenadas por d_{pen} crescente (menor = melhor); † = vencedor do sinal

#	Configuração	$\bar{d}_{\mathbf{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
1†	Haar + LFCC (c1)	1.5883	1.5143	0.0000	0.3087	0.8177
2	Haar + LFCC (c2)	1.6345	1.5348	0.0000	0.3225	0.8478
3	Daubechies 4 + LFCC (c1)	1.7644	1.6630	0.0000	0.2596	0.9136
4	Daubechies 4 + LFCC (c2)	1.7802	1.6580	0.0000	0.2818	0.9267
5	Daubechies 28 + LFCC (c2)	1.7905	1.6403	0.0000	0.3181	0.9382
6	Daubechies 26 + LFCC (c2)	1.7933	1.6458	0.0000	0.3127	0.9391
7	Daubechies 30 + LFCC (c2)	1.7949	1.6513	0.0000	0.3061	0.9391
8	Daubechies 6 + LFCC (c1)	1.7963	1.6922	0.0000	0.2474	0.9303
9	Daubechies 40 + LFCC (c2)	1.7963	1.6511	0.0000	0.3078	0.9401
10	Daubechies 20 + LFCC (c2)	1.7966	1.6520	0.0000	0.3067	0.9402
11	Daubechies 24 + LFCC (c2)	1.7991	1.6555	0.0000	0.3039	0.9412
12	Daubechies 14 + LFCC (c2)	1.7995	1.6465	0.0000	0.3179	0.9435
13	Daubechies 18 + LFCC (c2)	1.7996	1.6578	0.0000	0.3011	0.9411
14	Daubechies 34 + LFCC (c2)	1.7998	1.6612	0.0000	0.2962	0.9404
15	Daubechies 46 + LFCC (c2)	1.7999	1.6597	0.0000	0.2985	0.9409
16	Daubechies 36 + LFCC (c2)	1.8006	1.6598	0.0000	0.2991	0.9414
17	Daubechies 44 + LFCC (c2)	1.8009	1.6649	0.0000	0.2918	0.9404
18	Daubechies 32 + LFCC (c2)	1.8011	1.6675	0.0000	0.2881	0.9399
19	Daubechies 38 + LFCC (c2)	1.8011	1.6640	0.0000	0.2932	0.9407
20	Daubechies 22 + LFCC (c2)	1.8011	1.6612	0.0000	0.2974	0.9414
21	Daubechies 42 + LFCC (c2)	1.8011	1.6641	0.0000	0.2931	0.9407
22	Daubechies 6 + LFCC (c2)	1.8016	1.6770	0.0000	0.2746	0.9381
23	Daubechies 16 + LFCC (c2)	1.8024	1.6619	0.0000	0.2978	0.9422
24	Daubechies 28 + LFCC (c1)	1.8025	1.6762	0.0000	0.2767	0.9390
25	Daubechies 30 + LFCC (c1)	1.8044	1.6838	0.0000	0.2674	0.9385
26	Daubechies 10 + LFCC (c2)	1.8045	1.6593	0.0000	0.3036	0.9443
27	Daubechies 26 + LFCC (c1)	1.8056	1.6820	0.0000	0.2711	0.9398
28	Daubechies 40 + LFCC (c1)	1.8061	1.6840	0.0000	0.2686	0.9397
29	Daubechies 8 + LFCC (c1)	1.8068	1.7042	0.0000	0.2399	0.9351
30	Daubechies 20 + LFCC (c1)	1.8083	1.6867	0.0000	0.2669	0.9407
31	Daubechies 8 + LFCC (c2)	1.8085	1.6809	0.0000	0.2755	0.9423
32	Daubechies 46 + LFCC (c1)	1.8099	1.6916	0.0000	0.2612	0.9407
33	Daubechies 10 + LFCC (c1)	1.8107	1.6943	0.0000	0.2581	0.9407
34	Daubechies 34 + LFCC (c1)	1.8108	1.6956	0.0000	0.2563	0.9404

#	Configuração	$\bar{d}_{\text{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
35	Daubechies 24 + LFCC (c1)	1.8110	1.6920	0.0000	0.2617	0.9414
36	Daubechies 18 + LFCC (c1)	1.8111	1.6935	0.0000	0.2596	0.9411
37	Daubechies 22 + LFCC (c1)	1.8111	1.6941	0.0000	0.2588	0.9410
38	Daubechies 12 + LFCC (c2)	1.8115	1.6765	0.0000	0.2850	0.9455
39	Daubechies 44 + LFCC (c1)	1.8119	1.6991	0.0000	0.2522	0.9403
40	Daubechies 36 + LFCC (c1)	1.8121	1.6963	0.0000	0.2564	0.9412
41	Daubechies 14 + LFCC (c1)	1.8124	1.6841	0.0000	0.2746	0.9444
42	Daubechies 16 + LFCC (c1)	1.8124	1.6940	0.0000	0.2601	0.9420
43	Daubechies 42 + LFCC (c1)	1.8125	1.6988	0.0000	0.2533	0.9409
44	Daubechies 32 + LFCC (c1)	1.8128	1.7029	0.0000	0.2476	0.9401
45	Daubechies 38 + LFCC (c1)	1.8132	1.7004	0.0000	0.2516	0.9410
46	Daubechies 12 + LFCC (c1)	1.8191	1.7081	0.0000	0.2459	0.9435

Tabela 11 – Configurações de características manuais (voz) ordenadas por $\bar{d}_{\text{pen}}$ crescente (menor = melhor); † = vencedor do sinal

#	Configuração	$\bar{d}_{\text{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
1†	Haar + LFCC (c1)	1.4835	1.4053	0.0000	0.3674	0.7663
2	Haar + MEL (c1)	1.4972	1.4189	0.0000	0.3606	0.7731
3	Daubechies 22 + LFCC (c1)	1.4973	1.3929	0.0000	0.3984	0.7798
4	Daubechies 26 + LFCC (c1)	1.4988	1.3983	0.0000	0.3920	0.7797
5	Daubechies 30 + LFCC (c1)	1.5011	1.4039	0.0000	0.3859	0.7800
6	Daubechies 42 + LFCC (c1)	1.5014	1.4043	0.0000	0.3856	0.7801
7	Daubechies 20 + LFCC (c1)	1.5016	1.4075	0.0000	0.3812	0.7795
8	Daubechies 32 + LFCC (c1)	1.5016	1.4068	0.0000	0.3822	0.7797
9	Daubechies 38 + LFCC (c1)	1.5018	1.4058	0.0000	0.3838	0.7800
10	Daubechies 36 + LFCC (c1)	1.5019	1.4024	0.0000	0.3889	0.7810
11	Daubechies 40 + LFCC (c1)	1.5021	1.4077	0.0000	0.3814	0.7798
12	Daubechies 16 + LFCC (c1)	1.5022	1.4014	0.0000	0.3907	0.7814
13	Daubechies 46 + LFCC (c1)	1.5028	1.4094	0.0000	0.3796	0.7799
14	Daubechies 34 + LFCC (c1)	1.5037	1.4098	0.0000	0.3797	0.7805
15	Daubechies 28 + LFCC (c1)	1.5041	1.4127	0.0000	0.3760	0.7800
16	Daubechies 18 + LFCC (c1)	1.5043	1.4078	0.0000	0.3832	0.7814
17	Daubechies 14 + LFCC (c1)	1.5043	1.4078	0.0000	0.3833	0.7814
18	Daubechies 24 + LFCC (c1)	1.5043	1.4119	0.0000	0.3774	0.7804
19	Daubechies 44 + LFCC (c1)	1.5052	1.4125	0.0000	0.3773	0.7809
20	Daubechies 12 + LFCC (c1)	1.5058	1.4097	0.0000	0.3819	0.7821
21	Haar + BARK (c1)	1.5099	1.4243	0.0000	0.3647	0.7815
22	Daubechies 4 + LFCC (c1)	1.5145	1.4478	0.0000	0.3353	0.7786
23	Daubechies 18 + MEL (c1)	1.5191	1.4123	0.0000	0.3909	0.7914

#	Configuração	$\bar{d}_{\mathbf{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
24	Daubechies 32 + MEL (c1)	1.5199	1.4188	0.0000	0.3822	0.7904
25	Daubechies 22 + MEL (c1)	1.5200	1.4180	0.0000	0.3834	0.7907
26	Daubechies 26 + MEL (c1)	1.5203	1.4178	0.0000	0.3840	0.7909
27	Daubechies 16 + MEL (c1)	1.5205	1.4164	0.0000	0.3862	0.7914
28	Daubechies 20 + MEL (c1)	1.5211	1.4232	0.0000	0.3768	0.7902
29	Daubechies 30 + MEL (c1)	1.5211	1.4224	0.0000	0.3781	0.7905
30	Daubechies 40 + MEL (c1)	1.5217	1.4218	0.0000	0.3795	0.7910
31	Daubechies 24 + MEL (c1)	1.5218	1.4220	0.0000	0.3794	0.7911
32	Daubechies 28 + MEL (c1)	1.5228	1.4241	0.0000	0.3772	0.7913
33	Daubechies 34 + MEL (c1)	1.5229	1.4245	0.0000	0.3767	0.7913
34	Daubechies 14 + MEL (c1)	1.5233	1.4207	0.0000	0.3826	0.7925
35	Daubechies 42 + MEL (c1)	1.5234	1.4255	0.0000	0.3757	0.7914
36	Daubechies 10 + LFCC (c1)	1.5234	1.4546	0.0000	0.3339	0.7837
37	Daubechies 36 + MEL (c1)	1.5236	1.4228	0.0000	0.3800	0.7923
38	Daubechies 44 + MEL (c1)	1.5252	1.4283	0.0000	0.3733	0.7920
39	Daubechies 38 + MEL (c1)	1.5252	1.4295	0.0000	0.3716	0.7917
40	Daubechies 8 + LFCC (c1)	1.5253	1.4580	0.0000	0.3307	0.7842
41	Daubechies 46 + MEL (c1)	1.5258	1.4338	0.0000	0.3659	0.7911
42	Daubechies 12 + MEL (c1)	1.5265	1.4318	0.0000	0.3696	0.7922
43	Daubechies 4 + MEL (c1)	1.5294	1.4577	0.0000	0.3349	0.7874
44	Daubechies 6 + LFCC (c1)	1.5342	1.4772	0.0000	0.3117	0.7857
45	Daubechies 22 + BARK (c1)	1.5385	1.4382	0.0000	0.3717	0.7996
46	Daubechies 10 + MEL (c1)	1.5395	1.4595	0.0000	0.3417	0.7948
47	Daubechies 32 + BARK (c1)	1.5404	1.4448	0.0000	0.3638	0.7993
48	Daubechies 4 + BARK (c1)	1.5406	1.4543	0.0000	0.3502	0.7970
49	Daubechies 26 + BARK (c1)	1.5406	1.4448	0.0000	0.3639	0.7995
50	Daubechies 30 + BARK (c1)	1.5413	1.4450	0.0000	0.3644	0.8000
51	Daubechies 24 + BARK (c1)	1.5415	1.4470	0.0000	0.3617	0.7997
52	Daubechies 20 + BARK (c1)	1.5427	1.4503	0.0000	0.3580	0.7997
53	Daubechies 36 + BARK (c1)	1.5432	1.4459	0.0000	0.3648	0.8012
54	Daubechies 8 + MEL (c1)	1.5433	1.4737	0.0000	0.3250	0.7939
55	Daubechies 42 + BARK (c1)	1.5433	1.4480	0.0000	0.3620	0.8008
56	Daubechies 34 + BARK (c1)	1.5436	1.4499	0.0000	0.3594	0.8005
57	Daubechies 40 + BARK (c1)	1.5436	1.4493	0.0000	0.3604	0.8007
58	Daubechies 14 + BARK (c1)	1.5440	1.4466	0.0000	0.3647	0.8016
59	Daubechies 12 + BARK (c1)	1.5445	1.4470	0.0000	0.3645	0.8019
60	Daubechies 28 + BARK (c1)	1.5446	1.4531	0.0000	0.3557	0.8004
61	Daubechies 16 + BARK (c1)	1.5450	1.4527	0.0000	0.3567	0.8009

#	**Configuração**	$\bar{d}_{\mathbf{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
62	Daubechies 44 + BARK (c1)	1.5451	1.4499	0.0000	0.3609	0.8016
63	Daubechies 46 + BARK (c1)	1.5451	1.4536	0.0000	0.3555	0.8007
64	Daubechies 18 + BARK (c1)	1.5459	1.4536	0.0000	0.3563	0.8013
65	Daubechies 38 + BARK (c1)	1.5476	1.4581	0.0000	0.3513	0.8015
66	Daubechies 6 + MEL (c1)	1.5517	1.4893	0.0000	0.3104	0.7960
67	Daubechies 10 + BARK (c1)	1.5572	1.4722	0.0000	0.3400	0.8050
68	Daubechies 8 + BARK (c1)	1.5631	1.4925	0.0000	0.3164	0.8041
69	Daubechies 6 + BARK (c1)	1.5638	1.4903	0.0000	0.3203	0.8053
70	Haar + MEL (c2)	1.6068	1.5117	0.0000	0.3296	0.8326
71	Haar + LFCC (c2)	1.6076	1.5045	0.0000	0.3408	0.8351
72	Haar + BARK (c2)	1.6123	1.5129	0.0000	0.3332	0.8365
73	Daubechies 22 + LFCC (c2)	1.6199	1.4893	0.0000	0.3752	0.8478
74	Daubechies 32 + MEL (c2)	1.6205	1.4913	0.0000	0.3728	0.8477
75	Daubechies 26 + MEL (c2)	1.6207	1.4898	0.0000	0.3753	0.8482
76	Daubechies 22 + MEL (c2)	1.6211	1.4929	0.0000	0.3710	0.8478
77	Daubechies 42 + LFCC (c2)	1.6213	1.4934	0.0000	0.3705	0.8479
78	Daubechies 30 + MEL (c2)	1.6215	1.4942	0.0000	0.3695	0.8478
79	Daubechies 26 + LFCC (c2)	1.6216	1.4958	0.0000	0.3673	0.8475
80	Daubechies 18 + MEL (c2)	1.6216	1.4896	0.0000	0.3765	0.8489
81	Daubechies 20 + MEL (c2)	1.6216	1.4968	0.0000	0.3658	0.8473
82	Daubechies 24 + MEL (c2)	1.6218	1.4936	0.0000	0.3708	0.8482
83	Daubechies 30 + LFCC (c2)	1.6220	1.4966	0.0000	0.3664	0.8476
84	Daubechies 16 + MEL (c2)	1.6221	1.4917	0.0000	0.3737	0.8488
85	Daubechies 40 + MEL (c2)	1.6222	1.4941	0.0000	0.3704	0.8484
86	Daubechies 32 + LFCC (c2)	1.6225	1.4998	0.0000	0.3622	0.8473
87	Daubechies 34 + MEL (c2)	1.6227	1.4941	0.0000	0.3709	0.8487
88	Daubechies 42 + MEL (c2)	1.6229	1.4944	0.0000	0.3705	0.8488
89	Daubechies 46 + LFCC (c2)	1.6233	1.5001	0.0000	0.3626	0.8478
90	Daubechies 38 + LFCC (c2)	1.6233	1.4995	0.0000	0.3634	0.8480
91	Daubechies 40 + LFCC (c2)	1.6234	1.5010	0.0000	0.3612	0.8476
92	Daubechies 28 + MEL (c2)	1.6235	1.4968	0.0000	0.3676	0.8487
93	Daubechies 20 + LFCC (c2)	1.6238	1.5036	0.0000	0.3578	0.8474
94	Daubechies 36 + LFCC (c2)	1.6239	1.4976	0.0000	0.3668	0.8488
95	Daubechies 36 + MEL (c2)	1.6242	1.4949	0.0000	0.3711	0.8496
96	Daubechies 34 + LFCC (c2)	1.6244	1.5015	0.0000	0.3615	0.8483
97	Daubechies 38 + MEL (c2)	1.6250	1.4992	0.0000	0.3656	0.8492
98	Daubechies 16 + LFCC (c2)	1.6251	1.5008	0.0000	0.3632	0.8489
99	Daubechies 46 + MEL (c2)	1.6253	1.5025	0.0000	0.3609	0.8487

#	**Configuração**	$\bar{d}_{\mathbf{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
100	Daubechies 28 + LFCC (c2)	1.6253	1.5065	0.0000	0.3550	0.8478
101	Daubechies 44 + MEL (c2)	1.6254	1.4997	0.0000	0.3651	0.8494
102	Daubechies 44 + LFCC (c2)	1.6254	1.5035	0.0000	0.3595	0.8486
103	Daubechies 22 + BARK (c2)	1.6257	1.5017	0.0000	0.3624	0.8492
104	Daubechies 24 + LFCC (c2)	1.6257	1.5065	0.0000	0.3554	0.8481
105	Daubechies 14 + MEL (c2)	1.6257	1.4997	0.0000	0.3655	0.8497
106	Daubechies 32 + BARK (c2)	1.6260	1.5064	0.0000	0.3558	0.8483
107	Daubechies 30 + BARK (c2)	1.6269	1.5055	0.0000	0.3580	0.8492
108	Daubechies 26 + BARK (c2)	1.6270	1.5073	0.0000	0.3555	0.8489
109	Daubechies 18 + LFCC (c2)	1.6271	1.5077	0.0000	0.3550	0.8489
110	Daubechies 42 + BARK (c2)	1.6272	1.5039	0.0000	0.3607	0.8498
111	Daubechies 24 + BARK (c2)	1.6275	1.5087	0.0000	0.3539	0.8489
112	Daubechies 14 + LFCC (c2)	1.6278	1.5105	0.0000	0.3515	0.8487
113	Daubechies 34 + BARK (c2)	1.6281	1.5078	0.0000	0.3557	0.8495
114	Daubechies 12 + MEL (c2)	1.6281	1.5078	0.0000	0.3558	0.8496
115	Daubechies 12 + LFCC (c2)	1.6287	1.5104	0.0000	0.3525	0.8494
116	Daubechies 36 + BARK (c2)	1.6289	1.5079	0.0000	0.3564	0.8501
117	Daubechies 46 + BARK (c2)	1.6289	1.5099	0.0000	0.3535	0.8497
118	Daubechies 20 + BARK (c2)	1.6293	1.5131	0.0000	0.3492	0.8492
119	Daubechies 40 + BARK (c2)	1.6293	1.5107	0.0000	0.3528	0.8498
120	Daubechies 44 + BARK (c2)	1.6300	1.5096	0.0000	0.3550	0.8505
121	Daubechies 28 + BARK (c2)	1.6312	1.5167	0.0000	0.3457	0.8498
122	Daubechies 4 + MEL (c2)	1.6313	1.5352	0.0000	0.3189	0.8453
123	Daubechies 38 + BARK (c2)	1.6321	1.5158	0.0000	0.3479	0.8506
124	Daubechies 16 + BARK (c2)	1.6327	1.5172	0.0000	0.3464	0.8507
125	Daubechies 14 + BARK (c2)	1.6330	1.5154	0.0000	0.3493	0.8514
126	Daubechies 12 + BARK (c2)	1.6337	1.5160	0.0000	0.3492	0.8518
127	Daubechies 18 + BARK (c2)	1.6342	1.5205	0.0000	0.3430	0.8511
128	Daubechies 4 + LFCC (c2)	1.6361	1.5447	0.0000	0.3096	0.8464
129	Daubechies 4 + BARK (c2)	1.6373	1.5391	0.0000	0.3189	0.8488
130	Daubechies 10 + MEL (c2)	1.6427	1.5415	0.0000	0.3204	0.8522
131	Daubechies 8 + MEL (c2)	1.6452	1.5525	0.0000	0.3069	0.8513
132	Daubechies 10 + LFCC (c2)	1.6459	1.5547	0.0000	0.3044	0.8513
133	Daubechies 8 + LFCC (c2)	1.6477	1.5585	0.0000	0.3006	0.8516
134	Daubechies 10 + BARK (c2)	1.6490	1.5489	0.0000	0.3157	0.8551
135	Daubechies 6 + MEL (c2)	1.6541	1.5702	0.0000	0.2898	0.8534
136	Daubechies 8 + BARK (c2)	1.6544	1.5676	0.0000	0.2937	0.8543
137	Daubechies 6 + LFCC (c2)	1.6557	1.5758	0.0000	0.2833	0.8531

#	Configuração	$\bar{d}_{\text{pen}}$	$\bar{d}$	σ_d	$\bar{\alpha}$	$\bar{\beta}$
138	Daubechies 6 + BARK (c2)	1.6571	1.5689	0.0000	0.2945	0.8561

4.2 Fase 01 — Autenticação DSNN

A Fase 01 treina a DSNN de autenticação sobre a extração de características vencedora da Fase 00 (Seção 4.1: *handcrafted*, *wavelet* Haar, LFCC, categoria c1, para EEG e voz) e mede EER e AUC por dobra de validação cruzada. A Tabela 12 cobre as 32 configurações do experimento — produto cartesiano de fonte (EEG, voz, fusão precoce, fusão tardia) × modo de texto (dependente, independente) × esquema de validação cruzada (k-fold plano, k-fold aninhado) × padronização de características (bruto, padronizado) — ordenadas de forma crescente por EER médio (menor é melhor) sobre as três repetições de cada configuração. Os dados são gerados automaticamente pelo script `software/nn/scripts/pipeline/thesis/thesis_build_phase01_auth_tables.py` a partir dos arquivos `results/thesis/phase01/*_summary.json`, e devem ser regenerados sempre que a fase 01 for reexecutada.

A melhor configuração (fusão precoce, dependente de texto, k-fold plano, características brutas) atinge EER = 0,4534 e AUC = 0,5452 — próximo do nível de acaso (EER = 0,5, AUC = 0,5 correspondem a um classificador aleatório) e, portanto, ainda distante do desempenho necessário para uma autenticação prática. Três padrões emergem da tabela completa: (i) EEG e as fusões superam voz isoladamente nas configurações mais bem ranqueadas; (ii) a validação cruzada em k-fold plano supera a aninhada dentro de cada combinação de fonte/modo de texto; e (iii) a padronização de características piora sistematicamente o EER nesta fase, apesar de elevar o AUC — uma divergência entre as duas métricas que sugere um limiar de decisão inadequado sob padronização, e não necessariamente uma perda real de poder discriminativo, ficando como investigação futura.

Tabela 12 – Configurações da Fase 01 (autenticação DSNN) ordenadas por EER médio

#	Configuração	EER	σ_{EER}	AU
1	Fusão precoce · dependente de texto · k-fold plano · bruto	0.4534	0.0214	0.545
2	EEG · independente de texto · k-fold plano · bruto	0.4651	0.0365	0.560
3	Fusão tardia · dependente de texto · k-fold plano · bruto	0.4731	0.0472	0.571
4	EEG · dependente de texto · k-fold plano · bruto	0.4768	0.0746	0.553
5	EEG · dependente de texto · k-fold aninhado · bruto	0.4826	0.0411	0.559
6	EEG · independente de texto · k-fold aninhado · bruto	0.5027	0.0824	0.569
7	Fusão precoce · independente de texto · k-fold plano · bruto	0.5039	0.0655	0.561
8	Fusão precoce · dependente de texto · k-fold aninhado · bruto	0.5105	0.0577	0.562

#	Configuração	EER	σ_{EER}	AUC
9	Fusão tardia · independente de texto · k-fold plano · bruto	0.5138	0.0742	0.565
10	Fusão precoce · independente de texto · k-fold aninhado · bruto	0.5235	0.0428	0.581
11	Fusão tardia · independente de texto · k-fold aninhado · bruto	0.5300	0.0186	0.563
12	Fusão tardia · dependente de texto · k-fold aninhado · bruto	0.5614	0.0192	0.573
13	Voz · independente de texto · k-fold plano · padronizado	0.5905	0.0126	0.635
14	Voz · dependente de texto · k-fold plano · bruto	0.5935	0.0641	0.641
15	Voz · independente de texto · k-fold plano · bruto	0.6049	0.0318	0.655
16	Voz · independente de texto · k-fold aninhado · padronizado	0.6134	0.0355	0.664
17	Voz · dependente de texto · k-fold aninhado · bruto	0.6139	0.0126	0.659
18	Voz · dependente de texto · k-fold plano · padronizado	0.6263	0.0336	0.681
19	Voz · independente de texto · k-fold aninhado · bruto	0.6263	0.0216	0.667
20	Voz · dependente de texto · k-fold aninhado · padronizado	0.6270	0.0105	0.686
21	Fusão tardia · independente de texto · k-fold plano · padronizado	0.6344	0.0256	0.686
22	EEG · dependente de texto · k-fold plano · padronizado	0.6536	0.0223	0.727
23	Fusão tardia · independente de texto · k-fold aninhado · padronizado	0.6606	0.0241	0.706
24	Fusão tardia · dependente de texto · k-fold aninhado · padronizado	0.6630	0.0630	0.729
25	Fusão precoce · independente de texto · k-fold plano · padronizado	0.6660	0.0274	0.723
26	EEG · independente de texto · k-fold aninhado · padronizado	0.6670	0.0276	0.723
27	EEG · independente de texto · k-fold plano · padronizado	0.6725	0.0200	0.726
28	EEG · dependente de texto · k-fold aninhado · padronizado	0.6772	0.0531	0.737
29	Fusão tardia · dependente de texto · k-fold plano · padronizado	0.6820	0.0247	0.749
30	Fusão precoce · independente de texto · k-fold aninhado · padronizado	0.6855	0.0356	0.750
31	Fusão precoce · dependente de texto · k-fold aninhado · padronizado	0.6896	0.0338	0.762
32	Fusão precoce · dependente de texto · k-fold plano · padronizado	0.6934	0.0243	0.766

4.3 Retomada das questões de pesquisa

Esta seção confronta as quatro questões formuladas na Capítulo 1 com o que foi efetivamente medido. Duas delas são respondidas pela Fase 00, uma pela Fase 01, e uma — a Questão 3 — só pode ser respondida em parte, por uma limitação de desenho que se explicita adiante.

Questão 1 — O agrupamento espectral influencia a representação?

Sim, mas fracamente, e apenas para a voz. Restringindo a comparação às representações de Categoria 1 do sinal de voz — as únicas em que o eixo é aplicável, pois no EEG as escalas Mel e Bark degeneram na linear (subseção 2.1.3.9) — a melhor

configuração de cada escala é 1,4835 (LFCC, linear), 1,4972 (Mel) e 1,5099 (Bark) em $\bar{d}_{\text{pen}}$. A ordem favorece consistentemente o particionamento linear, mas a amplitude total entre as três escalas (0,026) é pequena diante da diferença que separa as categorias (Questão 2) ou as famílias de extrator (Questão 3). A leitura honesta é que a escala importa, e importa menos do que se poderia supor: nenhuma das escalas cocleares compensou, na voz, o custo de agrupar bandas de forma não uniforme.

Questão 2 — A transformação cepstral melhora a representação?

Não para a voz, onde ela degrada a representação de forma inequívoca; para o EEG, o efeito é ambíguo. Na voz, as 69 configurações de Categoria 1 têm $\bar{d}_{\text{pen}}$ médio de 1,5247 (melhor: 1,4835), contra 1,6285 (melhor: 1,6068) nas 69 configurações de Categoria 2. A separação é *total*, e não apenas de médias: a pior configuração não cepstral (1,5638) ainda supera a melhor cepstral (1,6068), de modo que as duas categorias não se sobrepõem em nenhum ponto do ranking. Como a única diferença entre cada par é a etapa de logaritmo mais DCT, o resultado é atribuível a ela.

No EEG o quadro é distinto e não deve ser reportado como o mesmo achado: a melhor configuração continua sendo de Categoria 1 (1,5883 contra 1,6345), mas na *média* das 23 configurações de cada categoria as duas praticamente empatam, com leve vantagem cepstral (1,7921 contra 1,7980). Ou seja, no EEG a etapa cepstral não custa desempenho de forma sistemática — ela apenas não produz o melhor extrator. A conclusão forte da Questão 2 vale, portanto, para a voz, e não para ambos os sinais.

Uma explicação plausível é que a DCT concentra energia nos primeiros coeficientes e descarta estrutura de banda fina que o critério paraconsistente — sensível à amplitude intraclasse — justamente aproveita. Vale notar que isso não contradiz a utilidade consagrada dos coeficientes cepstrais em reconhecimento de fala: ali o objetivo é representar o *conteúdo* falado de forma robusta ao locutor, ao passo que aqui se busca exatamente o oposto — preservar o que distingue locutores. Uma etapa que normaliza diferenças de trato vocal é benéfica no primeiro caso e prejudicial no segundo.

Questão 3 — Autoencoders superam as técnicas manuais?

[PENDENTE] *Questão ainda em aberto. O que segue responde apenas pelo critério paraconsistente da Fase 00; a comparação das três categorias através de um classificador comum — a forma como a questão foi formulada no Capítulo 1 — ainda precisa ser executada. Ver a limitação ao final desta subseção e o encaminhamento no Capítulo 5.*

Não, segundo o critério paraconsistente — mas a questão, como originalmente formulada, não pôde ser respondida. Pelo critério da Fase 00 a resposta é inequívoca: a melhor configuração manual supera o melhor *autoencoder* com folga em

ambos os sinais (1,5883 contra 1,8491 no EEG; 1,4835 contra 1,8370 na voz), e a extração manual venceu a seleção para os dois. Nenhuma variante de *autoencoder*, em nenhum dos três tamanhos ou três codificações, chegou perto.

A ressalva é de desenho experimental, e é importante o bastante para ser registrada como limitação. A Questão 3 fora formulada supondo que as três categorias seriam comparadas *através de um mesmo classificador*. Não foi o que ocorreu: a Fase 00 compara extratores sem treinar classificador algum, e a Fase 01 treina apenas a combinação vencedora. As Categorias 1, 2 e 3 nunca foram, portanto, confrontadas pelo desempenho de autenticação — apenas pela separabilidade paraconsistente. A conclusão acima é válida no escopo desse critério e não deve ser lida como "*autoencoders* produzem pior autenticação", afirmação que este experimento não testou.

Há ainda um agravante interpretativo. Os valores de $\bar{\beta}$ dos *autoencoders* são altíssimos — acima de 0,97 nas 24 configurações, e acima de 0,99 em metade delas — o que indica sobreposição quase total entre classes, isto é, latentes próximos do colapso discutido na subseção 2.1.5.1. O que a Fase 00 mediu pode ser menos "*autoencoders* são piores extratores" e mais "estes *autoencoders*, neste regime de treino, não aprenderam representações discriminativas". Distinguir as duas hipóteses exigiria variar orçamento de treino e capacidade, o que não foi feito.

Há um teste mais barato, e ele já faz parte do desenho: a PCA, incluída como extrator linear de referência (seção 3.2). Ela não tem regime de treino, de modo que o resultado dela não pode ser atribuído a um treino insuficiente. Se o latente da PCA, na mesma dimensão, também apresentar $\bar{\beta}$ próximo de 1, a sobreposição é da representação de dimensão k desses sinais; se a PCA separar as classes, a falha é do treino dos *autoencoders*.

[PENDENTE] *A PCA já está no pipeline da Fase 00, mas ainda não foi executada; este teste aguarda esses resultados.*

Questão 4 — O desempenho é afetado pela dependência de texto?

Não de forma perceptível. Sobre as 16 configurações de cada nível, o EER médio é 0,5923 no modo dependente de texto e 0,5913 no independente — uma diferença de 0,001, muito menor que o desvio padrão entre repetições de uma única configuração. As melhores configurações de cada modo também praticamente empatam (0,4534 contra 0,4651).

Esse resultado precisa ser lido com cautela, e a cautela é a mesma que atravessa toda a Fase 01: com o desempenho global tão próximo do acaso (Capítulo 4), a ausência de diferença entre os dois modos é pouco informativa. Dois classificadores que não discriminam não se distinguem entre si. A questão permanece, na prática, em aberto: só faria sentido reavaliá-la sobre um sistema que primeiro demonstrasse desempenho de

autenticação acima do acaso.

5 Conclusões e Trabalhos Futuros

5.1 Síntese dos resultados

Esta tese investigou a autenticação biométrica combinando fala fonada e fala imaginada, comparando extração manual de características (DTWPT com agrupamento energético) e extração automatizada (*autoencoders*), com seleção conduzida pela engenharia paraconsistente de características e classificação por uma rede neural de pulso residual profunda (DSNN). O experimento foi organizado em duas fases: 208 execuções de *ranking* na Fase 00 e 32 execuções de autenticação na Fase 01, cada uma repetida três vezes.

A extração manual venceu a seleção paraconsistente para os dois sinais, com a mesma configuração — *wavelet* Haar, escala linear (LFCC), Categoria 1 — em voz e EEG. Nenhuma das 24 variantes de *autoencoder* se aproximou do melhor extrator manual (seção 4.3). A convergência dos dois sinais sobre a mesma configuração é, em si, um resultado: a *wavelet* mais simples do conjunto avaliado, com o particionamento espectral mais simples, superou 22 famílias *Daubechies* e dois esquemas cocleares.

O desempenho de autenticação, porém, permaneceu próximo do acaso. A melhor das 32 configurações da Fase 01 — fusão precoce, dependente de texto, k-fold plano, características brutas — atingiu EER = 0,4534 e AUC = 0,5452, contra 0,5 e 0,5 de um classificador aleatório. Nenhuma configuração ficou substancialmente abaixo desse patamar. Trata-se, portanto, de um **resultado negativo**: nas condições avaliadas, o sistema proposto não sustenta autenticação prática, e as conclusões desta tese devem ser lidas sob essa limitação central.

Registre-se, ainda assim, o que o experimento indicou dentro desse regime: o EEG e as fusões superaram a voz isolada entre as configurações mais bem colocadas, o que é consistente com a motivação original de complementar a fala degradada com sinal cerebral; e a padronização das características piorou sistematicamente o EER enquanto elevava o AUC, divergência que sugere um problema de calibração de limiar e não de poder discriminativo.

5.2 Contribuições

Além dos resultados experimentais, três contribuições de natureza metodológica emergiram do trabalho, todas motivadas por defeitos detectados durante a própria execução:

1. **A métrica $D_{\textbf{penalizado}}$** (subseção 2.1.5.1). O grau de verdade $D_{1,0}$, usado isoladamente, premia extratores degenerados: uma rede que emite o mesmo vetor para toda entrada atinge $\alpha = \beta = 1$ e pontua melhor que extratores genuinamente separáveis. A correção proposta soma um termo $\lambda|G_2|$ proporcional ao grau de contradição, com $\lambda = 2 - \sqrt{2}$ escolhido de modo que os três vértices degenerados sejam penalizados igualmente. A métrica é função apenas de α e β, e pôde ser aplicada retroativamente sem reexecutar experimento algum.

2. **O diagnóstico do piso de ruído das codificações temporais** (subseção 3.2.1). A ordem observada entre as codificações — `direct` $\succ$ `latency` $\succ$ `poisson` — coincide exatamente com a ordem prevista pelo ruído que cada codificação injeta *antes* de qualquer aprendizado, calculável em fechado ($\sigma_{\text{poisson}} = 0{,}125$ para $T = 16$). Como α é definido a partir da amplitude intraclasse, ele é estruturalmente incapaz de separar "o codificador não aprendeu" de "esta codificação tem piso estatístico irredutível". A conclusão metodológica é que a comparação entre codificações temporais sob este critério exige controlar T, e não apenas medir.

3. **A aplicação da engenharia paraconsistente à seleção de extratores**, com um protocolo em duas fases que separa a escolha da representação da medição de desempenho, evitando que a seleção do extrator seja contaminada pela capacidade do classificador.

5.3 Limitações

A limitação dominante é o **tamanho da base**: 15 locutores e 1974 amostras. Sob protocolo de verificação, com partições disjuntas por locutor, cada dobra treina com um número de identidades muito aquém do usual em biometria de voz, e o risco de o classificador não dispor de sinal suficiente para generalizar é alto — hipótese compatível com o desempenho próximo do acaso observado.

Três limitações de desenho devem acompanhar qualquer leitura dos resultados:

- as três categorias de representação **não** foram comparadas através de um classificador comum, apenas pelo critério paraconsistente, de modo que a Questão 3 não foi respondida na forma em que fora originalmente formulada (seção 4.3);
- os *autoencoders* apresentaram $\bar{\beta}$ acima de 0,97 em todas as configurações, indicando latentes próximos do colapso; não se pode distinguir, com os dados disponíveis, "pior extrator" de "treino insuficiente";
- a Questão 4 (dependência de texto) foi avaliada sobre um sistema que opera no nível do acaso, o que torna a ausência de diferença entre os modos pouco informativa.

5.4 Trabalhos futuros

O encaminhamento mais direto decorre das limitações acima:

1. **Ampliar a base**, em número de locutores antes que em amostras por locutor: sob protocolo de verificação, é a contagem de identidades que limita o que o classificador pode aprender a discriminar.

2. **Testar o piso de ruído em $T = 64$.** Como o piso decresce com $1/T$, quadruplicar o número de passos reduz σ_{poisson} a 0,0625. Se a desvantagem da codificação `poisson` encolher proporcionalmente, ela era artefato de medição; se persistir, é propriedade da codificação. O teste decide uma questão que a Fase 00 deixou explicitamente em aberto.

3. **Comparar as três categorias através do classificador**, estendendo a Fase 01 para além da combinação vencedora, de modo a responder à Questão 3 na forma originalmente pretendida.

4. **Investigar a divergência EER–AUC sob padronização**, verificando a hipótese de calibração de limiar — se confirmada, a padronização passaria de prejudicial a benéfica mediante ajuste do limiar de decisão.

5. **Variar orçamento de treino e capacidade dos *autoencoders***, para separar "pior extrator" de "treino insuficiente" e permitir uma comparação justa com a extração manual.

Referências

ABDULGHANI, M. M.; WALTERS, W. L.; ABED, K. H. Imagined speech classification using eeg and deep learning. *Bioengineering*, v. 10, n. 6, 2023. ISSN 2306-5354. Disponível em: <https://www.mdpi.com/2306-5354/10/6/649>.

ABDULGHANI, M. M.; WALTERS, W. L.; ABED, K. H. Imagined speech classification using eeg and deep learning. *Bioengineering (Basel)*, MDPI AG, Switzerland, v. 10, n. 6, p. 649–, 2023. ISSN 2306-5354.

ADDISON, P. S.; WALKER, J.; GUIDO, R. C. Time–frequency analysis of biosignals. *IEEE Engineering in Medicine and Biology Magazine*, v. 28, n. 5, p. 14–29, Sep 2009.

AGARWAL, P.; KUMAR, S. Electroencephalography based imagined alphabets classification using spatial and time-domain features. *International journal of imaging systems and technology*, John Wiley and Sons, Inc, Hoboken, USA, v. 32, n. 1, p. 111–122, 2022. ISSN 0899-9457.

ALI, Y. M.; NOORSAL, E.; MOKHTAR, N. F.; SAAD, S. Z. M.; ABDULLAH, M. H.; CHIN, L. C. Speech-based gender recognition using linear prediction and mel-frequency cepstral coefficients. *Indonesian J Electric Eng Comput Sci*, v. 28, n. 2, p. 753–761, 2022.

BENGIO, Y.; COURVILLE, A.; VINCENT, P. *Representation Learning: A Review and New Perspectives*. 2014.

BERANEK, L. L. *Acoustic Measurements*. [S.l.]: J. Wiley, 1949.

BIANCHI, G. *Electronic Filter Simulation & Design*. [S.l.]: McGraw-Hill Education, 2007. ISBN 9780071712620.

BIDGOLY, A. J.; BIDGOLY, H. J.; AREZOUMAND, Z. A survey on methods and challenges in eeg based authentication. *Computers and Security*, v. 93, p. 101788, 2020. ISSN 0167-4048. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167404820300730>.

BOSI, M.; GOLDBERG, R. E. *Introduction to digital audio coding and standards*. [S.l.]: Springer Science & Business Media, 2002. v. 721.

BUTTERWORTH, S. On the theory of filters amplifiers. 1930.

COMŞA, I.-M.; VERSARI, L.; FISCHBACHER, T.; ALAKUIJALA, J. Spiking autoencoders with temporal coding. *Frontiers in Neuroscience*, v. 15, p. 712667, 2021. Finite-penalty-at-T convention for a never-firing unit under time-to-first-spike loss.

CORETTO, G. A. P.; GAREIS, I. E.; RUFINER, H. L. Open access database of EEG signals recorded during imagined speech. In: ROMERO, E.; LEPORE, N.; BRIEVA, J.; BRIEVA, J.; AND, I. L. (Ed.). *12th International Symposium on Medical Information Processing and Analysis*. SPIE, 2017. v. 10160, p. 1016002. Disponível em: <https://doi.org/10.1117/12.2255697>.

COSTA, N. C. d.; ABE, J. M. Paraconsistência em informática e inteligência artificial. *Estudos Avançados*, scielo, v. 14, p. 161 – 174, 08 2000.

COSTA, N. da; BÉZIAU, J.; BUENO, O. *Elementos de teoria paraconsistente de conjuntos*. Centro de Lógica, Epistemologia e História da Ciência, UNICAMP, 1998. (Coleção CLE). Disponível em: <https://books.google.com.br/books?id=MGCKGwAACAAJ>.

DASALLA, C. S.; KAMBARA, H.; SATO, M.; KOIKE, Y. Single-trial classification of vowel speech imagery using common spatial patterns. *Neural networks : the official journal of the International Neural Network Society*, Elsevier, v. 22, n. 9, p. 1334–1339, 2009.

DAUBECHIES, I. *Ten Lectures on Wavelets*. [S.l.]: Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104), 1992. (CBMS-NSF Regional Conference Series in Applied Mathematics).

DEWITT, I.; RAUSCHECKER, J. P. Wernicke's area revisited: parallel streams and word processing. *Brain and Language*, Netherlands, v. 127, p. 181–191, 2013. ISSN 0093-934X. Copyright © 2012 Elsevier Inc. All rights reserved.

DUAN, C.; DING, J.; CHEN, S.; YU, Z.; HUANG, T. Temporal effective batch normalization in spiking neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2022.

ESHRAGHIAN, J. K.; WARD, M.; NEFTCI, E. O.; WANG, X.; LENZ, G.; DWIVEDI, G.; BENNAMOUN, M.; JEONG, D. S.; LU, W. D. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, v. 111, n. 9, p. 1016–1054, 2023.

FAN, Y.; KANG, J.; LI, L.; LI, K.; CHEN, H.; CHENG, S.; ZHANG, P.; ZHOU, Z.; CAI, Y.; WANG, D. Cn-celeb: a challenging chinese speaker recognition dataset. In: IEEE. *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. [S.l.], 2020. p. 7604–7608.

FANT, G. *Acoustic Theory of Speech Production*: With calculations based on x-ray studies of russian articulations. 2. ed. The Hague: Mouton, 1960.

FLINKER, A.; KORZENIEWSKA, A.; SHESTYUK, A. Y.; FRANASZCZUK, P. J.; DRONKERS, N. F.; KNIGHT, R. T.; CRONE, N. E. Redefining the role of broca's area in speech. *Proceedings of the National Academy of Sciences*, v. 112, n. 9, p. 2871–2875, 2015. Disponível em: <https://www.pnas.org/doi/abs/10.1073/pnas.1414491112>.

FREITAS, S. Avaliação acústica e áudio percetiva na caracterização da voz humana. Faculdade de Engenharia da Universidade do Porto, 2013.

FURLAN, A. *Caracterização de voice spoofing para fins de verificação de locutores com base na Transformada Wavelet e na Análise Paraconsistente de Características*. Dissertação (Mestrado) — Universidade Estadual Paulista - campus de São José do Rio Preto-SP, São José do Rio Preto, Brasil, 2021.

FURLAN, A. What does spiking cost an autoencoder? A group-disjoint comparison with a non-spiking control on speech and EEG. Artigo em revisão. Dados, perfis e *scripts* em `software/nn/results/meeting01` e `software/nn/scripts/pipeline/meeting01`. 2026.

GAO, S.-H.; CHENG, M.-M.; ZHAO, K.; ZHANG, X.-Y.; YANG, M.-H.; TORR, P. Res2net: A new multi-scale backbone architecture. *IEEE TPAMI*, 2021.

GAO, Y.; JIN, Y.; CHAUHAN, J.; CHOI, S.; LI, J.; JIN, Z. Voice in ear: Spoofing-resistant and passphrase-independent body sound authentication. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, Association for Computing Machinery, New York, NY, USA, v. 5, n. 1, mar 2021. Disponível em: <https://doi.org/10.1145/3448113>.

GERSTNER, W.; KISTLER, W.; NAUD, R.; PANINSKI, L. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014. ISBN 9781107060838. Disponível em: <https://neuronaldynamics.epfl.ch/online/Ch2.S2.html>.

GLOROT, X.; BENGIO, Y. Understanding the difficulty of training deep feedforward neural networks. In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. [S.l.: s.n.], 2010. p. 249–256. Xavier/Glorot weight initialization.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>.

GOODMAN, D.; FIERS, T.; GAO, R.; GHOSH, M.; PEREZ, N. *Spiking Neural Network Models in Neuroscience - Cosyne Tutorial 2022*. Zenodo, 2022. Disponível em: <https://doi.org/10.5281/zenodo.7044500>.

GROTHER, P.; NGAN, M.; HANAOKA, K. NIST Interagency/Internal Report, *Face Recognition Vendor Test (FRVT) Part 3: Demographic Effects*. 2019. Disponível em: <https://doi.org/10.6028/NIST.IR.8280>.

GUIDO, R. C. Practical and useful tips on discrete wavelet transforms [sp tips tricks]. *IEEE Signal Processing Magazine*, v. 32, n. 3, p. 162–166, May 2015.

GUIDO, R. C. Paraconsistent feature engineering [lecture notes]. *IEEE Signal Processing Magazine*, v. 36, n. 1, p. 154–158, Jan 2019.

GUO, Y.; ZHANG, Y.; CHEN, Y.; PENG, W.; LIU, X.; ZHANG, L.; HUANG, X.; MA, Z. Membrane potential batch normalization for spiking neural networks. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. [s.n.], 2023. Disponível em: <https://arxiv.org/abs/2308.08359>.

HANILÇI, C. Linear prediction residual features for automatic speaker verification anti-spoofing. *Multimedia Tools and Applications*, v. 77, n. 13, p. 16099–16111, Jul 2018.

HAYKIN, S.; MOHER, M. *Sistemas de Comunicação-5*. [S.l.]: Bookman Editora, 2011.

HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. Disponível em: <http://arxiv.org/abs/1512.03385>.

HE, K.; ZHANG, X.; REN, S.; SUN, J. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. [S.l.: s.n.], 2015. p. 1026–1034. Kaiming/He weight initialization.

HERNÁNDEZ-DEL-TORO, T.; REYES-GARCÍA, C. A.; PINEDA, L. Villaseñor. Toward asynchronous eeg-based bci: Detecting imagined words segments in continuous eeg signals. *arXiv.org*, Cornell University Library, arXiv.org, Ithaca, 2021. ISSN 2331-8422.

HILLENBRAND, J.; GETTY, L. A.; CLARK, M. J.; WHEELER, K. Acoustic characteristics of American English vowels. *The Journal of the Acoustical Society of America*, v. 97, n. 5, p. 3099–3111, 05 1995. ISSN 0001-4966. Disponível em: <https://doi.org/10.1121/1.411872>.

IOFFE, S.; SZEGEDY, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2015. p. 448–456.

JENSEN, A.; COUR-HARBO, A. la. *Ripples in Mathematics*. [S.l.]: Springer Berlin Heidelberg, 2001.

JIANG, P.; WANG, Q.; LIN, X.; ZHOU, M.; DING, W.; WANG, C.; SHEN, C.; LI, Q. Securing liveness detection for voice authentication via pop noises. *IEEE Transactions on Dependable and Secure Computing*, v. 20, n. 2, p. 1702–1718, 2023.

JOLLIFFE, I. *Principal Component Analysis*. Springer, 2002. (Springer Series in Statistics). ISBN 9780387954424. Disponível em: <https://books.google.com.br/books?id=_olByCrhjwIC>.

JONES, I. S.; KORDING, K. P. *Can Single Neurons Solve MNIST? The Computational Power of Biological Dendritic Trees*. 2020. Disponível em: <https://arxiv.org/abs/2009.01269>.

KASABOV, N. K. *Time-space, spiking neural networks and brain-inspired artificial intelligence*. [S.l.]: Springer, 2019.

KAUFMAN, S.; ROSSET, S.; PERLICH, C.; STITELMAN, O. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, v. 6, n. 4, p. 1–21, 2012.

KIM, Y.; PANDA, P. Revisiting batch normalization for training low-latency deep spiking neural networks from scratch. *Frontiers in Neuroscience*, v. 15, 2021. Batch Normalization Through Time (BNTT).

KREMER, R. L.; GOMES, M. d. C. A eficiência do disfarce em vozes femininas: uma análise da frequência fundamental. *ReVEL*, v. 12, p. 23, 2014.

KROGH, A.; HERTZ, J. A. A simple weight decay can improve generalization. In: *Advances in Neural Information Processing Systems 4 (NIPS)*. [S.l.]: Morgan Kaufmann, 1991. p. 950–957.

LECUN, Y.; BOTTOU, L.; ORR, G. B.; MÜLLER, K.-R. Efficient backprop. In: *Neural Networks: Tricks of the Trade*. [S.l.]: Springer, 1998, (Lecture Notes in Computer Science, v. 1524). p. 9–50.

LOSHCHILOV, I.; HUTTER, F. Decoupled weight decay regularization. In: *Proceedings of the 7th International Conference on Learning Representations (ICLR)*. [s.n.], 2019. AdamW; source of the decoupled weight decay update rule. Disponível em: <https://openreview.net/forum?id=Bkg6RiCqY7>.

LOTTE, F.; BOUGRAIN, L.; CICHOCKI, A.; CLERC, M.; CONGEDO, M.; RAKOTOMAMONJY, A.; YGER, F. A review of classification algorithms for eeg-based brain-computer interfaces: A 10-year update. *Journal of Neural Engineering*, v. 15, n. 3, p. 031005, 2018.

MAASS, W.; NATSCHLäGER, T.; MARKRAM, H. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, v. 14, n. 11, p. 2531–2560, 2002.

MAHAPATRA, N. C.; BHUYAN, P. Multiclass classification of imagined speech vowels and words of electroencephalography signals using deep learning. *Advances in human-computer interaction*, Hindawi, New York, v. 2022, p. 1–10, 2022. ISSN 1687-5893.

MAHAPATRA, N. C.; BHUYAN, P. Eeg-based classification of imagined digits using a recurrent neural network. *Journal of neural engineering*, IOP Publishing, England, v. 20, n. 2, p. 26040–, 2023. ISSN 1741-2560.

MANNA, D. L.; VICENTE-SOLA, A.; KIRKLAND, P.; BIHL, T. J.; CATERINA, G. D. Time series forecasting via derivative spike encoding and bespoke loss functions for spiking neural networks. *Computers*, v. 13, n. 8, p. 202, 2024.

MASTRANGELO BARBARA SCELSAM, F. P. M. Normal awake adult eeg. In: MECARELLI, O. (Ed.). *Clinical Electroencephalography*. [S.l.]: Springer, 2019. cap. 11. ISBN 9783030045722.

MECARELLI, O. Normal awake adult eeg. In: MECARELLI, O. (Ed.). *Clinical Electroencephalography*. [S.l.]: Springer, 2019. cap. 9. ISBN 9783030045722.

MENG, Z.; ALTAF, M. U. B.; JUANG, B.-H. F. Active voice authentication. *DIGITAL SIGNAL PROCESSING*, v. 101, JUN 2020. ISSN 1051-2004.

MOCTEZUMA, L. A.; TORRES-GARCÍA, A. A.; PINEDA, L. V. nor; CARRILLO, M. Subjects identification using eeg-recorded imagined speech. *Expert Systems with Applications*, v. 118, p. 201–208, 2019. ISSN 0957-4174. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0957417418306468>.

NG, A. *Sparse Autoencoder*. 2011. <https://web.stanford.edu/class/cs294a/sparseAutoencoder.pdf>. CS294A Lecture Notes, Stanford University, pp. 1–19.

NGUYEN, C. H.; KARAVAS, G. K.; ARTEMIADIS, P. Inferring imagined speech using eeg signals: a new approach using riemannian manifold features. *Journal of neural engineering*, v. 15, n. 1, p. 016002, 2018. Disponível em: <https://doi.org/10.1088/1741-2552/aa8235>.

PANACHAKEL, J.; RAMAKRISHNAN, A. Decoding imagined speech from eeg using transfer learning. *IEEE Access*, PP, 10 2021.

PANACHAKEL, J. T.; RAMAKRISHNAN, A.; ANANTHAPADMANABHA, T. Decoding imagined speech using wavelet features and deep neural networks. In: *2019 IEEE 16th India Council International Conference (INDICON)*. IEEE, 2019. Disponível em: <http://dx.doi.org/10.1109/INDICON47234.2019.9028925>.

PANACHAKEL, J. T.; RAMAKRISHNAN, A. G.; ANANTHAPADMANABHA, T. V. *A Novel Deep Learning Architecture for Decoding Imagined Speech from EEG*. 2020.

PARK, H.-j.; LEE, B. Multiclass classification of imagined speech eeg using noise-assisted multivariate empirical mode decomposition and multireceptive field convolutional neural network. *Frontiers in human neuroscience*, Frontiers Media S.A, v. 17, p. 1186594–1186594, 2023. ISSN 1662-5161.

PINTO, F. C. G. *Manual de Iniciação em Neurocirurgia*. 2. ed. São Paulo: Santos, 2012. 384 p. ISBN 978-85-7288-979-7.

POLIKAR, R. *et al. The wavelet tutorial*. 1996.

RABINER, L. R.; JUANG, B.-H. *Fundamentals of Speech Recognition*. [S.l.]: Prentice Hall, 1993. (Prentice Hall Signal Processing Series).

SALOMON, D.; MOTTA, G.; BRYANT, D. *Data Compression: The Complete Reference*. Springer London, 2007. (Molecular biology intelligence unit). ISBN 9781846286032. Disponível em: <https://books.google.com.br/books?id=ujnQogzx_2EC>.

SANEI, S.; CHAMBERS, J. A. *EEG Signal Processing and Machine Learning*. 2. ed. [S.l.]: John Wiley & Sons, 2021. ISBN 9781119386957.

SEN, O.; SHEEHAN, A. M.; RAMAN, P. R.; KHARA, K. S.; KHALIFA, A.; CHATTERJEE, B. Machine-learning methods for speech and handwriting detection using neural signals: A review. *Sensors*, v. 23, n. 12, 2023. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/23/12/5575>.

SHAH, U.; ALZUBAIDI, M.; MOHSEN, F.; ABD-ALRAZAQ, A.; ALAM, T.; HOUSEH, M. The role of artificial intelligence in decoding speech from eeg signals: A scoping review. *Sensors (Basel, Switzerland)*, MDPI AG, Basel, v. 22, n. 18, p. 6975–, 2022. ISSN 1424-8220.

TAMM, M.-O.; MUHAMMAD, Y.; MUHAMMAD, N. Classification of vowels from imagined speech with convolutional neural networks. *Computers*, MDPI, v. 9, n. 2, p. 46, 2020.

VALENÇA, E. H. O. *et al.* Análise acústica dos formantes em indivíduos com deficiência isolada do hormônio do crescimento. Universidade Federal de Sergipe, 2014.

VANDERAH, T. W. *Nolte's The Human Brain in Photographs and Diagrams*. 5th. ed. [S.l.: s.n.], 2020.

VASKOVIĆ ALEXANDRA OSIKA, N. M. J. *Neuroanatomia*. 2024. Disponível em: <https://www.kenhub.com/pt/library/anatomia/neuroanatomia>.

VICENTE, E. *Sistema 10-20 para localizar alvos terapêuticos em EMT*. 2023. Disponível em: <https://www.kandel.com.br/post/como-localizar-os-pontos-de-estimulacao-para-estimulacao-magnetica-transcraniana>.

VIIKKI, O.; LAURILA, K. Cepstral domain segmental feature vector normalization for noise robust speech recognition. *Speech Communication*, v. 25, n. 1–3, p. 133–147, 1998. Foundation of cepstral mean and variance normalization (CMVN).

VIVANCOS, D. *MindBigData*. 2023. Disponível em: <https://mindbigdata.com/opendb/>.

WANG, Q.; WU, B.; ZHU, P.; LI, P.; ZUO, W.; HU, Q. Eca-net: Efficient channel attention for deep convolutional neural networks. *CoRR*, abs/1910.03151, 2019. Disponível em: <http://arxiv.org/abs/1910.03151>.

WANG, S.; ZHANG, H.; ZHANG, X.; SU, Y.; WANG, Z. Voiceprint recognition under cross-scenario conditions using perceptual wavelet packet entropy-guided efficient-channel-attention–res2net–time-delay-neural-network model. *Mathematics*, v. 11, n. 19, 2023. ISSN 2227-7390. Disponível em: <https://www.mdpi.com/2227-7390/11/19/4205>.

WERTZNER, H. F.; SCHREIBER, S.; AMARO, L. Análise da freqüência fundamental, jitter, shimmer e intensidade vocal em crianças com transtorno fonológico. *Revista Brasileira de Otorrinolaringologia*, SciELO Brasil, v. 71, p. 582–588, 2005.

ZHAO, S.; RUDZICZ, F. Classifying phonological categories in imagined and articulated speech. In: *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. [S.l.: s.n.], 2015. p. 992–996.

ZHENG, H.; WU, Y.; DENG, L.; HU, Y.; LI, G. Going deeper with directly-trained larger spiking neural networks. In: *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI)*. [s.n.], 2021. p. 11062–11070. Threshold-Dependent Batch Normalization (tdBN). Disponível em: <https://arxiv.org/abs/2011.05280>.

ZHOU, X.; GARCIA-ROMERO, D.; DURAISWAMI, R.; ESPY-WILSON, C.; SHAMMA, S. Linear versus mel frequency cepstral coefficients for speaker recognition. In: *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. [S.l.: s.n.], 2011. p. 559–564.

ZWICKER, E. Subdivision of the Audible Frequency Range into Critical Bands (Frequenzgruppen). *The Journal of the Acoustical Society of America*, v. 33, n. 2, p. 248–248, 02 1961. ISSN 0001-4966. Disponível em: <https://doi.org/10.1121/1.1908630>.

APÊNDICE A – Regularização em redes neurais

A.1 Decaimento de Peso (Regularização L2)

Suponha que tenhamos uma rede neural simples com pesos W. Sejam L_{dados} a perda de dados (por exemplo, erro quadrático médio para tarefas de regressão), λ o parâmetro de regularização e $\|W\|_2^2$ a norma L2 dos pesos. Durante o treinamento, adicionamos um termo de regularização à função de perda, que penaliza o quadrado dos pesos (KROGH; HERTZ, 1991):

$$L_{\text{total}} = L_{\text{dados}} + \lambda \|W\|_2^2 \tag{A.1}$$

Vamos supor que nossa matriz de pesos W seja:

$$W = \begin{bmatrix} 1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

Se definirmos $\lambda = 0.1$, o termo de regularização seria $0.1 \times (1^2 + 0.5^2 + (-0.3)^2 + 0.8^2) = 0.1 \times (1 + 0.25 + 0.09 + 0.64) = 0.1 \times 1.98 = 0.198$. Durante o treinamento, o otimizador atualiza os pesos considerando tanto a perda de dados quanto o termo de regularização. O termo de regularização incentiva pesos menores, prevenindo o sobreajuste.

O exemplo acima soma a penalidade diretamente à perda, que é a forma clássica da regularização L2. Neste trabalho usa-se uma forma equivalente, porém mais estável quando combinada com o otimizador Adam: o **decaimento de peso desacoplado** (*decoupled weight decay*), proposto por (LOSHCHILOV; HUTTER, 2019) no algoritmo AdamW.

A ideia é simples: em vez de modificar o gradiente, encolhe-se cada peso por uma pequena fração a cada passo de treinamento, logo após a atualização normal do otimizador. Sejam θ um peso qualquer da rede, η a taxa de aprendizado e λ o parâmetro que controla a força do decaimento:

$$\theta \leftarrow \theta - \eta\, \lambda\, \theta, \tag{A.2}$$

Em outras palavras, a cada iteração o peso perde uma fração $\eta\lambda$ do seu valor. Por exemplo, com $\eta = 0{,}01$ e $\lambda = 0{,}1$, cada peso é multiplicado por $(1 - 0{,}001) = 0{,}999$ a cada passo, sendo lentamente puxado em direção a zero. Pesos realmente úteis recebem gradientes que compensam esse encolhimento e sobrevivem; pesos irrelevantes, que quase não recebem gradiente, definham até desaparecer.

Por que separar o encolhimento do gradiente? O Adam divide cada gradiente por uma estimativa da sua própria escala (o fator $1/\sqrt{\hat{v}}$). Se a penalidade fosse somada ao

gradiente, ela também seria dividida por esse fator e o decaimento ficaria mais forte para alguns pesos e mais fraco para outros. Mantendo o encolhimento à parte, todos os pesos diminuem na mesma proporção.

Um cuidado importante: o decaimento é aplicado **apenas às matrizes de peso** das camadas densas. Os vieses (*bias*) e os parâmetros biofísicos dos neurônios de pulso — resistência R, capacitância C e limiar de disparo V_{th} — ficam de fora. O motivo é direto: esses valores não são "pesos" a serem reduzidos. Empurrar R ou C para zero destruiria a constante de tempo da membrana $\tau = R \cdot C$, e empurrar V_{th} para zero faria o neurônio disparar o tempo todo.

A.2 Penalidade de Esparsidade

Vamos considerar um autoencoder esparsificado com uma penalidade de esparsidade adicionada à função de perda para encorajar a esparsidade nas ativações das unidades ocultas.

Suponha que tenhamos uma camada oculta com valores de ativação $h = [0.9, 0.1, 0.8, 0.05]$.

Segundo (NG, 2011), o termo de penalidade de esparsidade pode ser definido como a divergência Kullback-Leibler (KL) entre o nível de esparsidade desejado ρ e $\hat{\rho}_i$, a média da ativação da unidade oculta i, somada sobre todas as unidades ocultas:

$$\Omega(h) = \sum_i (\rho \log(\rho/\hat{\rho}) + (1-\rho) \log((1-\rho)/(1-\hat{\rho}))) \tag{A.3}$$

Vamos supor $\rho = 0.2$. Se a média da ativação da unidade oculta i for $\hat{\rho}_i = 0.5$, então a penalidade de esparsidade para essa unidade seria:

$$\begin{aligned} \Omega(h_i) &= (0.2 \log(0.2/0.5) + 0.8 \log(0.8/0.5)) \\ &= (0.2 \times -0.3219 + 0.8 \times 0.3219) \\ &= -0.0644 + 0.2575 \\ &= 0.1931 \end{aligned} \tag{A.4}$$

Durante o treinamento, a penalidade de esparsidade incentiva a rede a ter ativações esparsas, focando em características importantes dos dados.

A.3 Regularização da Taxa de Disparo em Redes Neurais de Pulso

A penalidade de taxa de disparo descrita nesta seção é uma técnica **proposta neste trabalho** para o treinamento das RNPs do Capítulo 2 — não é uma reprodução

de uma técnica publicada, mas uma adaptação da ideia geral de penalidade de esparsidade (seção anterior) ao caráter binário e temporal dos neurônios de pulso.

Diferentemente de um neurônio clássico, que produz um número real, um neurônio de pulso só pode emitir 0 ou 1 a cada instante. A informação que ele transmite está, portanto, na **frequência** com que dispara ao longo do tempo — a chamada *taxa de disparo*. Um detalhe simples ilustra o problema: se um neurônio quase nunca dispara, ou se dispara a toda hora, ele praticamente não distingue um exemplo de outro. Surgem assim dois extremos que prejudicam o aprendizado:

- **Neurônios mortos** (*dead neurons*): a taxa de disparo cai para perto de zero. O neurônio para de contribuir para a saída. Pior: como o gradiente substituto só existe nas proximidades do limiar, deixa de chegar gradiente ao neurônio, que nunca mais é ajustado e fica "preso" inativo.
- **Neurônios saturados** (*bursting neurons*): a taxa de disparo sobe para perto de um. O neurônio dispara em quase todo instante, responde igual a qualquer entrada (perde seletividade) e ainda gasta energia à toa — o oposto da economia que motiva o uso de RNP.

A solução é tratar a taxa de disparo como algo que deve ficar dentro de uma faixa saudável $[r_{\min}, r_{\max}]$ — nem baixa demais, nem alta demais. Seja r_ℓ a média dos pulsos (0 ou 1) da camada ℓ sobre todos os neurônios e instantes de tempo, e λ o parâmetro que regula a intensidade da penalidade. Adiciona-se à perda uma penalidade que só "acende" quando r_ℓ sai da faixa saudável, funcionando como duas cercas elásticas que a empurram de volta:

$$\Omega_{\text{disparo}} = \lambda \sum_{\ell \,\in\, \text{camadas}} \Big(\underbrace{\max(0,\ r_{\min} - r_\ell)^2}_{\text{pune taxa baixa}} + \underbrace{\max(0,\ r_\ell - r_{\max})^2}_{\text{pune taxa alta}} \Big), \tag{A.5}$$

Enquanto r_ℓ estiver dentro da faixa, os dois termos $\max(0, \cdot)$ valem zero e nada é penalizado; fora dela, a penalidade cresce com o quadrado da distância até a borda.

Um exemplo torna isso concreto. Sejam $r_{\min} = 0{,}05$, $r_{\max} = 0{,}80$ e $\lambda = 0{,}01$. Para uma camada com taxa média $r_\ell = 0{,}02$ (quase morta), apenas o primeiro termo atua:

$$\Omega = 0{,}01 \times (0{,}05 - 0{,}02)^2 = 0{,}01 \times 0{,}0009 = 9 \times 10^{-6}. \tag{A.6}$$

Já uma camada com $r_\ell = 0{,}40$, dentro da faixa, recebe penalidade nula. Sejam s_j o pulso (0 ou 1) do j-ésimo neurônio-instante da camada, n o número total desses elementos (n neurônios $\times$ passos de tempo) e $\text{clamp}(r_\ell, r_{\min}, r_{\max})$ a função que satura r_ℓ nos limites $r_{\min}$ e $r_{\max}$. O gradiente dessa penalidade em relação a cada pulso é

$$\frac{\partial \Omega_{\text{disparo}}}{\partial s_j} = \frac{2\lambda}{n} \big(r_\ell - \text{clamp}(r_\ell, r_{\min}, r_{\max}) \big), \tag{A.7}$$

Exemplo numérico. Continuando o caso $r_\ell = 0{,}02$ (quase morta) com $\lambda = 0{,}01$, para uma camada de $n = 20$ elementos (por exemplo, 4 neurônios $\times$ 5 passos de tempo): clamp(0,02, 0,05, 0,80) = 0,05 (satura no limite inferior), logo $\frac{\partial \Omega_{\text{disparo}}}{\partial s_j} = \frac{2 \times 0{,}01}{20}(0{,}02 - 0{,}05) = 0{,}001 \times (-0{,}03) = -3 \times 10^{-5}$. O gradiente negativo empurra s_j para cima durante a atualização (que subtrai o gradiente), ou seja, incentiva mais disparos — consistente com a camada estar abaixo de $r_{\min}$.

Esse gradiente é somado ao que já chega a cada camada pulsante durante a retropropagação: se a taxa está baixa, ele empurra os neurônios a disparar mais; se está alta, a disparar menos. Em resumo, esta é uma forma de regularização de esparsidade adaptada ao mundo temporal e binário das RNP, complementar à penalidade de esparsidade da seção anterior.

APÊNDICE B – Aplicação de um filtro *Wavelet*

Figura 46 – Demonstração numérica das transformadas Wavelet e *wavelet-packet*: Por razões didáticas, a *Wavelet* Haar foi considerada como tendo os valores $1/2$ e $-1/2$

***Packet-wavelet* para filtro haar = {0.5, 0.5}**

Sinal	32	10	20	38	37	28	38	34	18	24	24	9	23	24	28	34
	h								g							
Nível 01	21	29	32,5	36	21	16,5	23,5	31	11	-9	4,5	2	-3	7,5	-0,5	-3
	h				g				g				h			
Nível 02	25	34,25	18,75	27,25	-4	-1,75	2,25	-3,75	10	1,25	-5,25	1,25	1	3,25	2,25	-1,75
	h		g		g		h		h		g		g		h	
Nível 03	29,62	23	-4,625	-4,25	-1,125	3	-2,875	-0,75	5,625	-2	4,375	-3,25	-1,125	2	2,125	0,25
	h	g	g	h	h	g	g	h	h	g	g	h	h	g	g	h
Nível 04	26,3125	3,3125	-0,1875	-4,4375	0,9375	-2,0625	-1,0625	-1,8125	1,8125	3,8125	3,8125	0,5625	0,4375	-1,5625	0,9375	1,1875

← **resultado do passa-baixas (h)**
← **resultado do passa-altas (g)**

Comparação

Transformada wavelet regular

Sinal	32	10	20	38	37	28	38	34	18	24	24	9	23	24	28	34
Nível 01	21	29	32,5	36	21	16,5	23,5	31	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 02	25	34,25	18,75	27,25	-4	-1,75	2,25	-3,75	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 03	29,62	23	-4,625	-4,25	-4	-1,75	2,25	-3,75	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 04	26,3125	3,3125	-4,625	-4,25	-4	-1,75	2,25	-3,75	11	-9	4,5	2	-3	7,5	-0,5	-3

Transformada packet-wavelet

Sinal	32	10	20	38	37	28	38	34	18	24	24	9	23	24	28	34
Nível 01	21	29	32,5	36	21	16,5	23,5	31	11	-9	4,5	2	-3	7,5	-0,5	-3
Nível 02	25	34,25	18,75	27,25	-4	-1,75	2,25	-3,75	10	1,25	-5,25	1,25	1	3,25	2,25	-1,75
Nível 03	29,62	23	-4,625	-4,25	-1,125	3	-2,875	-0,75	5,625	-2	4,375	-3,25	-1,125	2	2,125	0,25
Nível 04	26,3125	3,3125	-0,1875	-4,4375	0,9375	-2,0625	-1,0625	-1,8125	1,8125	3,8125	3,8125	0,5625	0,4375	-1,5625	0,9375	1,1875

Fonte: Elaborado pelo autor, 2023.